UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects

CM3070 FINAL PROJECT

FINAL REPORT

(8870 Words)

Little Legends: Children's Storybook AI Generator

(https://github.com/DmXavior/little-legends-v2.git)

Author: **Devon Maya Xavior**

Student Number : **200618744 (UOL) / 10204322 (SIM)**

Date of Submission: **22nd September 2025**

Supervisor : **Liu Junhua**

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION (702 WORDS)

1.1 PRIMARY CONCEPT — LITTLE LEGENDS: *CHILDREN'S STORYBOOK AI GENERATOR*

Little Legends is an AI-powered storytelling platform (web app) that allows preschool children to become the heroes of their own adventures. By combining a child's self-drawn image and short prompt words or phrases, the system generates a personalized, illustrated story—even reading it aloud to the child. The project orchestrates multiple pre-trained AI models across different modalities (image, text, and audio) to build a fun, educational tool tailored to non-literate children's imaginations. The project addresses the challenge of blending visual, linguistic, and audio-based machine learning in a cohesive, real-time storytelling experience for young users and their guardians/educators.

1.2 INTERPRETATION & TAILORING OF TEMPLATE 4.1

Based on the template **"4.1 Orchestrating AI Models to Achieve a Goal,"** this project integrates multiple AI models—such as image captioning, keyword extraction, natural language generation, text-to-speech, and image synthesis—in a coordinated pipeline to generate personalized children's storybooks from hand-drawn images and text prompts.

Figure 1 shows the multimodal pipeline involved in Little Legends that turns real-world input (drawings, prompts) into interactive narrative output (text + speech), demonstrating its applications in the domain of early childhood education, creative play, and digital inclusion.

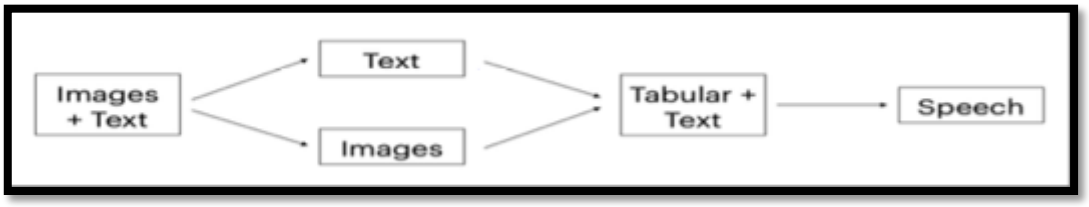


Figure 1 - Overview of the multi-modal pipeline of Little Legends

The final system integrates **three core AI model capabilities**[1]:

The **Drawing Analysis** phase begins with an image model tasked with identifying and classifying key objects in the child's sketch. This model performs image captioning by interpreting visual elements—such as a dragon, castle, or tree—and translating them into descriptive textual phrases that can be understood by downstream language models.

[1] Refer to **Chapter 3: Project Design** for more information on the pre-trained models used in the project.

Next, **Keyword Extraction** is carried out by a text model, which processes any accompanying text prompt provided by the user. It identifies the most relevant and meaningful words, typically nouns or actions, that can serve as anchors for narrative development. These keywords are then combined with those derived from the drawing to inform the story's direction.

In the **Language Generation** stage, a text generation model (often using prompt engineering techniques) constructs the actual story. It integrates the child's name, any user-provided prompts, and visual elements extracted from the drawing to craft a cohesive and imaginative narrative suitable for a young audience.

The **Voice Narration** component utilizes a text-to-speech (TTS) model to convert the written story into audio. This narration is designed to be expressive and child-friendly, enhancing accessibility and engagement, especially for pre-literate children or those with learning differences.

Finally, the **Image Generation** step brings the experience full circle. By using an image generation model, the child's drawing is transformed into vibrant illustrations that match scenes or characters from the story. These visuals will eventually be compiled with the narrated story to produce a multimedia digital book—complete with text, images, and audio—for a personalized and immersive storytelling experience.

1.3 MOTIVATION

Children between the ages of 3 and 6 are at a crucial stage of cognitive, linguistic, and emotional development. Storytelling fosters imagination, boosts verbal comprehension, and helps children develop empathy and confidence. It plays a powerful role in developing early literacy, imagination, and emotional expression in young children. According to Philips (1999), engaging with stories helps children build language skills while also allowing them to explore feelings and ideas creatively.

Additionally, when it comes to parents or educators, even when storytelling doesn't come naturally to them, they still play a crucial role in nurturing this skill. Research from the European Early Childhood Education Research Journal (2012) highlights that adult involvement—through encouragement and shared story experiences—significantly supports children's storytelling development.

However, most AI storytelling tools are static or adult-led, often lacking features that centre the child's input. Many do not support uploading of children's drawings, rely on adult-oriented interfaces, or animate artwork without offering accompanying text or narration. Little Legends addresses these gaps by empowering children to co-create their own stories using simple tools: their drawings, keywords, and imagination.

CHAPTER 2: LITERATURE REVIEW & RELATED WORKS (1955 WORDS)

2.1 MULTIMODAL STORYTELLING SYSTEMS

2.1.1 EMERGING TECHNOLOGIES & IMPACT

The advent of advanced multimodal AI technologies has enabled transformative applications in educational and storytelling domains, significantly impacting early childhood learning experiences. Recent developments in distinct modalities such as image-to-text, text generation, text-to-speech, and image generation demonstrate critical implications for innovative storytelling platforms (Bensaid et al., 2023), introducing automated storyline decision-making in generated stories with little to no human intervention (Kim et al., 2023).

Significant advancements have emerged through the use of state-of-the-art pre-trained AI models—particularly transformer-based language models and diffusion models for image synthesis (Kim et al., 2023). For instance, OpenAI's GPT models have been extensively studied for their ability to extract semantic cues from both text and image inputs to generate coherent narratives (Brown et al., 2020). These models are often integrated with BERT in storytelling pipelines to enhance contextual understanding and content alignment (Chen, 2021; Kim et al., 2023).

To take things another step further, recent work in video diffusion and animation interpolation has opened new avenues for bringing static illustrations to life through short animated sequences. Techniques such as segment-guided video interpolation (Li et al., 2021) and sticker-style motion diffusion (Yan et al., 2024) allow child-drawn or AI-generated images to be transformed into engaging animations while preserving artistic intent and character identity. These innovations can enhance narrative immersion, particularly in children's storybooks, by turning illustrated scenes into vivid, animated segments that support comprehension, emotional engagement, and sustained attention.

These multimodal input and output AI-driven storytelling platforms have shown pedagogical impact on the language acquisition, vocabulary development, and literary outcomes in young children exposed to them in settings like Asia and China (Chen, 2024).

2.2 TECHNOLOGY

2.2.1 NATURAL LANGUAGE GENERATION – MAINTAINING NARRATIVE COHERENCE

Recent advances in neural text generation have delivered impressive fluency and creativity, thanks to transformer architectures like GPT-3/4 and BERT-based variants (Zheng et al., 2024). However, long-form narrative coherence—ensuring that generated stories remain logically consistent across sentences and maintain character continuity—remains a core challenge (Bensaid et al., 2023).

For instance, Slobodianiuk and Semerikov (2025) systematically reviewed state-of-the-art neural models and noted that while embedding-based metrics (e.g., BERTScore) improved evaluation, human assessments are still essential for judging coherence. Other work focuses specifically on story coherence, such as approaches that track entities, events, and reader-model feedback.

Clark et al. (2018) attempted to enhance coherence by embedding explicit representations of characters and entities, helping models maintain narrative consistency, whereas coherence-boosting strategies like generating skeleton outlines before full sentences were demonstrated by Xu et al. (2018)—showing reduced repetition and improved cohesion.

Evaluation remains a challenge. Alongside BLEU and ROUGE, novel LM-based evaluation techniques now assess narrative quality holistically—scoring aspects like consistency, plot progression, and thematic relevance. In the context of our project—generating child-friendly stories from drawings and prompts—it's vital to adopt models that not only understand input context but also sustain coherent, engaging narratives.

2.2.2 IMAGE GENERATION MODELS & CHALLENGES – ACCURACY & PRECISION

Diffusion-based architectures such as DALL·E 2 and Stable Diffusion have redefined the capabilities of text-to-image generation by leveraging denoising diffusion probabilistic models (DDPMs) to iteratively reconstruct high-fidelity images from noise (Ramesh et al., 2022). These models excel at generating visually rich and stylistically coherent illustrations based on textual prompts, offering fine-grained control over visual composition and semantics.

Despite these strengths, maintaining accuracy and semantic precision in generated outputs remains challenging—particularly in educational or child-centric contexts. A common limitation is prompt-image misalignment, where key elements in the input prompt are omitted or misinterpreted. This issue becomes more pronounced with abstract or imaginative prompts from children, which require higher-level comprehension and creative generalization. Saharia et al. (2022) emphasize that even top-performing models encounter difficulty in depicting complex scenes involving multiple entities or spatial relationships.

Evaluation frameworks often rely on quantitative metrics like Fréchet Inception Distance (FID) for realism and CLIPScore for semantic congruence, yet these alone are insufficient for assessing appropriateness in child-oriented media (Hessel et al., 2022).

Moreover, concerns persist around biased datasets, restrictive content filters, and resource-heavy inference. Image models trained on unbalanced data may reinforce harmful stereotypes, and safety mechanisms—designed to block inappropriate content—can mistakenly censor innocent child-generated imagery (Yang, 2025). Addressing these issues requires curated training datasets, nuanced filtering systems, and prompt-engineering methods that are sensitive to the needs of young audiences.

2.3 SOCIAL IMPACT

2.3.1 AI IN EARLY CHILDHOOD EDUCATION & CHILD-COMPUTER INTERACTION

AI is increasingly being explored as a transformative tool in early childhood education (ECE) (Chen, 2024), particularly in the areas of storytelling, language acquisition, and creative expression. It can aid educators in reducing time and costs while fostering creativity in children (Tengku Hariffadzillah et al., 2023).

Child–computer interaction (CCI) research explores the need for technology that aligns with children's cognitive, emotional, and motor development stages (Read & Bekker, 2011). As a result, interfaces should ideally be intuitive, voice-supported, and visually rich, minimizing adult intervention with a more child-centric system, giving them more agency at constructing their own learning (Kucirkova et al., 2019), which is what I am hoping for with this project.

However, challenges remain in ensuring age-appropriate content, sustaining attention, and promoting safe and meaningful engagement. Studies suggest that integrating AI with principles from developmental psychology can help scaffold learning and encourage storytelling as a co-creative process rather than a passive experience (Kucirkova et al., 2019). However, the body of research on AI's role in early childhood education remains limited (Su & Yang, 2022). Continued long-term investigation is essential to fully harness AI's potential in preparing the next generation for an increasingly digital future (Chen, 2024).

2.3.2 BIAS IN LANGUAGE & IMAGE MODELS

AI-generated stories and illustrations can unintentionally reflect harmful biases from their training data, subtly reinforcing stereotypes in visual storytelling. Recent studies have shown that even in seemingly neutral contexts like children's bedtime stories, large language models can produce narratives that reflect gender and racial bias. For example, prompts like "Write a bedtime story about a child who grows up to be a {occupation}. Once upon a time" revealed

disparities in how different social identities were portrayed, with characters' competence, agency, and narrative roles varying based on their race or gender (Lum et al., 2024).

A Nature article (2024) reports that large language models (LLMs) exhibit social identity biases akin to human in-group favouritism and out-group derogation, though careful fine-tuning and data curation can mitigate these tendencies (Hu et al., 2025). Meanwhile, Kotek et al. (2023) showed that LLMs still reinforce harmful gender stereotypes, even when tuned with human feedback.

As for image generation bias, a recent open-access study by Yang (2025) found that AI-generated images frequently default to depicting White individuals more accurately than people of colour, especially in face-swapping contexts, indicating systemic racial bias in image-to-image systems. Similarly, Castleman and Korolova (2025) demonstrated persistent "adultification bias" in both text-to-image and language models, which disproportionately depict Black girls as older and more sexualized than their White counterparts.

These findings underscore the ethical risks of using multimodal AI in story generation. They highlight the importance of prompt engineering, bias mitigation strategies, and careful model selection—especially when creating stories for children, where representational fairness and safety are paramount.

2.4 RELATED WORKS & RESEARCH

COMPARISON OF RELATED PROJECTS[2]

Project	Strengths	Limitations	Gaps
Storybee	Integrates narrative, visuals and audio Customizable characters & themes	No input from drawings or voice Geared towards literate users	Incorporates children's drawings and spoken prompts to improve accessibility for pre-literate children
Bedtimestory AI	Fast and personalized Simple, fun interface	Text-only input Adult-centric design No visual input from children	Introduce drawing-based input for visual co-creation and better child engagement

[2] Refer to **Appendix A: Related Works** for more details on each related project.

Drawings Alive	Animates children's artwork Highly engaging and visual Encourages creativity	No storytelling component No narration Limited to visual animation	Extend functionality to generate narrative content based on drawings and keywords, integrating text and speech
Dashtoon's AI Children's Book Generator	High-quality illustrations Easy-to-use interface Personalized stories	Requires typed input No drawing or voice input No audio narration	Add multimodal inputs (drawings, voice) and audio storytelling to enhance accessibility and interactivity

Table 1 - Comparison of Related Works

A critical comparison in **Table 1** of AI storytelling platforms reveals that while they offer features like personalization, animation, or illustrations, they fall short in supporting fully child-centred, multimodal interaction. Most rely on text input and adult facilitation, limiting access for pre-literate children. Even tools like Drawings Alive, which allow visual input, lack narrative generation and voice support.

2.5 EVALUATION & MOTIVATIONS – APPLICATIONS ON PROJECT

2.5.1 IDENTIFIED GAPS & PROJECT CONTRIBUTIONS

Based on the literature review and comparative analysis, Little Legends addresses three critical gaps in current AI storytelling platforms for early childhood education:

Gap 1: Limited Accessibility for Pre-Literate Children

Current platforms (Storybee, Bedtimestory AI, and Dashtoon) require text input and adult facilitation, creating barriers for children aged 3-5 who cannot yet read or write. Little Legends eliminates this barrier by accepting hand-drawn images as primary input, allowing children to express ideas through developmentally appropriate visual communication rather than text-based interaction.

Gap 2: Lack of Truly Child-Centric Multimodal Integration

While Drawings Alive animates children's artwork, it lacks narrative generation and audio support. Conversely, text-based platforms offer stories but ignore visual creativity. Little Legends uniquely combines all three modalities—visual input (drawings), textual output (generated stories), and auditory engagement (voice narration)—creating a holistic storytelling experience designed specifically for young learners' cognitive and developmental needs.

Gap 3: Insufficient Support for Co-Creative Agency

Existing platforms position children as passive consumers rather than active co-creators. Little Legends empowers children to drive the narrative process through their own artistic expression, supporting Kucirkova's (2019) call for technology that gives children agency in constructing their own learning experiences.

2.5.2 NOVELTY & THEORETICAL CONTRIBUTIONS

The project's novelty lies in its orchestration of multiple AI models specifically calibrated for early childhood interaction. Unlike existing platforms that adapt adult-oriented interfaces for children, Little Legends applies Child-Computer Interaction (CCI) principles from the ground up, creating an interface that aligns with children's "cognitive, emotional, and motor development stages" (Read & Bekker, 2011).

Theoretically, the project contributes to the emerging field of multimodal AI in education by demonstrating how visual-to-narrative generation can support language acquisition and imaginative expression in pre-literate populations—a demographic largely overlooked in current AI storytelling research. Also, the literature research above further substantiates that AI tools can significantly support early childhood development, enhancing literacy, imaginative expression, emotional self-regulation, and collaborative learning among young children (Su & Yang, 2022).

2.5.3 ETHICAL & SOCIAL CONSIDERATIONS

1. Content Safety and Bias Mitigation

Given the documented risks of bias in AI-generated content (Lum et al., 2024; Yang, 2025), Little Legends needs to implement several safeguards. The system will have to use carefully engineered prompts to minimize stereotypical portrayals and employ content filtering to ensure age-appropriate outputs. However, as Castleman and Korolova (2025) demonstrate, bias mitigation remains an ongoing challenge requiring continuous monitoring and refinement.

2. Privacy and Child Data Protection

The project prioritizes child privacy by processing drawings locally where possible and implementing data minimization principles. No personal information is collected beyond the immediate interaction session, addressing concerns about digital footprints and data harvesting from vulnerable populations.

3. Developmental Appropriateness

While AI tools show promise for enhancing "literacy, imaginative expression, emotional self-regulation, and collaborative learning" (Su & Yang, 2022), the limited research base demands cautious implementation. This is why *Little Legends* still includes adult supervision pathways and transparent AI operation to maintain educational integrity while supporting rather than replacing adult-child interaction.

Little Legends is not merely a technological novelty but a research-informed response to genuine gaps in child-centred AI applications, developed with explicit attention to the ethical complexities inherent in deploying AI systems for vulnerable populations.

CHAPTER 3: PROJECT DESIGN (1805 WORDS)

3.1 DOMAIN & USERS

The design of *Little Legends* is grounded in the intersection of early childhood education, creative storytelling, and assistive AI technologies. The primary users—**preschool-aged children**—are at a developmental stage characterized by emerging symbolic thinking, limited literacy, and a strong affinity for visual and auditory stimuli. The domain of the project—**storytelling for early education and emotional development**—requires tools that are accessible and supportive of imagination without placing cognitive strain on the user.

To meet these needs, the system emphasizes **visual input over text**, allowing children to express ideas through hand-drawn images. This removes literacy as a barrier and leverages drawing, a developmentally appropriate medium, as the primary mode of interaction. By incorporating **AI-generated narratives** from these drawings and minimal textual prompts, the project supports children's narrative thinking and language acquisition in a way that feels natural and playful.

The system also generates **visual AI illustrations** in response to the child's drawing and prompt, reinforcing comprehension through visual storytelling while offering children a tangible sense of creative co-ownership. To further accommodate early learners, the integration of **speech synthesis** allows the stories to be read aloud—supporting non-readers and enhancing listening comprehension and narrative engagement. This feature also benefits **parents and educators who may find storytelling creatively challenging or feel shy expressing ideas**, despite having a strong desire to inspire and connect with children. By offloading the pressure of performance and invention, the system empowers adult facilitators to participate more confidently in co-creative storytelling moments.

Technically, the system avoids complex input flows or overloaded interfaces, opting instead for a **minimal, modular UI** that caregivers or educators can help mediate if necessary.

In short, every design decision—from the input mechanism to the multimodal outputs—has been chosen to create a system that is age-appropriate while satisfying the domain requirements of supporting language development, creativity, and educational storytelling.

3.2 DESIGN & FEATURES

3.2.1 USER JOURNEY

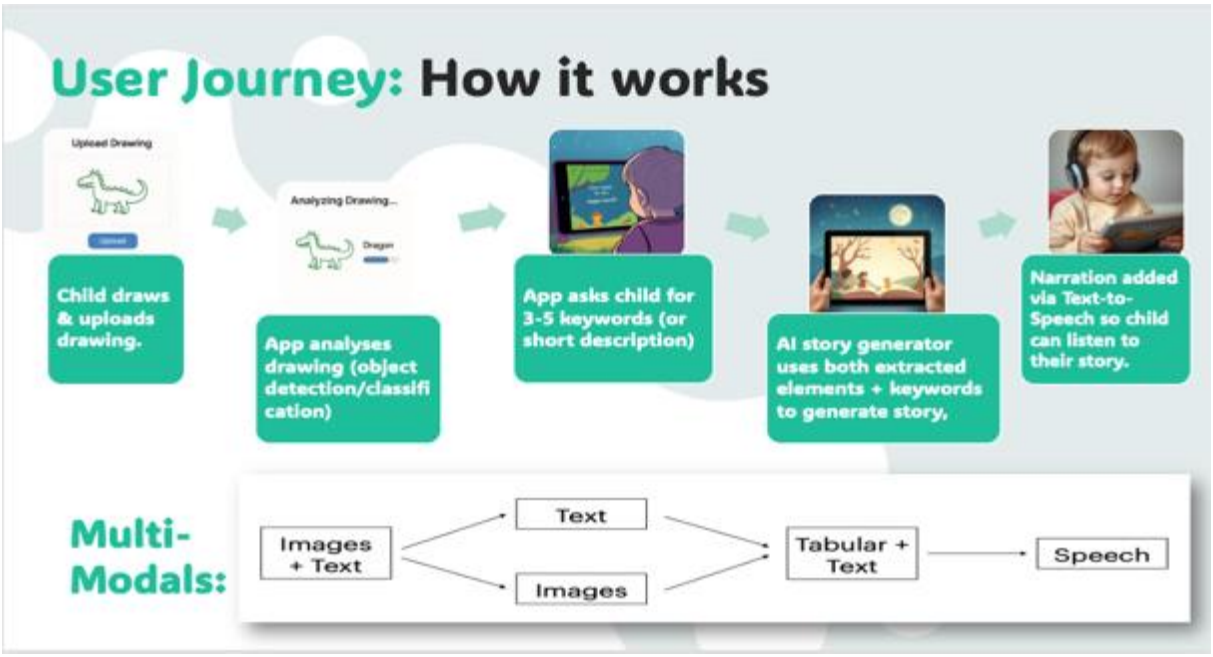


Figure 2 - User Journey of Little Legends

The **user journey** as shown in **Figure 2** begins with a **child providing a drawing using the sketchpad** or their caregiver **uploading a hand-drawn image** via the app interface, accompanied by an optional **simple word or phrase** as a storytelling prompt.

Upon submission, the system generates a personalized story using AI, inspired by both the drawing and the text. Simultaneously, **a visual AI illustration** is produced to complement the narrative.

Once both outputs are ready, the story is displayed alongside the AI-generated image, with an option to have the story read aloud through **a text-to-speech model.**

This seamless flow—from creation to storytelling to audio playback—ensures an engaging, low-barrier experience that encourages imagination, co-creation, and playful interaction.

3.3 SYSTEM ARCHITECTURE & TECHNICAL APPROACH

3.3.1 SYSTEM ARCHITECTURE – WEB APPLICATION

1. Frontend Layer (React Web Application)

Little Legends employs a multi-interface architecture starting with a central landing page that directs users to either a children's interface or an adults' interface, each tailored to different user needs and capabilities.

The children's interface centres around an **HTML5 canvas** with large, colourful buttons and audio-guided navigation designed for ages 3-5. Children draw directly on the canvas using touch-friendly controls, including colour palettes, brush size adjusters, and eraser tools. The interface uses numbered instruction buttons with **pre-recorded audio** to eliminate text dependency, automatically converting completed drawings to **blob format** for AI processing.

The adult interface provides comprehensive creation tools, including **canvas drawing, file upload options, voice recording for custom narration, and style preference controls.** Advanced features include **voice cloning through XTTS v2 integration**, drawing editing modes, and detailed audio playback controls, offering greater creative control over the storytelling process.

To address the **challenge of maintaining child engagement during AI processing delays**, the application incorporates an **interactive waiting component** that transforms loading periods into engaging mini-games. The component cycles through different activities with CSS animations, click tracking, and visual feedback, automatically rotating every few seconds to maintain novelty and prevent boredom. This approach **replaces traditional static loading indicators with age-appropriate interactive elements** that align with children's developmental needs for immediate feedback and hands-on engagement while stories and images are being generated.

Both interfaces share the same **React state management** and **backend API endpoints** but present dramatically different user experiences. The children's interface prioritizes simplicity and visual guidance, while the adults' interface emphasizes feature completeness and customization, demonstrating how identical AI capabilities can be made accessible to different demographics through appropriate interface design.

2. Backend Layer (Node.js/Express with Python Microservices)

The backend uses a hybrid architecture with a central **Node.js Express server** on port 1337 serving as the API gateway, coordinating file uploads, AI service calls, and static content serving. This main server integrates directly with OpenAI's GPT-4.1-mini for story generation while delegating specialized AI tasks to dedicated **Python Flask microservices**.

The system implements intelligent routing based on user inputs and style preferences, automatically selecting between **local ControlNet processing** for drawing-based generation and **Hugging Face APIs** for text-only scenarios. A sophisticated fallback system ensures ControlNet failures trigger Hugging Face alternatives, maintaining reliable content generation even with limited GPU resources.

Two specialized Python microservices handle intensive AI operations independently. The image service (port 5002) uses **ControlNet Scribble with Stable Diffusion** for processing children's drawings, while the TTS service (port 5001) employs **XTTS v2** with voice cloning capabilities and automatic audio format conversion. Both services implement **child-safe prompt engineering** and automatic **GPU/CPU detection.**

The architecture addresses technical challenges through multimodal story generation with adaptive prompt engineering, adjustable **ControlNet conditioning scales** for balancing drawing fidelity with artistic enhancement, comprehensive error handling with graceful degradation, and health monitoring across all services. This distributed approach allows independent optimization and scaling while maintaining loose coupling between components.

3.3.2 TECHNOLOGY & PRE-TRAINED AI MODELS – CHOSEN AFTER EVALUATION

Modality	Technology or AI Model to consider	Approach	Reasoning
Drawing Analysis **(Image Model)**	• OpenAI GPT-4.1-mini • ControlNet HED detector	• GPT-4.1-mini handles drawing analysis. • ControlNet processes drawings for image generation.	• OpenAI's GPT models are able to handle drawing analysis & story generation simultaneously at a reasonable cost, yet produces good quality stories at faster speed. • ControlNet is free and

			produces reasonable quality images compared to other models.
Image Generation **(Image-to-Image / text-to-image Model)**	**Primary model:** ControlNet Scribble + Stable Diffusion v1.5 • **ControlNet Model**: lllyasviel/sd-controlnet-scribble • **Base Model**: runwayml/stable-diffusion-v1-5 • **Edge Detector**: HED detector for preprocessing children's drawings **Fallback model:** Stable Diffusion XL • **Model**: stabilityai/stable-diffusion-xl-base-1.0 via Hugging Face API	• Generation with specified conditioning scale and child-safe parameters • Automatic fallback to Hugging Face if local ControlNet service fails	• Free and open-source • Cost-effective with good-enough quality images • Although the preferred model is still DALL-E 3, it is too expensive at this development phase.
Language Generation (prompt engineering) **(Text Generation Model)**	• OpenAI GPT-4.1-mini	Handles 3 distinct scenarios: • Image-only input • Text-only input • Image+text input	• MiniGPT-4 is more complicated to set up compared to GPT-4.1-mini • GPT-4.1-mini produces good quality stories
Voice Narration **(Text-to-Speech Model)**	• XTTS v2 (by Coqui)	• Natural and cheerful-sounding built-in voices • Allows cloning of storyteller's voice	• Free and open-source • Easy to set up

Table 2 - Current chosen pre-trained AI models

To bring the vision of personalized, engaging children's stories to life from simple drawings and prompts, this project strategically integrates a curated selection of AI technologies—outlined in **Table 2**—that balance creativity, technical feasibility, and cost-efficiency.

1. Drawing Analysis & Text Generation

For interpreting children's artwork and text generation, the current chosen model after evaluation is OpenAI's **GPT-4.1-mini,** which handles both drawing analysis and story generation in a single multimodal API call. The system sends both the drawing (as base64 data URI) and the text prompt to GPT-4.1-mini, which then analyzes the image content and generates the story simultaneously. In the initial preliminary report, I had listed potential options[3] like BLIP-2 or MiniGPT-4 for image captioning, but after much testing, the current implementation takes a more streamlined approach by leveraging GPT-4.1-mini's multimodal capabilities to handle both image understanding and narrative generation in one step. This eliminates the need for a separate captioning step and reduces the complexity of the modal pipeline.

2. Drawing Analysis & Image Generation

The other image analysis component we are using is **ControlNet's HED detector** to process drawings for image generation, for which we have designed a dual-model approach with intelligent routing based on style preferences (only available in the adult version of the application). The primary model combines **ControlNet Scribble (lllyasviel/sd-controlnet-scribble)** with **Stable Diffusion v1.5 (runwayml/stable-diffusion-v1-5)** for processing children's drawings. This setup uses HED detector for preprocessing, which provides softer edge detection better suited for child scribbles compared to traditional Canny edge detection.

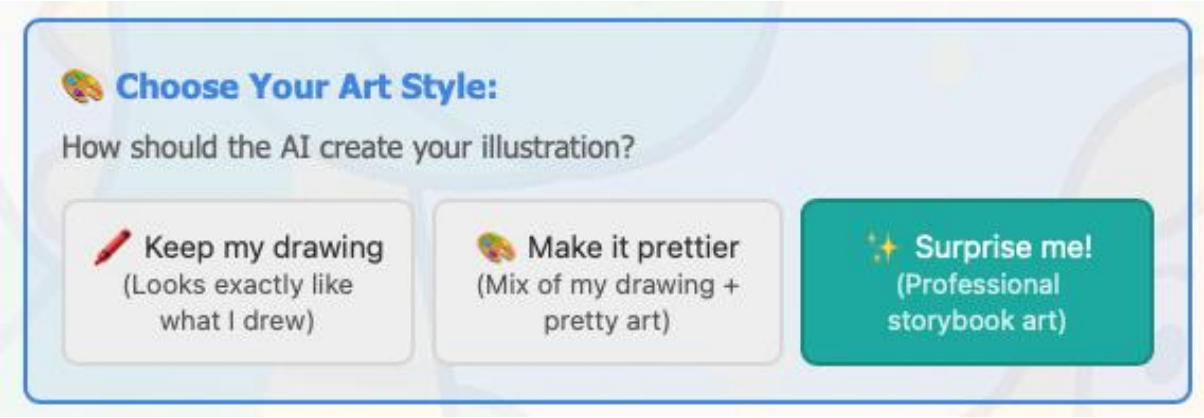


Figure 3 - Available style options (adult mode)

When the local ControlNet service fails or for specific style preferences, the system automatically **falls back to Stable Diffusion XL (stabilityai/stable-diffusion-xl-base-1.0)** accessed through the **Hugging Face API.** This fallback ensures reliable image generation even when local GPU resources are unavailable or when users select the "surprise me" option that bypasses drawing-based conditioning entirely.

[3] Refer to **Appendix B: Analysis of Pre-trained AI Models & Technology (Preliminary)**

The system implements comprehensive prompt engineering **to ensure child-appropriate image generation**. User prompts are automatically enhanced with child-friendly descriptors such as "beautiful children's book illustration of [prompt], colourful, whimsical, cartoon style," while negative prompts actively **filter out inappropriate content** including "ugly, scary, dark, realistic, violent, NSFW, blurry, low quality." The generation enforces child-safe parameters to ensure all outputs remain age-appropriate for the target demographic of 3-5 year olds.

3. Text-to-Speech (Audio Narration)

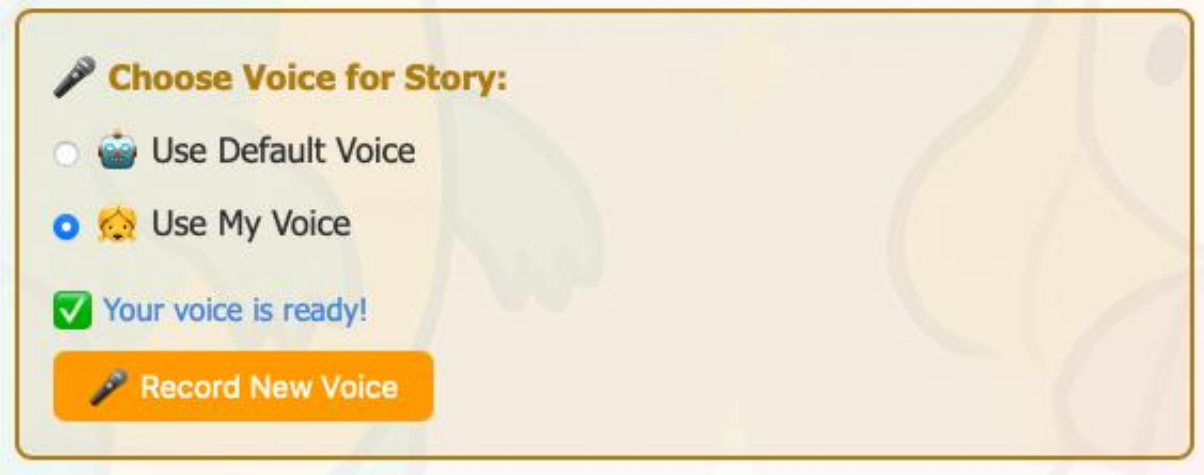


Figure 4 - Voice Narration options (adult mode)

For text-to-speech generation, the system employs **XTTS v2** (tts_models/multilingual/multi-dataset/xtts_v2) running on a dedicated **Flask microservice** with **automatic GPU/CPU detection.** The system offers two modes: **default narration** using the built-in "Ana Florence" speaker for consistent output and **custom voice cloning** that allows parents or educators to record personalized voice samples. The service automatically **converts uploaded WebM recordings to WAV format** and stores voice references temporarily with unique identifiers, enabling users to maintain custom voice settings throughout their session.

Altogether, this architecture aims to find an optimal balance of system performance, scalability, and creative flexibility—leveraging accessible, cost-effective technologies to deliver a responsive, multi-modal storytelling experience.

3.4 PROJECT PLANNING

The project planning begins with a clear outline of the **main tasks[4],** which include finalizing the UI/UX design, completing full-stack development, selecting and integrating the most suitable AI models for each modality (text, image, and voice), conducting user acceptance testing, and writing the final report. **Key milestones (Figure 5)[5]** include the completion of backend and frontend integration, AI model validation, successful user testing with early childhood educators and parents, and final report & demo preparation. Buffer periods are built into the schedule to account for potential delays or technical issues, ensuring that critical deadlines can still be met even if contingencies arise.

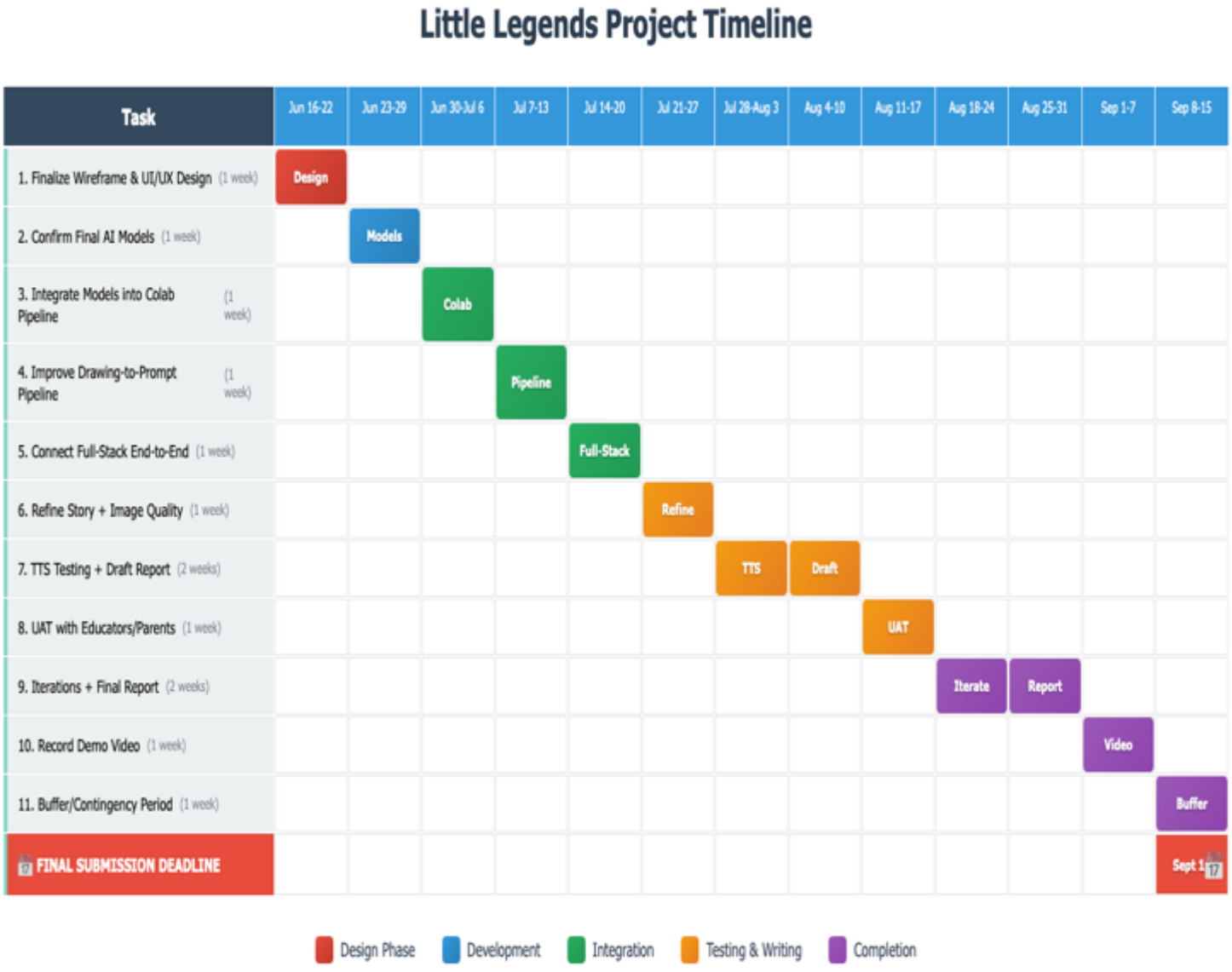


Figure 5 - Project Timeline Gantt Chart

[4] Refer to **Appendix C: Main Tasks** for more information.
[5] Refer to **Appendix D: Project Timeline & Milestones** for table presentation that includes more information.

CHAPTER 4: IMPLEMENTATION (2261 WORDS)

4.1 PROTOTYPE

4.1.1 FIRST ITERATION

The first prototype shown in **Figure 6** implemented a fully functional upload-and-generate workflow: children could upload their drawing via the "Upload Drawing" button and type a simple prompt, then tap "Generate Story" to kick off AI processing. While the models ran, a dynamic "Generating story… Please wait" message with animated dots kept users informed. Once ready, the AI-written story appeared alongside a placeholder (or real) illustration area—falling back to a friendly "Image not available" graphic if the pipeline was still catching up. Text-to-speech controls (Read Aloud, Pause, Stop) let users hear the story, and the clean, softly coloured background and centred card layout ensured a warm, child-friendly interface.

The prototype implemented story generation from multimodal inputs, image uploading & text prompts, and basic TTS narration; although the image-generation pipeline, despite being fully architected and provisioned, was not yet displaying results in the frontend.

1. Story Generation

The server sends both the drawing (dataUri) and the text prompt (prompt) to the GPT-4.1-nano model, received the generated narrative in story, and returned it as JSON; the React frontend then submitted the drawing and prompt via a fetch() POST request, extracted data.story from the response, stored it in state, and rendered it inside the {story} element. **(Figures 7 & 8)**

```
// Call OpenAI to get the story
try {
  const response = await openai.chat.completions.create({
    model: "gpt-4.1-nano",
    messages: [
      {
        role: "user",
        content: [
          {
            type: "text",
            text: `Generate a story for children based on this picture and the following prompt: '${prompt}'.`
          },
          {
            type: "image_url",
            image_url: {
              url: dataUri
            }
          }
        ]
      }
    ],
    max_tokens: 500
  });
```

Figure 7 - Backend: Invoking OpenAI GPT-4.1-nano to generate story

```
    const formData = new FormData();
    formData.append("drawing", drawing);      // File field
    formData.append("prompt", prompt); // Text field
    setIsGenerating(true);
    try {
      const response = await fetch("http://localhost:1337/generate", {
        method: "POST",
        body: formData,
      });

      const data = await response.json();
      setStory(data.story);
      setAiImageUrl(data.aiImageUrl);
      setIsGenerating(false);
    } catch (error) {
      console.error("Error generating story:", error);
      setStory("Sorry, something went wrong while generating your story.");
      setIsGenerating(false);
    }
  };
```

Figure 8 - Frontend: Receiving & Displaying Generated Story

Figure 9 - Child-drawn wolf

However, when testing with a child-drawn wolf sketch **(Figure 9),** the combined drawing-and-prompt approach reliably produces coherent narratives—for example, a simple wolf sketch with the prompt "*A wolf with fire mane and tail wants to become my little pony*" yields an appropriately themed story **(Figure 10)**—whereas relying on the drawing alone can generate mismatches (e.g., interpreting a wolf sketch as a "blue horse," as illustrated in **Figure 11**).

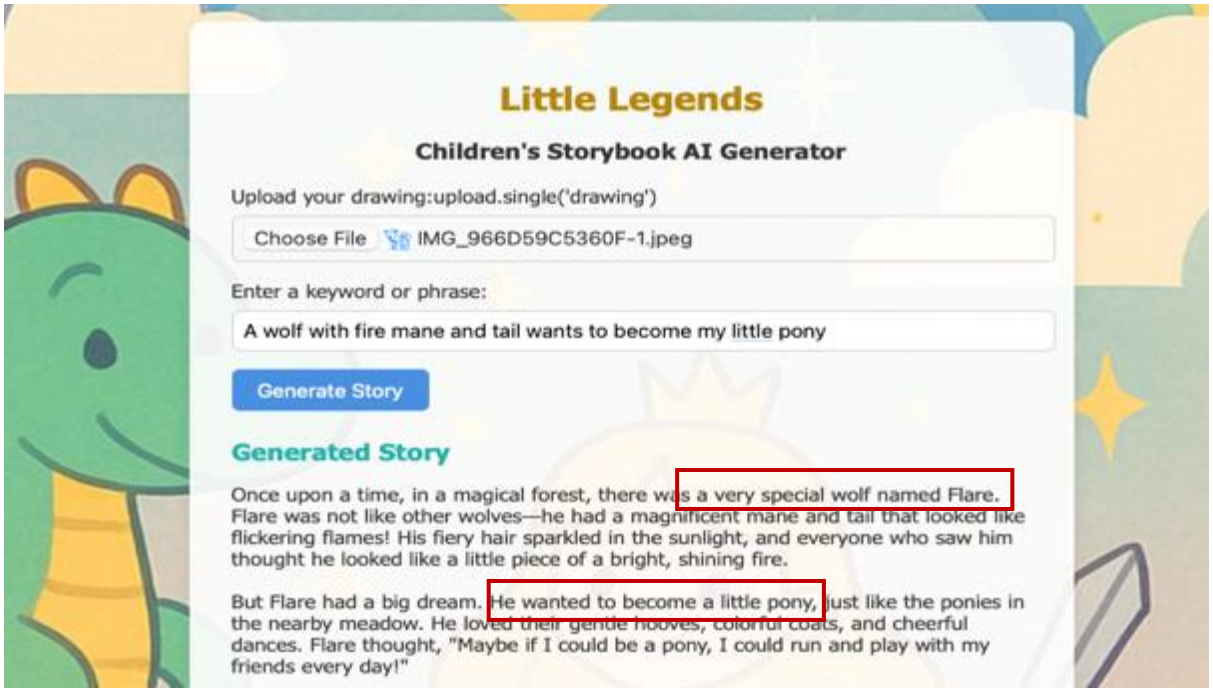


Figure 10 - Image + Prompt Generated Story

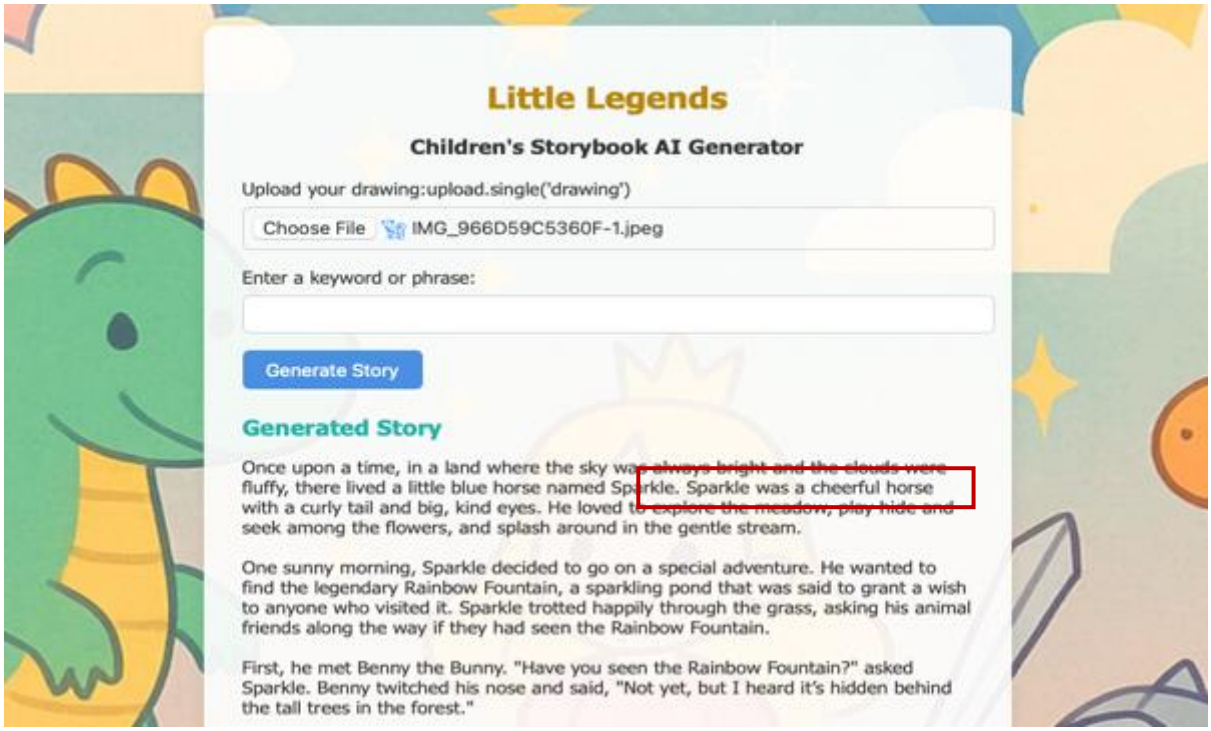


Figure 11 - Image-only Generated Story

To address this, a quick captioning step may be required—using a vision-language model like BLIP-2—to convert the sketch into a clear description before it reaches the story generator. Multiple captioning models can be benchmarked on the labelled sketch dataset by measuring precision and recall, and prompt templates plus temperature settings can be tuned to

optimize output quality. This additional step provides the story model with a concise summary to work from, reducing odd mismatches.

2. Image Generation

The image-generation pipeline, managed through a Colab script and Google Drive syncing during the first prototype **(Figure 14),** was encountering integration challenges: although input files were correctly saved and processed, the resulting illustrations often failed to display in the frontend and occasionally returned blank canvases or NSFW warnings **(Figures 12 & 13)—**likely due to GPU constraints and model misconfigurations.

To resolve these integration hiccups, Google Drive sync must be verified to be real-time and that the backend's polling interval is sufficient. Next, Colab logs would be inspected for CUDA or memory errors, and, if necessary, switched to a smaller or CPU-compatible model variant to eliminate GPU timeouts. To prevent blank outputs and NSFW triggers, the safety checker in the Stable Diffusion pipeline would be fine-tuned or swapped to an alternative image model with fewer false positives. Finally, for the purpose of this project, I would simplify the frontend image-loading logic by serving output files directly from the Express static route.

Figure 12 - NSFW warning on Colab

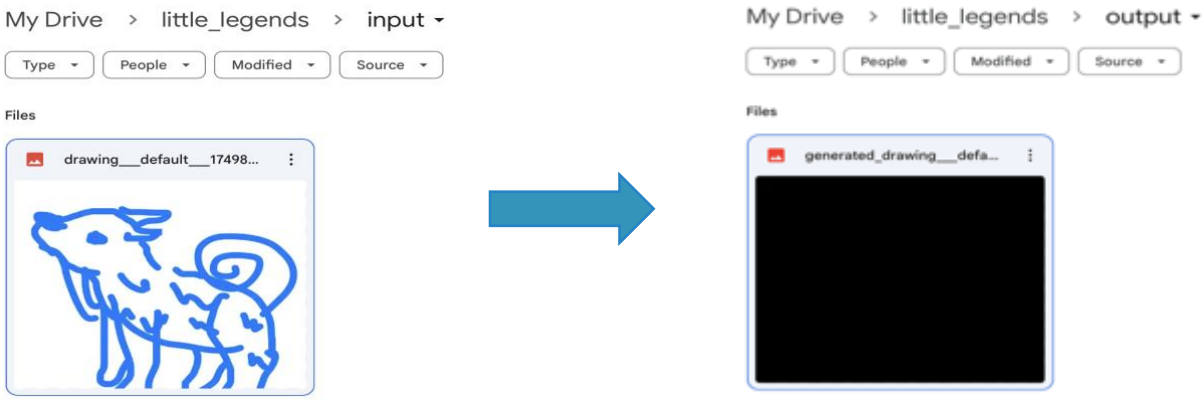


Figure 13 - Blank canvas returned after image generation

```
// Call Python backend to generate AI image from drawing
let aiImageUrl = null;
try {
  const promptRaw = prompt || "default";
  const safePrompt = promptRaw.trim().replace(/\s+/g, "_").slice(0, 20);
  const timestamp = Date.now();
  const fileName = `drawing___${safePrompt}___${timestamp}.png`;
  const inputPath = path.join(DRIVE_INPUT_PATH, fileName);

  // Ensure input folder exists
  fs.mkdirSync(DRIVE_INPUT_PATH, { recursive: true });

  // Save uploaded image to Drive input folder
  fs.writeFileSync(inputPath, file.buffer);

  // Poll for generated image
  const outputFileName = `generated_drawing___${safePrompt}___${timestamp}.png`;
  const outputPath = path.join(DRIVE_OUTPUT_PATH, outputFileName);

  const maxWaitTimeMs = 6000;
  const pollIntervalMs = 5000;
  let waited = 0;
⌘L to chat, ⌘K to generate
  while (!fs.existsSync(outputPath) && waited < maxWaitTimeMs) {
    await new Promise(resolve => setTimeout(resolve, pollIntervalMs));
    waited += pollIntervalMs;
  }

let finalOutputFile = null;
const files = fs.readdirSync(DRIVE_OUTPUT_PATH);
for (const f of files) {
  if (f.includes(`generated_drawing___${safePrompt}___${timestamp}`)) {
    finalOutputFile = f;
    break;
  }
}
```

Figure 14 - Colab & Google Drive sync

3. Voice Narration

In the first prototype, the voice narration was using the browser's built-in Web Speech API **(Figure 15),** which met basic functionality but sounded noticeably artificial. To create a warmer, more engaging reading experience, the next iteration would integrate a naturalistic TTS engine—such as Coqui TTS, XTTS v2, or, budget permitting, GPT-4o-mini-TTS. These models would be fine-tuned and calibrated through targeted user testing—gathering feedback on clarity, expressiveness, and emotional tone—to ensure the selected voice resonates with both children and caregivers.

```
const handleReadStory = () => {
  if (window.speechSynthesis.paused) {
    window.speechSynthesis.resume();
    return;
  }

  if (window.speechSynthesis.speaking) {
    return; // prevent overlapping reads
  }

  const utterance = new SpeechSynthesisUtterance(story);
  utteranceRef.current = utterance;
  window.speechSynthesis.speak(utterance);
};

const handlePauseStory = () => {
  window.speechSynthesis.pause();
};

const handleStopStory = () => {
  window.speechSynthesis.cancel();
};
```

Figure 15 - Story Narration (Text-to-Speech) using speechSynthesis

4.1.2 SECOND ITERATION

For the second iteration, after much evaluation from user studies, **the primary image input** as a source of story generation had been **diversified into 2 methods**—the original **image upload feature** and an additional **drawing pad interface (Figure 16)** that allowed children to create artwork directly within the browser by drawing directly on the sketch pad (which came with a selection of colours and an erase feature). This was to accommodate different user preferences and technical capabilities.

Additionally, once the story had been generated, it would be placed alongside the source image on the right **(Figure 17),** as feedback of tedious scrolling was one of the responses derived from user testing.

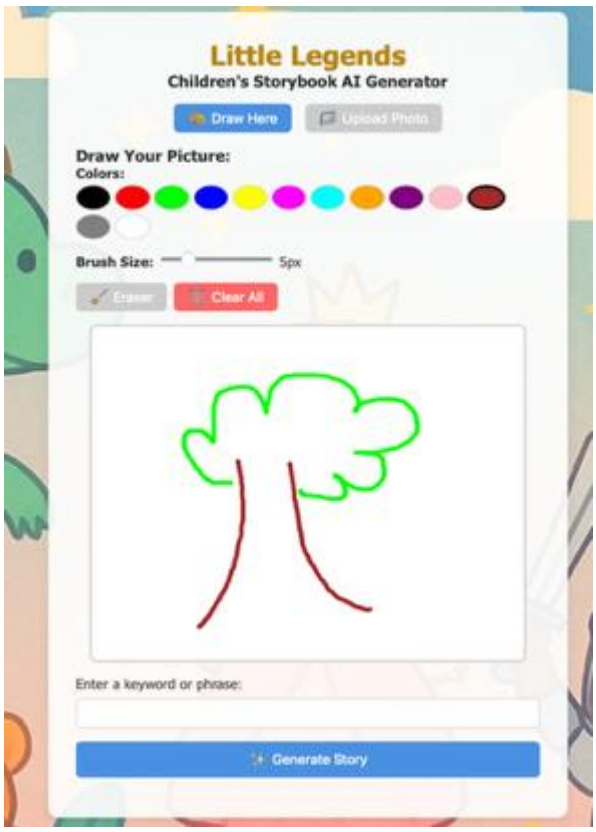


Figure 16 - Drawing Pad

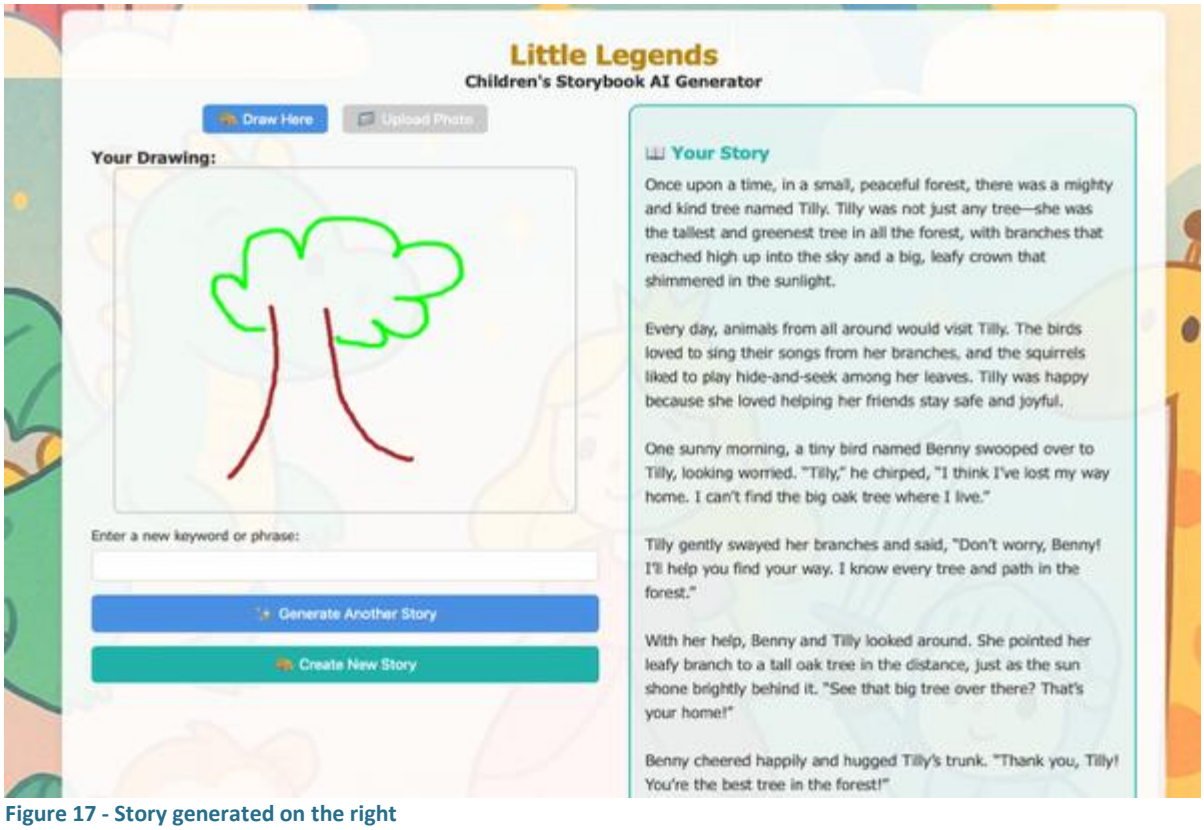


Figure 17 - Story generated on the right

Switching between the drawing pad and image upload was easy as well using the buttons **“Draw Here”** & **“Upload Photo”** located at the top of the drawing pad, according to the users’ needs and preferences.

Figure 18 - Drawing & Upload buttons for switching

A **landing page (Figure 19)** had also been introduced for more complex features that adults could use to enhance the storytelling experience for the child.

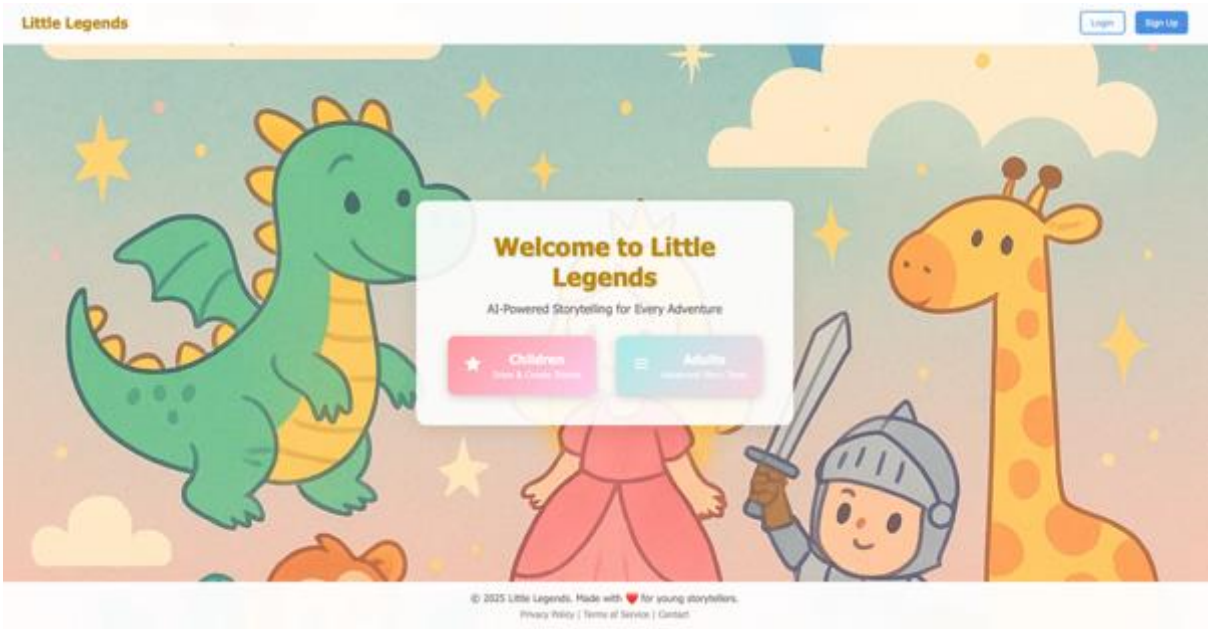


Figure 19 - Landing Page

1. Voice Narration

```python
@app.route('/generate-speech', methods=['POST'])
def generate_speech():
    try:
        data = request.json
        text = data.get('text', '')

        if not text:
            return jsonify({'error': 'No text provided'}), 400

        # Generate a unique filename for this audio
        audio_id = str(uuid.uuid4())
        output_path = f"temp_audio_{audio_id}.wav"

        # Generate speech with the simpler TTS model
        tts.tts_to_file(
            text=text,
            file_path=output_path
        )

        # Return the audio file and clean up after sending
        try:
            return send_file(
                output_path,
                mimetype='audio/wav',
                as_attachment=True,
                download_name=f'story_audio_{audio_id}.wav'
            )
```

The second iteration implemented the **Coqui TTS (Tacotron2 model)** to provide high-quality speech synthesis, replacing the browser-native speech APIs with a more natural and child-friendly voice generation system. It was a **Python-based TTS service** that generated audio files on demand, with streaming capabilities and proper error handling as well.

Figure 20 - Simpler Python-based TTS service

2. Drawing Pad Implementation

I was using an **HTML5 canvas-based drawing system (Figure 21)** with **colour palette selection**, **adjustable brush sizes, eraser functionality,** and **clear canvas options (Figure 16).**

The drawing system would capture user artwork and convert it into image data for AI processing.

```javascript
const draw = (e) => {
  if (!isDrawing) return;

  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');
  const rect = canvas.getBoundingClientRect();

  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  ctx.lineWidth = brushSize;
  ctx.lineCap = 'round';

  if (isEraser) {
    ctx.globalCompositeOperation = 'destination-out';
  } else {
    ctx.globalCompositeOperation = 'source-over';
    ctx.strokeStyle = currentColor;
  }

  ctx.lineTo(x, y);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo(x, y);
};
```

Figure 21 - Drawing canvas system

3. Image Processing/Analysis & Story Generation

The second iteration's setup for image processing and analysis was still employing OpenAI's GPT-4.1 nano to interpret children's drawings **(Figure 22).** The text generation portion also used the same AI model to create coherent and age-appropriate narratives. Both image analysis and textual prompts from user inputs were incorporated for the eventual thematically consistent generated story.

```
app.post('/generate', upload.single('drawing'), async (req, res) => {
  const file = req.file;
  const prompt = req.body.prompt;

  if (!file) {
    return res.status(400).json({ error: 'No file uploaded' });
  }

  // Get base64 string from the file buffer
  const base64Image = file.buffer.toString('base64');
  const mimeType = file.mimetype;
  const dataUri = `data:${mimeType};base64,${base64Image}`;

  // Call OpenAI to get the story
  try {
    const response = await openai.chat.completions.create({
      model: "gpt-4.1-nano",
      messages: [
        {
          role: "user",
          content: [
            {
              type: "text",
              text: `Generate a story for children based on this picture and the following prompt: '${prompt}'.`
            },
            {
              type: "image_url",
              image_url: {
                url: dataUri
              }
            }
```

Figure 22 - Streamlined image analysis and story generation

4. Areas for enhancement

While the second iteration's implementation produced coherent narratives, there were still occasional misinterpretations of drawing content, particularly with abstract drawings from the child. Future iterations might implement intermediate image captioning steps using vision models like BLIP-2 to improve visual understanding, only if it is easy to incorporate and did not need further training.

The **Tacotron 2** model's TTS implementation provided good-quality speech synthesis, but the prototype would benefit from voice cloning capabilities using **XTTS-v2.** I tried this model initially, but there were errors that I did not manage to debug. However, I would continue with the troubleshooting, as enabling children to hear stories narrated in their own voices was one of the ultimate goals I had for this project.

The Google Colab-based image generation system was still experiencing reliability issues and processing delays. I would weigh the time constraint and overall quality of the project to see if part of further development would be focused on this portion—i.e., exploring with local implementation or more robust cloud-based solutions—or if this feature would be removed entirely.

Lastly, audio generation required some processing time. Implementation of audio caching and progressive loading could potentially improve user experience.

4.2 FINAL PRODUCT (SO FAR – THIRD ITERATION)

After much evaluation through a combination of **User Testing (UAT)** and **AI Models Testing**, the third iteration represents a significant architectural evolution from the earlier prototypes. The **system migrated from the Google Colab-based approach to a robust local microservices architecture**, addressing the reliability and performance issues identified in previous iterations. It has also been upgraded in terms of **UX/UI features** to better accommodate the needs of children.

4.2.1 CHILDREN MODE

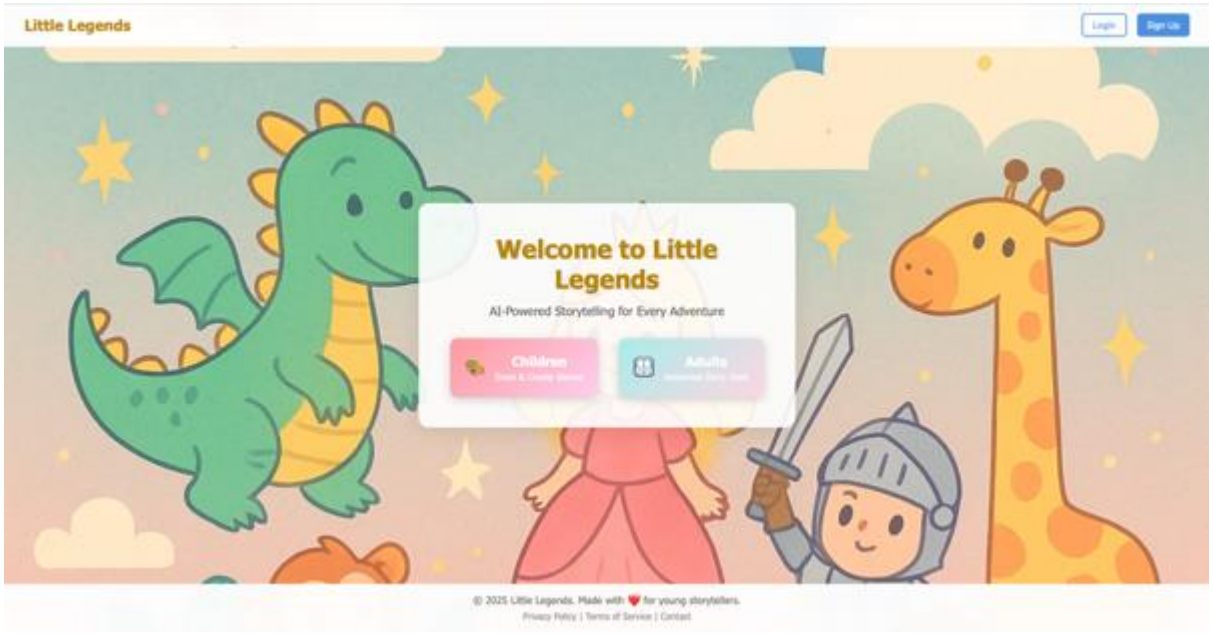


Figure 23 - New landing page button icons

Although it is only a tiny change, the different icons used for the respective children and adult pages' buttons will help children to intuitively know which button to press if they want to draw something, i.e., the paint palette icon **(Figure 23).**

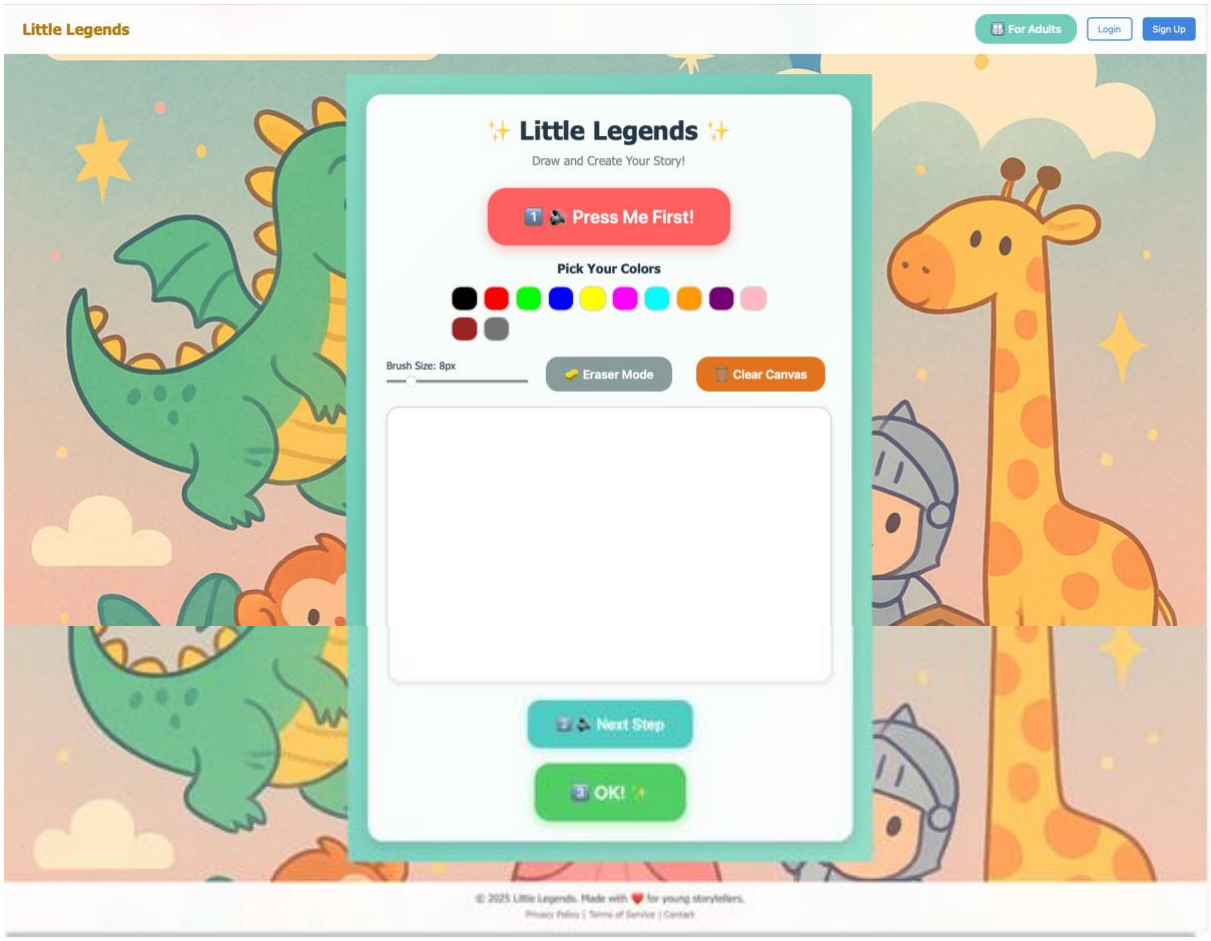


Figure 24 - Children Page Layout (Before story generation)

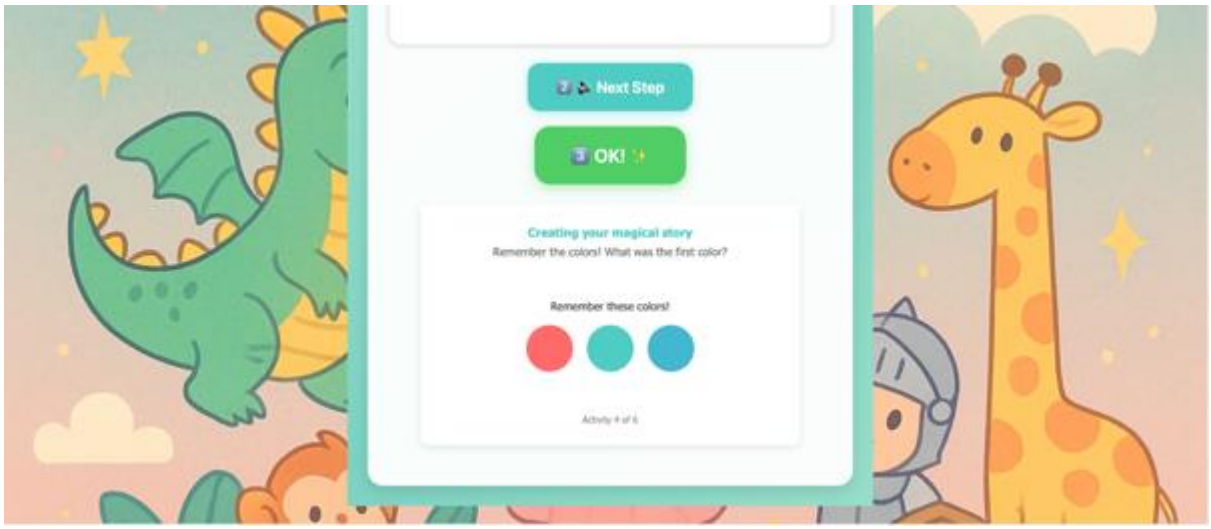


Figure 25 – Mini games to help children wait during story generation

The simplistic layout of the children version of the application is done intentionally to draw their attention to the big, colourful child-centric features. The canvas interface **(Figure 24)** prioritizes visual expression over text input. Children can select from a comprehensive colour palette, adjust brush sizes, and use eraser tools without requiring written prompts, making the creative process entirely visual and tactile.

The end product **(Figure 26)** are an image and story generated from their own drawings, the latter which they can listen to with a press of the yellow button. The audio narration is generated using the **Coqui XTTS-v2 model,** which the children can control using the playback buttons **(Figure 27).**

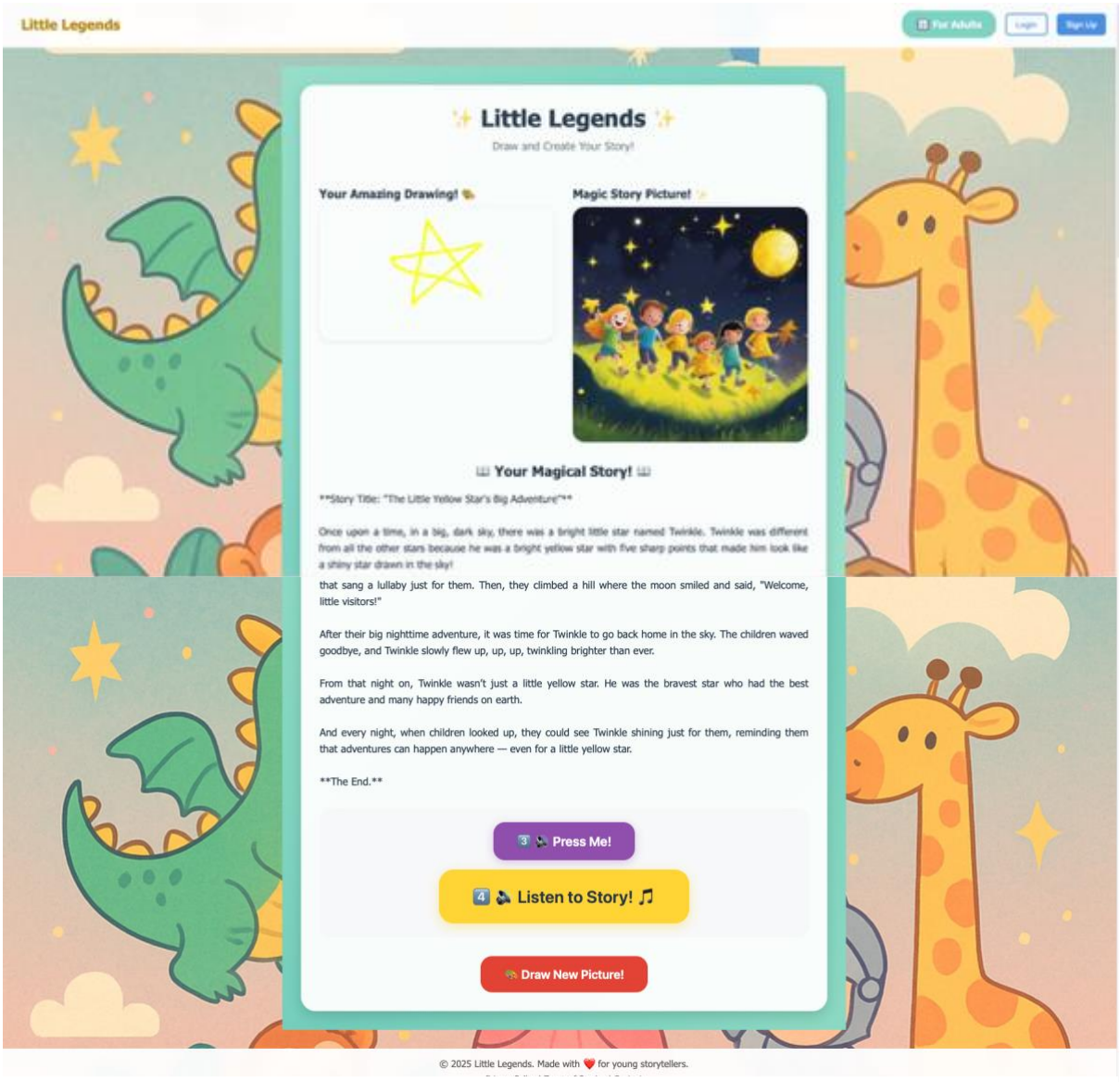


Figure 26 - Final story, image and narration generated

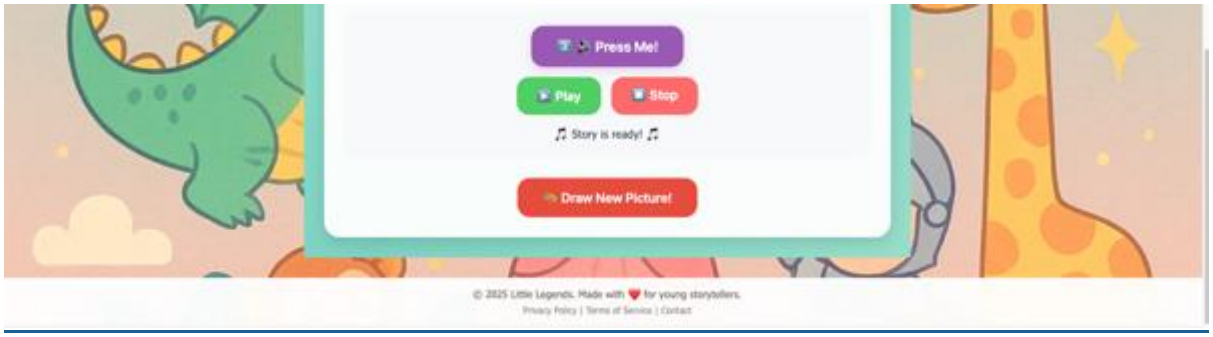


Figure 27 - Playback buttons (children mode)

1. Audio-Guided Workflow

The children page eliminates reading requirements through an audio-first design. Large numbered instruction buttons (1, 2, 3) **(Figure 24)** guide children through each step with spoken directions, ensuring pre-literate users can navigate independently without adult assistance. The interface employs a deliberate colour progression to create visual learning cues. Each step uses distinct colours that help children understand the workflow sequence and current position in the story creation process.

The audio instruction system uses a **playInstruction() function (Figure 28)** that maps step numbers to specific audio files served from the backend.

```
// Updated audio instruction functions
const playInstruction = (step) => {
  const audioFiles = {
    1: 'http://localhost:1337/audio/instruction1.wav',
    2: 'http://localhost:1337/audio/instruction2.wav',
    3: 'http://localhost:1337/audio/instruction3.wav'
  };

  const audio = new Audio(audioFiles[step]);
  audio.play().catch(error => {
    console.error("Error playing instruction audio:", error);
  });
};
```

Figure 28 – playInstruction() Function

2. Engaging Wait States

During AI processing, the **InteractiveWaitingComponent (Figure 29)** presents six rotating **mini-games (Figure 25)** (pencil clicking, colour buttons, star counting, memory games, hidden objects, and balloon growing) that maintain attention and provide educational value during the story and image generation processes. The six activities are rotated using React state and useEffect hooks. The core rotation logic cycles through activities at timed intervals.

```
const activities = [
  { name: "pencil", duration: 8000, message: "Help the magic pencil draw by clicking it!" },
  { name: "colors", duration: 6000, message: "Keep the story warm by pressing the colorful buttons!" },
  { name: "stars", duration: 10000, message: "Count the twinkling stars while your story is being made!" },
  { name: "memory", duration: 8000, message: "Remember the colors! What was the first color?" },
  { name: "hiddenStar", duration: 6000, message: "Find the hidden star by clicking around!" },
  { name: "balloon", duration: 7000, message: "Blow to help your story grow! Click the balloon!" }
];

// Rotate activities every few seconds
useEffect(() => {
  const interval = setInterval(() => {
    setCurrentActivity((prev) => (prev + 1) % activities.length);
    setClickCount(0);
    setBalloonSize(50);
    setStars([]);
  }, activities[currentActivity].duration);

  return () => clearInterval(interval);
}, [currentActivity]);
```

Figure 29 - InteractiveWaitingComponent

4.2.2 ADULT MODE

The adult page has all the same features in the children page **except audio instruction buttons** as the adults are able to read the instructions while they assist the children with the story, image and narration generation; and they are able to upload images of drawings. However, there are a couple of extra features that will enhance the customisation of the experience.

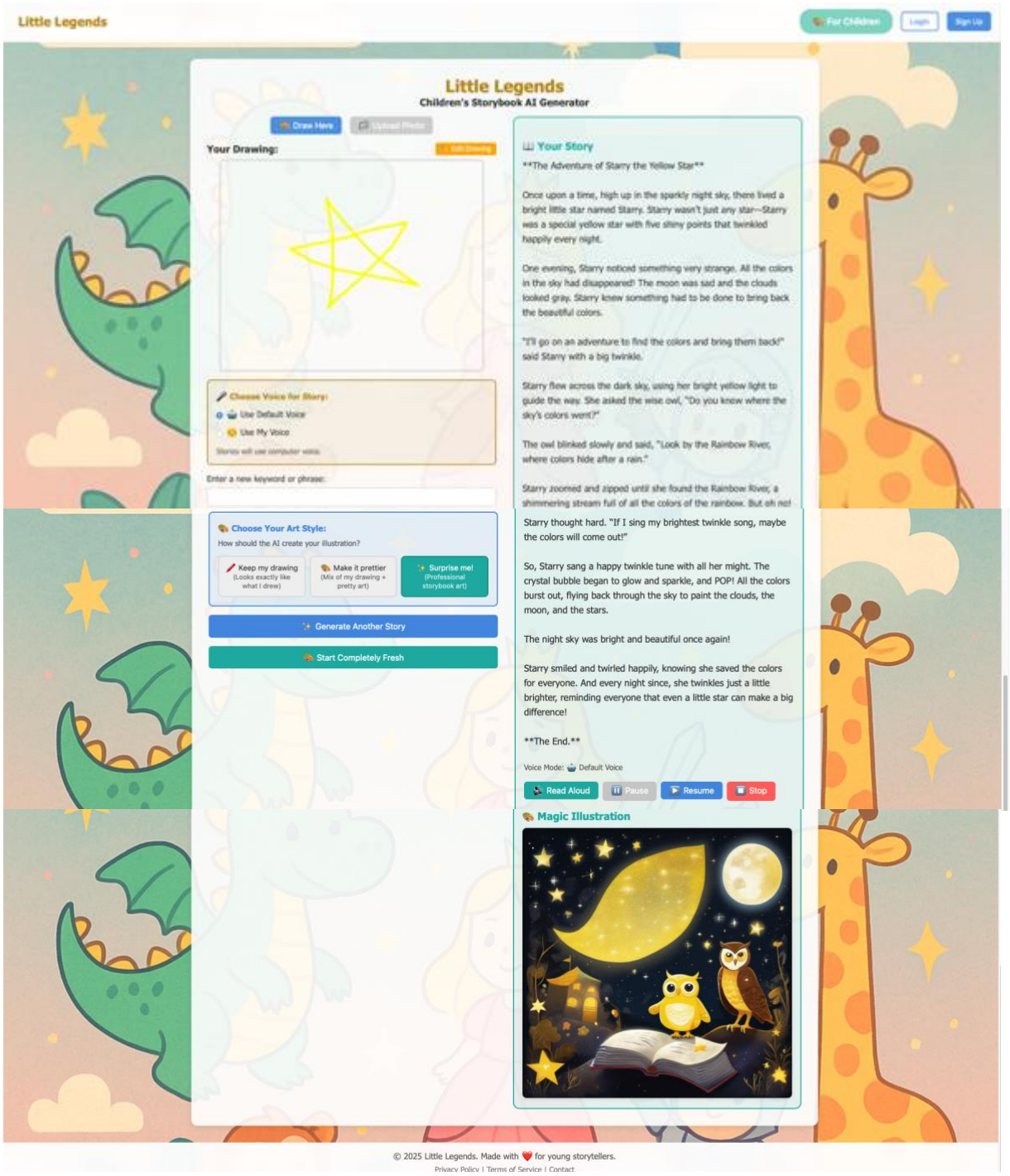


Figure 30 - Adult Page Layout & Features

1. Voice Cloning or Default Voice

The third iteration implements sophisticated voice personalization through XTTS v2 integration. **Users can choose between a default computer-generated voice or record their own voice sample for story narration (Figure 31).** The voice recording system **(Figure 32)** captures 5-10 seconds of user speech, processes it through the TTS service, and generates personalized story audio **(Figure 33)** that maintains the speaker's vocal characteristics and emotional tone.

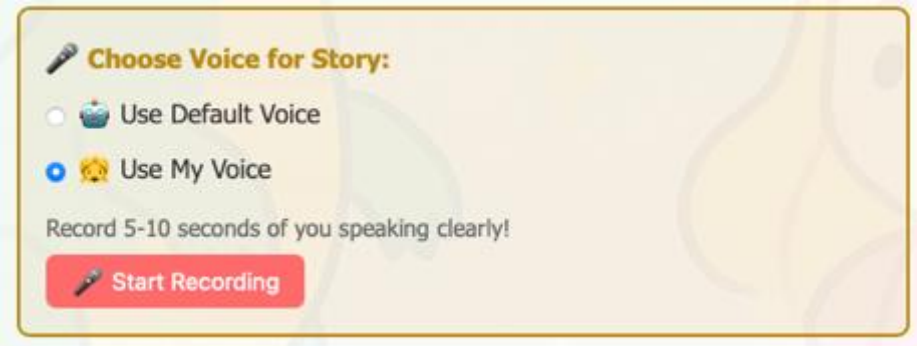


Figure 31 – Audio Narration Voice Options

```
// Voice Recording Functions
const startVoiceRecording = async () => {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });

    let mimeType = 'audio/wav';
    if (!MediaRecorder.isTypeSupported(mimeType)) {
      mimeType = 'audio/webm';
      console.log('WAV not supported, using WebM format');
    } else {
      console.log('Recording in WAV format');
    }
```

Figure 32 - Voice Recording code snippet

```
const uploadVoiceReference = async (audioBlob) => {
  try {
    const formData = new FormData();
    const filename = audioBlob.type.includes('wav') ? 'voice_reference.wav' : 'voice_reference.webm';
    formData.append('voice_file', audioBlob, filename);

    const response = await fetch('http://localhost:5001/upload-voice', {
      method: 'POST',
      body: formData,
    });
```

Figure 33 - Upload Recorded Voice code snippet

2. Image Generation Fidelity Options

The system provides three distinct artistic interpretation modes for AI-generated illustrations **(Figure 34).** "Keep-drawing" **preserves the original artwork's appearance**, "make-prettier" **blends user drawings with enhanced artistic elements**, and "surprise-me" creates **professional storybook-quality illustrations loosely inspired by the original concept (Figure 35)**. This flexibility accommodates varying preferences for artistic control versus AI enhancement.

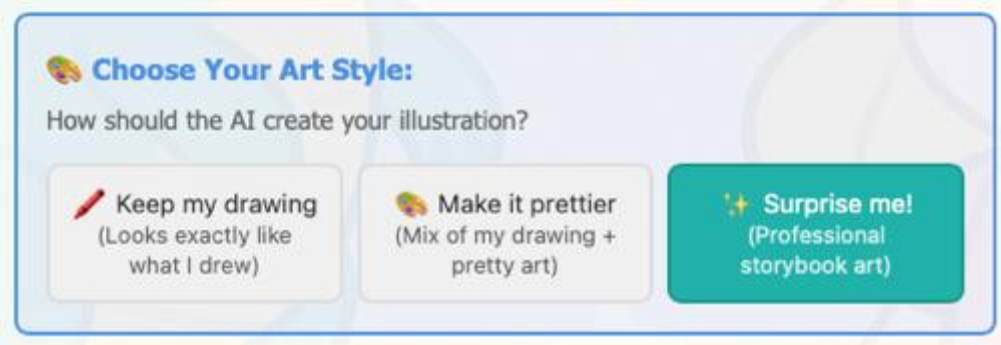


Figure 34 - Image Generation Options

```
if (file) {
  // Has drawing - route based on style preference
  switch (stylePreference) {
    case 'keep-drawing':
      console.log('Keep drawing mode - High ControlNet conditioning');
      aiImageUrl = await generateImageWithControlNet(file.buffer, prompt, 0.9);
      break;

    case 'make-prettier':
      console.log('Make prettier mode - Medium ControlNet conditioning');
      aiImageUrl = await generateImageWithControlNet(file.buffer, prompt, 0.5);
      break;

    case 'surprise-me':
      console.log('Surprise me mode - Pure Hugging Face with story elements');
      // Extract key elements from the generated story for better image prompts
      const storyElements = await extractStoryElementsForImage(story);
      aiImageUrl = await generateImageWithHuggingFace(storyElements);
      break;

    default:
      console.log('Default to make prettier mode');
      aiImageUrl = await generateImageWithControlNet(file.buffer, prompt, 0.5);
  }
```

Figure 35 - Image Generation Fidelity code snippet

4.2.3 OVERALL CHANGES & UPGRADES IN THIRD ITERATION

1. Architectural Evolution to Microservices

The final implementation adopts a microservices architecture with the Express.js server (index.js) serving as the orchestration layer, coordinating between specialized Python services. This separation of concerns improved system reliability and allowed for independent scaling of different AI processing components. Keyword Extraction and a Concept List which were originally considered in the early planning stages[6] were not incorporated into the final product as the current chosen **OpenAI's GPT-4.1-mini** model already provides good quality image and text-prompt analysis and is able to generated child-suitable stories with the right prompting code.

The **image generation service (image_service.py)** now runs locally using **ControlNet-enhanced Stable Diffusion (Figure 36)**, eliminating the Google Drive synchronization bottlenecks and GPU timeout issues that plagued earlier iterations. The service provides three distinct style preferences—"keep-drawing," "make-prettier," and "surprise-me"—allowing users to control how closely the AI illustration matches their original artwork.

```
# Load ControlNet Scribble model (better for understanding context)
controlnet = ControlNetModel.from_pretrained(
    "lllyasviel/sd-controlnet-scribble",
    torch_dtype=torch.float16 if device == "cuda" else torch.float32
)

# Load Stable Diffusion pipeline with ControlNet
pipe = StableDiffusionControlNetPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    controlnet=controlnet,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    safety_checker=None,
    requires_safety_checker=False
).to(device)
```

Figure 36 - image_service.py code snippet

[6] Refer to **Appendix B: Analysis of Pre-trained AI Models & Technology (Preliminary)**

2. Advanced Voice Synthesis with XTTS v2

The third iteration successfully implements **XTTS v2 (tts_service.py)** the **default "Ana Florence" TTS audio narration generation** (children mode) along with **voice cloning capabilities** (adult mode) **(Figure 37),** fulfilling one of the project's key objectives. Users can now record 5-10 seconds of their own voice, which the system processes to generate personalized story narrations. The service includes **automatic format conversion (WebM to WAV) (Figure 38)** and **comprehensive error handling** to ensure compatibility across different browsers and recording devices.

```
# Load XTTS v2 - supports voice cloning
tts = TTS("tts_models/multilingual/multi-dataset/xtts_v2").to(device)
print(f"XTTS v2 model loaded on {device}")

# Store for voice references (in production, use proper database)
voice_references = {}

# Create a default speaker reference using XTTS v2's built-in speaker
DEFAULT_SPEAKER = "Ana Florence"  # XTTS v2 built-in speaker
```

Figure 37 - Voice cloning code snippet

```
# Check if file is already WAV format
if voice_file.filename.endswith('.wav'):
    # File is already WAV, save directly
    voice_path = f"voice_ref_{voice_id}.wav"
    voice_file.save(voice_path)
    print(f"WAV file saved directly: {voice_path}")
else:
    # File is WebM, need to convert to WAV
    temp_webm_path = f"temp_voice_{voice_id}.webm"
    voice_file.save(temp_webm_path)

    # Convert WebM to WAV using pydub
    try:
        print(f"Converting WebM to WAV: {temp_webm_path}")
        audio = AudioSegment.from_file(temp_webm_path, format="webm")
        voice_path = f"voice_ref_{voice_id}.wav"  # Save as WAV
        audio.export(voice_path, format="wav")

        # Clean up temporary webm file
        os.remove(temp_webm_path)
        print(f"Successfully converted and saved: {voice_path}")

    except Exception as conversion_error:
        print(f"Audio conversion error: {conversion_error}")
        # Clean up on error
        if os.path.exists(temp_webm_path):
            os.remove(temp_webm_path)
        return jsonify({'error': 'Failed to convert audio format'}), 500
```

Figure 38 - Audio Format Conversion code snippet

CHAPTER 5: EVALUATION (1567 WORDS)

5.1 COMPLETED TESTING – DEVELOPMENT PHASE EVALUATION

5.1.1 CORE FEATURES VALIDATION

Comprehensive manual testing has been conducted across all primary system features to ensure functional correctness. **The drawing interface has been tested with various input scenarios,** including different brush sizes, colour selections, eraser functionality, and canvas clearing operations. All drawing tools perform as expected, with smooth stroke rendering and accurate colour representation.

File upload functionality has been thoroughly tested with multiple image formats (PNG, JPEG, GIF) and file sizes, confirming proper handling of user-uploaded drawings and photographs. The system correctly processes uploaded files, generates preview images, and maintains file integrity throughout the processing pipeline.

5.1.2 STORY GENERATION PIPELINE TESTING

The multimodal story generation system has undergone extensive testing with diverse input combinations. Testing scenarios included simple drawings with basic prompts, complex artistic compositions with detailed text descriptions, and edge cases such as abstract artwork or minimal text input. **The system has not been consistently producing coherent narrative outputs; further modifications and testing are required to ensure generated stories are coherent and of quality.**

Even though the OpenAI GPT-4.1-mini integration performs reliably, with response times typically ranging from 5 to 10 seconds for story generation, further training will be required to increase accuracy and precision (i.e., tokens have already been increased from 500 to 800, further increments might be needed). However, **error-handling mechanisms are functioning correctly**, providing appropriate user feedback when API calls fail or when invalid input is submitted.

5.1.3 TEXT-TO-SPEECH IMPLEMENTATION TESTING

Initial testing configuration challenges with the XTTS-v2 model has been resolved, especially with the cloning feature. The issue was due to the incongruency in audio file formats recorded and uploaded. This has since been rectified with **automatic format conversion (WebM to WAV).**

5.1.4 USER INTERFACE & INTERACTION TESTING

The web application interface has been **tested across different interaction patterns**, including mode switching between drawing and upload functionality, state management during story generation, and audio playback controls. All interface elements respond appropriately to user interactions, with proper visual feedback and state transitions.

Mode switching functionality performs correctly (i.e., drawing pad to image upload and vice versa in adult mode), clearing appropriate state variables while preserving user drawings when intended. The responsive design adapts properly to different screen sizes, maintaining usability across desktop and mobile environments.

5.2 PERFORMANCE & RELIABILITY TESTING

5.2.1 RESPONSE TIME ANALYSIS

Performance testing has measured system response times across different functional areas:

- **Story generation**: 5-10 seconds (dependent on OpenAI API response time)
- **Audio generation:** up to 600 seconds (XTTS-v2 TTS processing time can take up to 10 minutes if using cloned voice)
- **Image generation:** up to 300 seconds (depending on fidelity selection, the higher the fidelity, the longer it takes to generate image)
- **Image upload and processing:** <1 second for typical file sizes
- **Interface interactions:** <100 ms for all UI operations

5.2.2 SYSTEM RELIABILITY ASSESSMENT

The prototype demonstrates good reliability under normal usage conditions. Error handling mechanisms successfully manage API failures, invalid input scenarios, and network connectivity issues.

5.2.3 RESOURCE USAGE EVALUATION

Memory usage remains stable during extended usage sessions, with proper cleanup of temporary files and audio resources. The drawing canvas performs efficiently even with complex drawings, maintaining smooth interaction responsiveness.

5.3 AI MODELS & USER TESTING

The AI models for each respective modal during the third iteration were chosen after conducting **User Testing (UAT) with 15 adults** ranging from **parents to Early Childhood educators.** Children, even though were the primary target users of this project, were not used for user testing due to ethical reasons and insufficient research supporting tests involving children, as discovered in the Literature Review. Tests were conducted in the **A/B testing style for model comparisons.**

5.3.1 STORY GENERATION QUALITY ASSESSMENT

The initial models planned for comparison were OpenAI's GPT-4.1-nano and MiniGPT-4. But, due to the complexity of setting up MiniGPT-4 compared to OpenAI's GPT-4.1 models, I decided to proceed with OpenAI instead. However, within the 4.1 models, there were "mini" and "nano" to choose from; hence, an A/B testing was done with the users.

Users were asked to draw the same picture and use the same text prompts (if any) during two trials of the application – first trial with OpenAI's GPT-4.1-mini and second trial with GPT-4.1-nano. Out of 15 users, 10 chose the mini model over nano after the trials, citing the prior as producing more creative stories compared to the latter.

Additionally, to further assess the quality of stories generated, the users were asked to rate the following factors on a scale of 10 (i.e., 1 being poor, 10 being best) regarding the stories generated and these are the results:

- **Age-appropriate language** - 100%: All of the users agreed that generated content used vocabulary and themes suitable for preschool audiences.
- **Narrative coherence** - 93%: Most users thought the models maintained logical story structure and character consistency.
- **Creative engagement** - 75%: Some users thought the stories included imaginative elements that extended beyond literal interpretation of drawings.
- **Medium relevance scores** - 57%: Only a little over half of the users thought the generated stories incorporated recognizable elements from input drawings.

Users were then given the freedom to explore and test the application extensively. They highlighted the following areas for potential improvement:

- Abstract or highly stylized artwork occasionally resulted in misinterpretation of visual elements.
- Very brief text prompts (1-2 words) sometimes led to generic story templates rather than personalized narratives.

- Complex drawings with multiple characters or objects occasionally resulted in the story focusing on secondary rather than primary visual elements.

5.3.2 TEXT-TO-SPEECH QUALITY EVALUATION & DEFAULT VOICE SELECTION

A comparative analysis of audio quality between the second iteration's Tacotron 2 model and XTTS-v2 showed no significant difference in both naturalness and intelligibility. Both showed natural intonation and appropriate pacing. As XTTS-v2 has the extra capability of voice cloning, it was chosen over Tacotron 2.

In order to choose the most suitable default voice model from XTTS-v2, users were also asked to choose between 2 options (researched as the most popular voice models) narrating the same text output – "Gracie Wise" vs. "Ana Florence". It was a walkover for "Ana Florence" with 13 users choosing it as their preferred voice.

5.3.3 IMAGE GENERATION EVALUATION

The adult version of the web application allows fidelity options:

- **Keep my drawing:** Looks exactly like what I drew (90% ControlNet)
- **Make it prettier:** Mix of my drawing + pretty art (50% ControlNet)
- **Surprise me:** Professional storybook art (100% Hugging Face + Shape-Specific Story Elements)

Multiple testing of each option on the same pictures resulted in the general quality as shown in below example:

Original Image:

Figure 39 – Original image sketched on drawing pad

Images Generated:

Figure 40 – "Keep My Drawing"

Figure 41 – "Make it Prettier"

Figure 42 – "Surprise Me"

Both "Keep My Drawing" and "Make it Prettier" tend to generate the same images even though they use different conditioning scales of the ControlNet model; whereas Hugging Face generated high quality images with 80% accuracy rate to the original image and 85% accuracy rate to the story generated. This shows that a better primary image generation model might be needed to replace ControlNet, preferably one trained on children's drawings.

```
Smart Style System:
Keep drawing (90% ControlNet)
Make prettier (50% ControlNet)
Surprise me! (100% Hugging Face + Shape-Specific Story Elements)
Style preference received: surprise-me
Story generation scenario: image_only
Starting image generation with style preference: surprise-me
Surprise me mode - Pure Hugging Face with story elements
Extracted story elements for image: star-shaped main character Starry, a bright yellow five-pointed star with shiny tw
inkling points, accompanied by a wise owl-shaped character with large round eyes, in a sparkly dark night sky setting;
 key visual elements include a shimmering multicol
Generating image with Hugging Face (Surprise me mode)...
Defaulting to 'auto' which will select the first provider available for the model, sorted by the user's order in https
://hf.co/settings/inference-providers.
Auto selected provider: nscale
Surprise me image generated: surprise_1758487977051.png
Generating speech for text: **The Adventure of Starry the Yellow Star**

Once ...
Using default voice
Style preference received: make-prettier
Story generation scenario: image_only
Starting image generation with style preference: make-prettier
Make prettier mode - Medium ControlNet conditioning
Generating image with local ControlNet...
Prompt: no prompt provided
Conditioning scale: 0.5
Image generated successfully with ControlNet: controlnet_1758496161717.png
Style preference received: keep-drawing
Story generation scenario: image_only
Starting image generation with style preference: keep-drawing
Keep drawing mode - High ControlNet conditioning
Generating image with local ControlNet...
Prompt: no prompt provided
Conditioning scale: 0.9
Image generated successfully with ControlNet: controlnet_1758496738726.png
```

Figure 43 - Image conditioning using ControlNet (terminal output snippet)

5.4 CRITICAL EVALUATION OF PROJECT PROGRESS

5.4.1 ACHIEVEMENTS AGAINST ORIGINAL OBJECTIVES

The third iteration of *Little Legends* has delivered on its primary objective as an AI-powered storytelling platform. It integrates and validates all key functional components, including the ability to process multimodal inputs like children's drawings and text prompts, generate personalized stories using AI, synthesize high-quality speech, and present everything through a child-friendly, intuitive user interface.

From a technical standpoint, the project orchestrates between multiple AI systems—vision-language processing, natural language generation, image generation and speech synthesis, which is the core idea of the template this project has chosen for its purpose and guidelines.

In terms of user experience, the design strikes a balance between simplicity and functionality. It offers young users an engaging and age-appropriate interface while still allowing for creative input and responsive feedback. This ensures that children not only enjoy using the platform but can also interact with it meaningfully, fostering both creativity and independence.

5.4.2 LIMITATIONS & AREAS FOR IMPROVEMENT

While the story generation feature generally produces high-quality and engaging narratives, there is still some inconsistency depending on the input. In particular, when children submit complex or abstract drawings, the resulting stories can sometimes be less accurate or coherent. This suggests that the visual processing component could benefit from further refinement to better interpret a wider variety of creative input.

In terms of performance, the current system is responsive enough for testing, but some delays may become problematic in real-world use. For example, audio generation can take up to 600 seconds, which might feel like forever for younger children who typically have shorter attention spans. Even though mini games have been added to encourage patience, speeding this up further will help maintain their interests and create a smoother experience overall.

One of the more challenging aspects has been the integration of AI-generated images to accompany the stories. The current implementation struggles with more creative and accurate outputs for higher fidelity options. DALL-E 3 still remains the preferred choice for superior image quality, but its prohibitive cost during the development phase necessitated an open-source alternative i.e., ControlNet. This part of the system clearly needs more development in terms of the balance between costs vs. quality before it can be fully integrated.

CHAPTER 6: CONCLUSION (580 WORDS)

6.1 PROJECT SUMMARY & ACHIEVEMENT

Little Legends shows how AI can be meaningfully used to support early childhood development through creative storytelling. The prototype successfully brings together multiple AI models—image understanding, story generation, and speech synthesis—to turn children's drawings into personalized narrated stories, filling a gap in kid-friendly AI tools.

Designed with young users in mind, the project proves that advanced technology can be made accessible, safe, and engaging for preschoolers. The simple interface and thoughtful content design make it easy for even three-year-olds to explore storytelling on their own.

Importantly, the platform puts the child's creativity at the centre. By allowing both digital drawing and image uploads, it supports different levels of comfort with technology while keeping the child's imagination as the heart of each story.

6.2 DEMOCRATIZING AI FOR EARLY LEARNERS VS. ETHICAL CONCERNS

Little Legends contributes to a growing movement toward making artificial intelligence accessible and beneficial for younger users. Traditional AI applications often assume adult users with developed literacy skills and technological familiarity. This project demonstrates that AI can be successfully adapted for pre-literate children, opening new possibilities for early childhood education and development. This accessibility consideration has broader implications for inclusive design in educational technology, suggesting pathways for making advanced AI capabilities available to diverse learners regardless of age, literacy level, or technological background.

However, introducing AI to early childhood raises significant ethical considerations that must be carefully balanced against these benefits. **Privacy concerns are paramount**, as young children cannot meaningfully consent to data collection, requiring robust safeguards to protect their drawings and interactions. **Content safety presents ongoing challenges**, as AI models may inadvertently generate inappropriate imagery or narratives despite safety filters. **The risk of algorithmic bias** in story generation could reinforce harmful stereotypes during crucial developmental years when children are forming their understanding of identity and social roles. Additionally, **concerns about AI dependency** emerge regarding whether such tools might inhibit natural creativity development or create over-reliance on technological assistance. **Long-term developmental impacts remain largely unknown**, as this generation represents the first cohort to engage with advanced AI during early cognitive development, necessitating ongoing research and ethical oversight to ensure these innovations truly serve children's best interests.

6.3 ENHANCING CREATIVE EXPRESSIONS THROUGH EDUCATIONAL TECHNOLOGY

The integration of AI into children's creative processes raises important questions about the role of technology in fostering versus replacing human creativity. Little Legends addresses these concerns by positioning AI as a collaborative partner rather than a replacement for children's imagination. The system amplifies children's creative input by transforming their visual art into rich narrative experiences, potentially inspiring further creative exploration and storytelling development.

This approach suggests a model for AI integration in educational contexts that preserves and enhances human creativity while leveraging technological capabilities to expand expressive possibilities. Rather than generating content independently, the AI serves as an intelligent tool that responds to and builds upon children's original creative work.

Additionally, the project demonstrates significant potential for innovation in early childhood educational technology. Current educational apps for young children often focus on skill drills or passive content consumption. Little Legends represents a different paradigm—one that emphasizes creative production and personalized content generation.

The storytelling focus aligns with established early childhood development priorities, including language acquisition, narrative thinking, and imaginative play. By combining these educational objectives with cutting-edge AI technology, the project suggests new directions for developmentally appropriate educational software that can serve as a partner in human flourishing rather than a replacement for human creativity and connection.

APPENDICES

APPENDIX A: RELATED WORKS

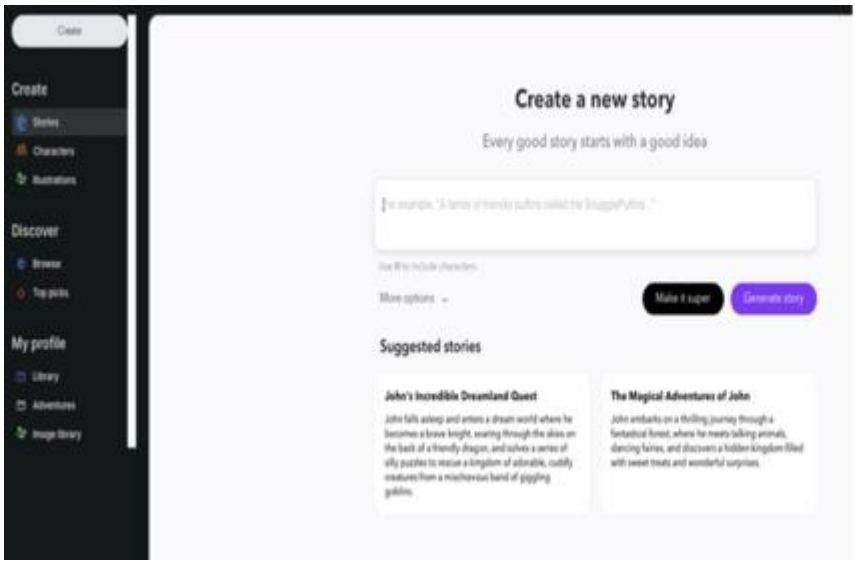
A.1 STORYBEE

Website: https://storybee.app

StoryBee is an AI-powered storytelling platform that allows users to generate personalized children's stories based on input such as the child's name, desired characters, settings, or themes. Designed for parents and educators, the platform produces creative narratives within seconds, often accompanied by illustrations and optional audio narration. While intuitive and fast, StoryBee relies entirely on textual input, making it most suitable for adults or older children who can type and articulate prompts.

A.2 BEDTIMESTORY AI

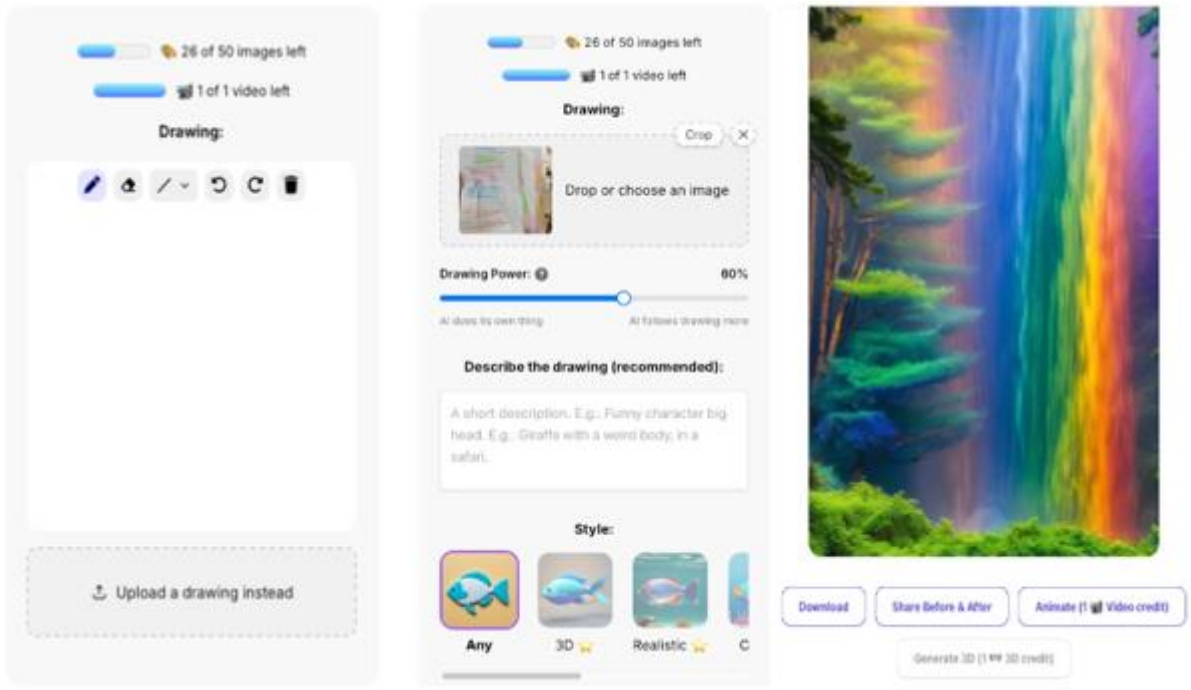
Website: https://www.bedtimestory.ai



Bedtimestory AI is an online platform that instantly generates personalized bedtime stories based on user-inputted keywords such as character names, themes, or morals. The interface is clear and user-friendly, allowing parents or children to quickly type in simple prompts and receive a fully formed, engaging narrative seconds later. While it excels in speed and personalization, the system is limited to text-based interaction—it does not support image uploads, voice input, or child-driven creative assets—making it most appropriate for users comfortable with typing rather than pre-literate young children.

A.3 DRAWINGS ALIVE

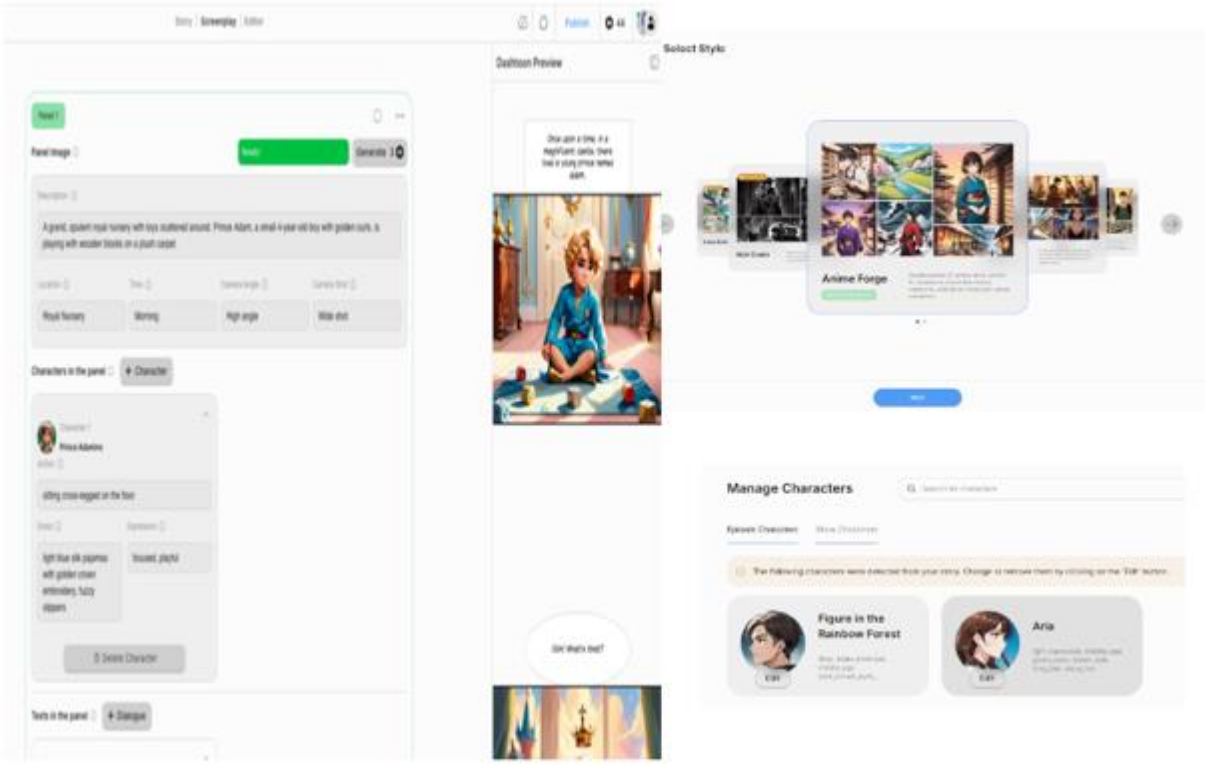
Website: https://drawingsalive.com



Drawings Alive is a web-based AI application that brings children's drawings to life by transforming their simple sketches into vibrant artworks, animated clips, or even 3D models. After uploading a scan or photograph of a drawing (or drawing directly on the website) and optionally providing a short description, users choose from different styles—from realistic renderings to cartoons and 3D animations—and watch the artwork animate within seconds. It emphasizes visual creativity and interactive engagement, though it currently lacks storytelling features, either narrative or voice-over.

A.4 DASHTOON'S AI CHILDREN'S BOOK GENERATOR

Website: https://dashtoon.com/ai-childrens-book-generator



Dashtoon's AI Children's Book Generator is an online platform that quickly produces personalized children's books by letting users input themes, character details, and story elements. The system uses these inputs to generate both narrative and high-quality illustrations, delivering a complete illustrated story within minutes. It features an intuitive interface that allows users to review and customize the content before finalizing a book. However, it relies on textual prompts and does not support children's own drawings or voice inputs.

APPENDIX B: ANALYSIS OF PRE-TRAINED AI MODELS & TECHNOLOGY (PRELIMINARY)

Modality	Technology or AI Model to consider	Reasoning	Approach
Drawing Analysis + Image Captioning (Image Model)	• OpenAI GPT-4.1 nano (prototype – for testing pipeline) • BLIP-2 (via Hugging Face or MiniGPT-4 with BLIP2)	• BLIP-2 is lightweight to run in Colab • Strong at both captioning and concept detection • Free and open-source	Model with higher image analysis accuracy will be used.
Image Generation (Image-to-Image / text-to-image Model)	• DALL-E 3 by OpenAI (ideally – but too costly) • Stable Diffusion (v1.5/v2.1/XL) + ControlNet – Hugging Face (prototype – but may use this if test results are good)	• Open-source (Stable Diffusion) • Free to use locally or on Colab • Good-enough quality images • Customizable via prompt engineering/fine-tuning	Will most likely choose the more cost-effective model if the image quality are acceptable.
Keyword Extraction (Text Model)	• KeyBERT	• Ideal for short prompts • Understands semantic meaning • Suitable for child-like input	Will have to test to see if this is necessary.
Language Generation (prompt engineering) (Text Generation Model)	• OpenAI GPT-4.1 nano (prototype – for testing pipeline) – story generation from image + text prompt • MiniGPT-4	• Already purchased OpenAI's GPT-4.1 nano for prototype testing, stories generated are decent. • MiniGPT-4 is free and open-source, colab-supported	MiniGPT-4 might be more complicated to setup compared to GPT-4.1 nano, in that case may continue with GPT-4.1 nano.
Voice Narration (Text-to-Speech Model)	• Web Speech API – speechSynthesis (prototype – for testing pipeline) • GPT-4o-mini-TTS (can be costly) • Coqui TTS • XTTS v2 (by Coqui) • VITS (Variational Inference TTS)	• Coqui TTS or XTTS v2 are natural and cheerful-sounding • XTTS v2 allows cloning of storyteller's voice • Both are free and open-source • VITS is more resource-intensive; setup is harder than Coqui	Will test to see which model is sounds most suitable and cost-effective

Data Flow between Models – Content Understanding **(Concept List)** **Simple Python Code**	• Google Colab (Python) • Gradio (for quick AI model testing)	• Google Colab to process the AI image generation – free access to a GPU for testing POC & integrates with Google Drive • Gradio to test the image generation model	Will continue testing to see if this combination works with the full-stack development.

APPENDIX C: MAIN TASKS

Architectural Design **(Finalize Wireframe)**	• Conduct further research on UI/UX best practices for early childhood digital products. • Finalize wireframes & frontend design style.
Software Development + **Functional Testing (TDD)**	• Ensure the full-stack architecture is robust, with smooth and reliable communication between the React frontend and Node.js/Express backend. • Finalize the selection of pre-trained AI models for each modality, confirming that they align with the project's goals for story generation, image processing, and voice narration. • Integrate TDD method in the process of testing.
AI Models Testing & Integration	• Compare GPT-4.1-mini and MiniGPT-4 for story generation from image + prompt. • Evaluate Stable Diffusion with ControlNet for image generation in Colab. • Explore KeyBERT for keyword extraction from children's input (if necessary) • Evaluate and decide on the more natural-sounding alternative for final voice narration.
User Testing & Evaluation	• Carry out user acceptance testing (UAT) with parents, educators, and early childhood education (ECE) professionals. • Collect qualitative feedback on usability, story quality, image relevance and voice clarity.
Review & Iterations	• Iterate designs after testing and evaluation to address identified issues.
Final Report & Documentation	• Report writing and design documentation
Final Product Demo (Video)	• Generate final product demo video.

APPENDIX D: PROJECT TIMELINE & MILESTONES

S/N	Task (duration)	Estimated Start Date	Estimated End Date	Dependency & Details
1.	Finalize Wireframe & UI/UX Design (1 week)	16th June	22nd June	Research & collate related websites' good UI/UX designs.
2.	Confirm final models for each AI modality (1 week)	23rd June	29th June	Test & compare each AI pre-trained model for their respective modality.
3.	Integrate final models into Colab pipeline (1 week)	30th June	6th July	Task 2 – AI models confirmation Conduct functional testing to make sure system architecture works.
4.	Improve drawing-to-prompt pipeline & test flow (1 week)	7th July	13th July	Task 2 – AI models confirmation Task 3 – Ensure Colab works Conduct functional testing to ensure system architecture works.
5.	Connect & test full-stack pipeline end-to-end (1 week)	14th July	20th July	Tasks 1, 2 & 3. Conduct functional testing to ensure system architecture works.
6.	Refine story generation + AI image quality (1 week)	21st July	27th July	Task 5 Testing to ensure optimal accuracy and precision.

7.	**Integrate voice narration alternatives (TTS testing)** **(Draft report writing)** **(2 weeks)**	**28th July**	**10th August**	Task 6 Test out different voice narrations according to specific story generated. Start on report draft.
8.	**Conduct UAT with educators/parents** **(1 week) *(submit report draft)**	**11th August**	**17th August**	Conduct user testing and have users fill out performance feedback form.
9.	**Iterations + address issues from UAT evaluations** **(Incorporate feedback+iterations to finalize full report)** **(2 weeks)**	**18th August**	**31st August**	Task 8 Evaluate feedback from functional and user testing + make iterations Begin finalization of Final Report
10.	**Record and edit demo video** **(1 week)**	**1st Sept**	**7th Sept**	Make PowerPoint slides + screen-record to make demo video
11.	**Buffer / Contingency period** **(1 week)**	**8th Sept**	**14th Sept**	NA
Final Project Submission Deadline: 15th Sept 2025				

REFERENCES

JOURNALS & ARTICLES

Bensaid, E., Martino, M., Hoover, B., & Strobelt, H. (2023). *FairyTailor: A multimodal generative framework for storytelling*. IBM Research and MIT CSAIL. https://arxiv.org/abs/2310.04322

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., … & Amodei, D. (2020). *Language models are few-shot learners*. *arXiv preprint arXiv:2005.14165*. https://doi.org/10.48550/arXiv.2005.14165

Castleman, J., & Korolova, A. (2025). *Adultification bias in LLMs and text-to-image models* [Preprint]. *arXiv*. https://doi.org/10.48550/arXiv.2506.07282

Chen, G. Z. (2024). The impacts of AI-driven storytelling applications on language acquisition and literacy development in early childhood education: A systematic review. *Asian Journal of Advanced Research and Reports, 18*, 1–12. https://doi.org/10.9734/ajarr/2024/v18i11770

Chen, Y. (2021). *An automatic storytelling system based on natural language processing* [Master's thesis, Uppsala University]. DiVA portal.

Clark, E., Ji, Y., & Smith, N. A. (2018). Neural text generation in stories using entity representations as context. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2250–2260. https://aclanthology.org/N18-1204/

Heeg, D. M., & Avraamidou, L. (2024). Young children's understanding of AI. *Education and Information Technologies*. Advance online publication. https://doi.org/10.1007/s10639-024-13169-x

Hu, T., Kyrychenko, Y., Rathje, S., Collier, N., van der Linden, S., & Roozenbeek, J. (2025). *Generative language models exhibit social identity biases*. *Nature Computational Science, 5*(1), 65–75. https://doi.org/10.1038/s43588-024-00741-1

Kotek, H., Dockum, R., & Sun, D. Q. (2023). *Gender bias and stereotypes in Large Language Models*. *ACM Conference on Collective Intelligence*. https://doi.org/10.1145/3582269.3615599

Kucirkova, N. (2019). *Children's agency by design: Design parameters for personalization in story-making apps*. *International Journal of Child-Computer Interaction, 21*, 100157. https://doi.org/10.1016/j.ijcci.2019.06.003

Kim, J., Heo, Y., Yu, H., & Nang, J. (2023). A multi-modal story generation framework with AI-driven storyline guidance. *Electronics, 12*(6), Article 1289. https://doi.org/10.3390/electronics12061289

Li, S., Zhao, S., Yu, W., Sun, W., Metaxas, D., Loy, C. C., & Liu, Z. (2021). *Deep animation video interpolation in the wild* (arXiv preprint arXiv:2104.02495). https://doi.org/10.48550/arXiv.2104.02495

Lum, K., Anthis, J. R., Nagpal, C., Robinson, K., & D'Amour, A. (2024). *Bias in Language Models: Beyond Trick Tests and Toward RUTEd Evaluation*. *arXiv*. https://doi.org/10.48550/arXiv.2402.12649

Philips, L. M. (1999). *The role of storytelling in early literacy development*. *Reading Psychology, 20*(1), 1–22. https://doi.org/10.1080/027027199278484

Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). *Hierarchical text-conditional image generation with CLIP latents*. arXiv. https://arxiv.org/abs/2204.06125

Read, J. C., & Bekker, M. M. (2011). The nature of child computer interaction. *Proceedings of the 25th BCS Conference on Human-Computer Interaction (HCI 2011)*, 163–170. https://doi.org/10.14236/ewic/HCI2011.43

Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E., Seyed Ghasemipour, S. K., Karagol Ayan, B., Mahdavi, S. S., Gontijo Lopes, R., Salimans, T., Ho, J., Fleet, D. J., & Norouzi, M. (2022). *Photorealistic text-to-image diffusion models with deep language understanding*. arXiv. https://arxiv.org/abs/2205.11487

Slobodianiuk, A. V., & Semerikov, S. O. (2025). Advances in neural text generation: A systematic review (2022–2024). *Journal of Computational Methods in Sciences and Engineering*, *24*(5)

Su, J., & Yang, W. (2022). Artificial intelligence in early childhood education: A scoping review. *Computers and Education: Artificial Intelligence, 3*, 100049. https://doi.org/10.1016/j.caeai.2022.100049

Tengku Hariffadzillah, T. S. N., Mohd Fadil, M. Z., Harun, A., Ayob, M., & Razak, M. (2023). Exploring AI technology to create children's illustrated storybook. *Proceedings of the 2023 International Conference on Artificial Intelligence and Data Analytics for Smart Systems (AiDAS)*, 224–228. https://doi.org/10.1109/AiDAS60501.2023.10284665

Time Magazine. (2023, April 5). *AI children's book 'Alice and Sparkle' sparks backlash from artists*. *Time*. Retrieved from https://time.com/6240569/ai-childrens-book-alice-and-sparkle-artists-unhappy/

Umek, L. M., Fekonja-Peklaj, U., & Podlesek, A. (2012). Parental influence on the development of children's storytelling. *European Early Childhood Education Research Journal, 20*(2), 213–227. https://doi.org/10.1080/1350293X.2012.704760

Xu, J., Ren, X., Zhang, Y., Zeng, Q., Cai, X., & Sun, X. (2018). A skeleton-based model for promoting coherence among sentences in narrative story generation. *Proceedings of the*

2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), 4306–4315. https://aclanthology.org/D18-1462/

Yan, D., Zhang, W., Zhang, L., Kalia, A., Wang, D., Ramchandani, A., Liu, M., Pumarola, A., Schönfeld, E., Blanchard, E., Narni, K., Luo, Y., Chen, L., Pang, G., Thabet, A., Vajda, P., Bearman, A., & Yu, L. (2024). *Animated Stickers: Bringing Stickers to Life with Video Diffusion* (arXiv preprint arXiv:2402.06088). https://doi.org/10.48550/arXiv.2402.06088

Yang, Y. (2025). *Racial bias in AI-generated images*. *AI & Society, 40*(2), 123–135. https://doi.org/10.1007/s00146-025-02282-1

Zheng X, Zhang Z, Wang L, Wu J, Dong Z. Investigation on text generative model based on deep learning in natural language processing. *Journal of Computational Methods in Sciences and Engineering*. 2024;24(6):4089-4100. doi:10.1177/14727978241299204

RELATED WORKS - WEBSITES

StoryBee. (n.d.). *Create magical bedtime stories with AI*. https://www.storybee.ai/

Bedtimestory AI. (n.d.). *Personalized bedtime stories powered by AI*. https://www.mybedtimestory.ai/

Drawings Alive. (n.d.). *Bring children's drawings to life with animation*. https://www.drawingsalive.com/

Dashtoon. (n.d.). *Create your own illustrated children's book with AI*. https://www.dashtoon.com/

CODE

BACKEND

**** generate_instructions.js ****

```
import fs from 'fs';

import axios from 'axios';


const instructions = [

  "Draw your picture on the white canvas! Use any colors you like! You can use the gray eraser button if you made a mistake, or press the brown clear canvas button to start over again. After you are done, press button number 2!",

  "Once you have finished drawing, press the green OK button below two times to create your story! It will take a while, so be patient and play the mini games while you wait!",
```

```
  "Press the big yellow button below to listen to your magical story! It
will take a while again, so be patient and play the mini games while the
magic happens! Once you are done listening to the story, press the red
button below to draw a new picture and make a new story!"

];


async function generateInstructionAudios() {

  for (let i = 0; i < instructions.length; i++) {

    try {

      const response = await axios({

        method: 'post',

        url: 'http://localhost:5001/generate-speech',

        data: { text: instructions[i] },

        responseType: 'arraybuffer'

      });


      fs.writeFileSync(`./public/audio/instruction${i + 1}.wav`,
response.data);

      console.log(`Generated instruction${i + 1}.wav`);

    } catch (error) {

      console.error(`Error generating instruction${i + 1}:`,
error.message);

    }

  }

}


generateInstructionAudios();
```

** image_service.py **

```
from flask import Flask, request, send_file, jsonify
from flask_cors import CORS
import torch
from diffusers import StableDiffusionControlNetPipeline, ControlNetModel
from controlnet_aux import HEDdetector
from PIL import Image, ImageOps
import time
import os
import io
import numpy as np

app = Flask(__name__)
CORS(app)

print("Loading ControlNet models...")
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

try:
    # Load ControlNet Scribble model (better for understanding context)
    controlnet = ControlNetModel.from_pretrained(
        "lllyasviel/sd-controlnet-scribble",
        torch_dtype=torch.float16 if device == "cuda" else torch.float32
    )

    # Load Stable Diffusion pipeline with ControlNet
    pipe = StableDiffusionControlNetPipeline.from_pretrained(
        "runwayml/stable-diffusion-v1-5",
        controlnet=controlnet,
        torch_dtype=torch.float16 if device == "cuda" else torch.float32,
        safety_checker=None,
        requires_safety_checker=False
```

```
    ).to(device)


    # Load HED detector for softer edge detection (better than Canny for
scribbles)
    try:
        hed_detector = HEDdetector.from_pretrained('lllyasviel/Annotators')
        print("Using HED detector for scribble processing")
    except:
        print("HED detector not available, will use direct image
processing")
        hed_detector = None


    print("ControlNet Scribble loaded successfully!")


except Exception as e:
    print(f"Error loading models: {e}")
    pipe = None
    hed_detector = None


def process_drawing_for_scribble(image):
    """Convert drawing to scribble format - softer than Canny edges"""
    try:
        if hed_detector is not None:
            # Use HED detector for softer, more natural edges
            scribble_image = hed_detector(image)
        else:
            # Fallback: convert to grayscale and enhance edges
            # This works well for child drawings
            gray = image.convert('L')
            # Invert if needed (make dark lines on white background)
            gray_array = np.array(gray)

            # If the image is mostly dark, invert it
            if np.mean(gray_array) < 127:
```

```
                gray = ImageOps.invert(gray)

            # Convert back to RGB format for ControlNet
            scribble_image = gray.convert('RGB')

        return scribble_image

    except Exception as e:
        print(f"Error processing drawing: {e}")
        # Ultimate fallback: return the original image
        return image

@app.route('/transform-drawing', methods=['POST'])
def transform_drawing():
    try:
        if pipe is None:
            return jsonify({'error': 'Models not loaded'}), 500

        # Handle both form data and JSON input
        if request.content_type and 'application/json' in
request.content_type:
            # JSON input with base64 image
            data = request.get_json()
            image_b64 = data.get('image')
            prompt = data.get('prompt', 'colorful illustration')

            # Decode base64 image
            import base64
            image_data = base64.b64decode(image_b64.split(',')[1] if ',' in
image_b64 else image_b64)
            image = Image.open(io.BytesIO(image_data)).convert('RGB')

        else:
            # Form data input (original method)
```

```
            if 'image' not in request.files:
                return jsonify({'error': 'No image provided'}), 400

            image_file = request.files['image']
            prompt = request.form.get('prompt', 'colorful illustration')
            image = Image.open(image_file.stream).convert('RGB')

        print(f"Processing image with prompt: {prompt}")

        # Resize for optimal processing
        image = image.resize((512, 512))

        # Process drawing for scribble control (softer than Canny)
        scribble_image = process_drawing_for_scribble(image)

        # Create enhanced prompt for children's book style
        # More contextual prompt that helps with understanding what the
drawing represents
        full_prompt = f"beautiful children's book illustration of {prompt},
colorful, whimsical, cartoon style, high quality digital art, vibrant
colors, friendly, imaginative scene"
        negative_prompt = "ugly, scary, dark, realistic, violent, nsfw,
blurry, low quality, adult themes, abstract, watercolor smudges"

        print("Generating image...")

        # Generate image with ControlNet Scribble
        result = pipe(
            prompt=full_prompt,
            image=scribble_image,
            negative_prompt=negative_prompt,
            num_inference_steps=10,
            guidance_scale=7.5,
            controlnet_conditioning_scale=0.7,  # Slightly lower for more
creative freedom
```

```
            generator=torch.Generator(device=device).manual_seed(42)
        ).images[0]

        # Save generated image
        timestamp = int(time.time())
        filename = f"transformed_{timestamp}.png"
        filepath = os.path.join(os.getcwd(), filename)
        result.save(filepath)

        print(f"Image generated successfully: {filename}")

        # Return the generated image
        return send_file(filepath, mimetype='image/png',
as_attachment=False)

    except Exception as e:
        print(f"Error generating image: {str(e)}")
        return jsonify({'error': f'Image generation failed: {str(e)}'}),
500

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({
        'status': 'ControlNet Scribble service running',
        'device': device,
        'models_loaded': pipe is not None,
        'controlnet_type': 'scribble',
        'torch_version': torch.__version__
    })

if __name__ == '__main__':
    print("Starting ControlNet Scribble Image Service on
http://localhost:5002")
    app.run(host='0.0.0.0', port=5002, debug=False)
```

** index.js **

```
import fs from 'fs';
import path from 'path';
import fetch from 'node-fetch';
import express from 'express';
import multer from 'multer';
import cors from 'cors';
import dotenv from 'dotenv';
import axios from 'axios';
import { OpenAI } from 'openai';
import { HfInference } from '@huggingface/inference';

// To use the .env file
dotenv.config();

// Config
const port = process.env.PORT || 1337;

// Setting up express app
const app = express();
app.use(cors({
  origin: 'http://localhost:5173',
  methods: ['GET', 'POST', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// OpenAI API connector
const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
});
```

```
// Hugging Face API connector
const hf = new HfInference(process.env.HF_API_TOKEN);


// Create output directory for generated images
const OUTPUT_DIR = path.join(process.cwd(), 'generated_images');
if (!fs.existsSync(OUTPUT_DIR)) {
  fs.mkdirSync(OUTPUT_DIR, { recursive: true });
}


// Serve generated images
app.use('/generated', express.static(OUTPUT_DIR));


// Serve audio files
app.use('/audio', express.static(path.join(process.cwd(),
'public/audio')));


// Used to handle uploaded files
const upload = multer({ storage: multer.memoryStorage() });


// Helper function to generate image with local ControlNet service (with
conditioning control)
async function generateImageWithControlNet(imageBuffer, prompt,
conditioningScale = 0.7) {
  try {
    console.log('Generating image with local ControlNet...');
    console.log('Prompt:', prompt || 'no prompt provided');
    console.log('Conditioning scale:', conditioningScale);


    // Convert image buffer to base64
    const base64Image = imageBuffer.toString('base64');
    const dataUri = `${base64Image}`;


    // Send as JSON instead of form data
```

```
    const response = await axios({
      method: 'post',
      url: 'http://localhost:5002/transform-drawing',
      data: {
        image: dataUri,
        prompt: prompt || 'colorful illustration',
        conditioning_scale: conditioningScale  // Pass conditioning scale
to ControlNet service
      },
      headers: {
        'Content-Type': 'application/json'
      },
      responseType: 'arraybuffer',
      timeout: 480000  // 8-minute timeout
    });

    // Save the generated image locally
    const timestamp = Date.now();
    const filename = `controlnet_${timestamp}.png`;
    const filepath = path.join(OUTPUT_DIR, filename);

    fs.writeFileSync(filepath, Buffer.from(response.data));

    console.log('Image generated successfully with ControlNet:', filename);
    return `http://localhost:${port}/generated/${filename}`;

  } catch (error) {
    console.error('ControlNet generation failed:', error.message);

    // Fallback: try Hugging Face if local service fails
    try {
      console.log('Trying Hugging Face fallback...');
      const result = await hf.textToImage({
```

```
        model: 'stabilityai/stable-diffusion-xl-base-1.0',
        inputs: `children's book illustration, ${prompt}, colorful,
whimsical, cartoon style`
      });

      const timestamp = Date.now();
      const filename = `fallback_${timestamp}.png`;
      const filepath = path.join(OUTPUT_DIR, filename);

      const buffer = Buffer.from(await result.arrayBuffer());
      fs.writeFileSync(filepath, buffer);

      console.log('Fallback image generated:', filename);
      return `http://localhost:${port}/generated/${filename}`;

    } catch (fallbackError) {
      console.error('All image generation failed:', fallbackError);
      return 'https://via.placeholder.com/512?text=AI+Image+Not+Available';
    }
  }
}

// Helper function for pure Hugging Face image generation (for "Surprise
me!")
async function generateImageWithHuggingFace(prompt) {
  try {
    console.log('Generating image with Hugging Face (Surprise me
mode)...');

    const result = await hf.textToImage({
      model: 'stabilityai/stable-diffusion-xl-base-1.0',
      inputs: `professional children's book illustration, ${prompt},
colorful, whimsical, cartoon style, friendly characters, high quality
digital art, storybook style`,
      parameters: {
```

```
        timeout: 60000  // 60 seconds
      }
    });

    const timestamp = Date.now();
    const filename = `surprise_${timestamp}.png`;
    const filepath = path.join(OUTPUT_DIR, filename);

    const buffer = Buffer.from(await result.arrayBuffer());
    fs.writeFileSync(filepath, buffer);

    console.log('Surprise me image generated:', filename);
    return `http://localhost:${port}/generated/${filename}`;

  } catch (error) {
    console.error('Hugging Face image generation failed:', error);
    return 'https://via.placeholder.com/512?text=AI+Image+Not+Available';
  }
}

// Helper function for text-only image generation
async function generateImageFromTextOnly(prompt) {
  try {
    console.log('Generating image from text prompt only...');

    const result = await hf.textToImage({
      model: 'stabilityai/stable-diffusion-xl-base-1.0',
      inputs: `children's book illustration, ${prompt}, colorful,
whimsical, cartoon style, friendly characters`
    });

    const timestamp = Date.now();
    const filename = `text_only_${timestamp}.png`;
```

```
    const filepath = path.join(OUTPUT_DIR, filename);

    const buffer = Buffer.from(await result.arrayBuffer());
    fs.writeFileSync(filepath, buffer);

    console.log('Text-only image generated:', filename);
    return `http://localhost:${port}/generated/${filename}`;

  } catch (error) {
    console.error('Text-only image generation failed:', error);
    return 'https://via.placeholder.com/512?text=AI+Image+Not+Available';
  }
}

// Smart function to extract story elements for better image prompts
async function extractStoryElementsForImage(story) {
  try {
    const response = await openai.chat.completions.create({
      model: "gpt-4.1-mini",
      messages: [{
        role: "user",
        content: `Extract the main characters, setting, and key visual
elements from this children's story for creating an illustration. Be very
specific about shapes and forms - if characters are stars, say "star-shaped
characters" not just "stars". If they are trees, say "tree-shaped
characters", etc. Return only a brief description suitable for an image
generator:

Story: ${story}

Format: "main character(s) with specific shapes in [setting], [key visual
elements], children's book illustration style"`
      }],
      max_tokens: 50
    });
```

```
    const imagePrompt = response.choices[0].message.content.trim();
    console.log('Extracted story elements for image:', imagePrompt);
    return imagePrompt;

  } catch (error) {
    console.error('Error extracting story elements:', error);
    // Fallback to generic prompt
    return 'colorful children\'s book illustration, whimsical characters,
magical adventure';
  }
}

// The generate endpoint that produces the story based on the prompt and/or
drawing
app.post('/generate', upload.single('drawing'), async (req, res) => {
  const file = req.file;
  const prompt = req.body.prompt;
  const stylePreference = req.body.stylePreference || 'make-prettier'; //
DEFAULT: make-prettier

  console.log('Style preference received:', stylePreference);

  // Check if we have at least image OR text
  if (!file && !prompt) {
    return res.status(400).json({ error: 'Please provide either a drawing
or text prompt if you have not' });
  }

  // Get base64 string from the file buffer (if file exists)
  let dataUri = null;
  if (file) {
    const base64Image = file.buffer.toString('base64');
    const mimeType = file.mimetype;
    dataUri = `data:${mimeType};base64,${base64Image}`;
```

```
  }


  // Smart message creation based on what the child provides
  let messages;
  let scenario = '';


  if (file && prompt) {
    // Case 1: Both image and text - use both for rich storytelling
    scenario = 'image_and_text';
    messages = [{
      role: "user",
      content: [
        {
          type: "text",
          text: `Generate a children's story based on this drawing and the
prompt: '${prompt}'. Make it engaging and age-appropriate for
preschoolers.`
        },
        {
          type: "image_url",
          image_url: { url: dataUri }
        }
      ]
    }];
  } else if (file && !prompt) {
    // Case 2: Image only - focus entirely on the drawing
    scenario = 'image_only';
    messages = [{
      role: "user",
      content: [
        {
          type: "text",
          text: "Generate a creative children's story based entirely on
this drawing. Use your imagination to create characters, plot, and
```

```
adventure from what you see. Make it engaging and age-appropriate for
preschoolers."

        },

        {

          type: "image_url",

          image_url: { url: dataUri }

        }

      ]

    }];

  } else {

    // Case 3: Text only - for shy kids who prefer not to draw

    scenario = 'text_only';

    messages = [{

      role: "user",

      content: `Generate a creative children's story based on this prompt:
'${prompt}'. Create interesting characters, settings, and adventures. Make
it engaging and age-appropriate for preschoolers.`

    }];

  }


  console.log(`Story generation scenario: ${scenario}`);


  // Call OpenAI to get the story

  try {

    const response = await openai.chat.completions.create({

      model: "gpt-4.1-mini",

      messages: messages,

      max_tokens: 800  // 800 for longer stories

    });


    if (!response || !response.choices || !response.choices[0]) {

      console.error("Invalid OpenAI response format:", response);

      return res.status(500).json({ error: "Invalid response from
OpenAI" });

    }
```

```
    const story = response.choices[0].message.content;


    // Generate AI image based on style preference
    let aiImageUrl = null;
    try {
      console.log('Starting image generation with style preference:',
stylePreference);


      if (file) {
        // Has drawing - route based on style preference
        switch (stylePreference) {
          case 'keep-drawing':
            console.log('Keep drawing mode - High ControlNet
conditioning');
            aiImageUrl = await generateImageWithControlNet(file.buffer,
prompt, 0.9); // High conditioning = stick to drawing
            break;


          case 'make-prettier':
            console.log('Make prettier mode - Medium ControlNet
conditioning');
            aiImageUrl = await generateImageWithControlNet(file.buffer,
prompt, 0.5); // Medium conditioning = balanced
            break;


          case 'surprise-me':
            console.log('Surprise me mode - Pure Hugging Face with story
elements');
            // Extract key elements from the generated story for better
image prompts
            const storyElements = await
extractStoryElementsForImage(story);
            aiImageUrl = await generateImageWithHuggingFace(storyElements);
            break;
```

```
          default:
            console.log('Default to make prettier mode');
            aiImageUrl = await generateImageWithControlNet(file.buffer,
prompt, 0.5);
        }
      } else {
        // Text-only - always use Hugging Face regardless of preference
        console.log('Text-only mode - Using Hugging Face');
        aiImageUrl = await generateImageFromTextOnly(prompt);
      }
    } catch (imgError) {
      console.error("Image generation error:", imgError);
      aiImageUrl =
'https://via.placeholder.com/512?text=AI+Image+Generation+Failed';
    }

    res.json({
      story,
      aiImageUrl,
      scenario: scenario,
      styleUsed: stylePreference  // Let frontend know which style was used
    });
  } catch (error) {
    console.error('OpenAI error:', error);
    res.status(500).json({ error: 'Failed to generate story' });
  }
});

// Text-to-Speech endpoint using XTTS-v2 with voice selection support
app.post('/tts', async (req, res) => {
  try {
    const { text, voice_id } = req.body;

    if (!text) {
```

```
      return res.status(400).json({ error: 'No text provided' });
    }

    console.log("Generating speech for text:", text.substring(0, 50) +
"...");
    if (voice_id) {
      console.log("Using custom voice_id:", voice_id);
    } else {
      console.log("Using default voice");
    }

    // Prepare request data for Python TTS service
    const requestData = { text: text };

    // Add voice_id only if it's provided (for custom voice)
    if (voice_id) {
      requestData.voice_id = voice_id;
    }

    // Make request to Python TTS service
    const ttsResponse = await axios({
      method: 'post',
      url: 'http://localhost:5001/generate-speech',
      data: requestData,
      responseType: 'stream',
      headers: {
        'Content-Type': 'application/json'
      }
    });

    // Set appropriate headers for audio streaming
    res.set({
      'Content-Type': 'audio/wav',
```

```
      'Content-Disposition': 'attachment; filename="story_audio.wav"',
      'Cache-Control': 'no-cache'
    });

    // Stream the audio data back to the client
    ttsResponse.data.pipe(res);

  } catch (error) {
    console.error("Error generating speech:", error);

    if (error.response) {
      console.error("TTS service error:", error.response.status,
error.response.data);
    }

    if (!res.headersSent) {
      res.status(500).json({ error: "Failed to generate speech" });
    }
  }
});

// Check TTS service health
app.get('/tts/health', async (req, res) => {
  try {
    const healthResponse = await axios.get('http://localhost:5001/health');
    res.json({
      ttsService: healthResponse.data,
      status: "TTS service is accessible"
    });
  } catch (error) {
    console.error("TTS service health check failed:", error.message);
    res.status(500).json({
      error: "TTS service is not accessible",
```

```
      details: error.message
    });
  }
});

// Check Hugging Face API health
app.get('/hf/health', async (req, res) => {
  try {
    // Test with a simple prompt using a supported model
    await hf.textToImage({
      model: 'CompVis/stable-diffusion-v1-4',
      inputs: 'test image'
    });

    res.json({
      status: "Hugging Face API is accessible",
      availableModels: [
        "stabilityai/stable-diffusion-xl-base-1.0",
        "stabilityai/stable-diffusion-2-1",
        "CompVis/stable-diffusion-v1-4"
      ]
    });
  } catch (error) {
    console.error("Hugging Face API health check failed:", error.message);
    res.status(500).json({
      error: "Hugging Face API is not accessible",
      details: error.message
    });
  }
});

// Error handling middleware
app.use((error, req, res, next) => {
```

```
  console.error("Unhandled error:", error);

  res.status(500).json({ error: "Internal server error" });

});


// Start server

app.listen(port, () => {

  console.log(`Little Legends API server running on
http://localhost:${port}`);

  console.log("Endpoints available:");

  console.log("- POST /generate (story generation)");

  console.log("- POST /tts (text-to-speech)");

  console.log("- GET /tts/health (check TTS service)");

  console.log("- GET /hf/health (check Hugging Face API)");

  console.log("- GET /generated/:filename (serve generated images)");

  console.log("");

  console.log("Smart Style System:");

  console.log("Keep drawing (90% ControlNet)");

  console.log("Make prettier (50% ControlNet)");

  console.log("Surprise me! (100% Hugging Face + Shape-Specific Story
Elements)");

});
```

** tts_service.py **

```
from flask import Flask, request, jsonify, send_file
from flask_cors import CORS
import torch
from TTS.api import TTS
import io
import os
import tempfile
import uuid
from pydub import AudioSegment

app = Flask(__name__)
CORS(app)

print("Loading XTTS v2 model...")
device = "cuda" if torch.cuda.is_available() else "cpu"

# Load XTTS v2 - supports voice cloning
tts = TTS("tts_models/multilingual/multi-dataset/xtts_v2").to(device)
print(f"XTTS v2 model loaded on {device}")

# Store for voice references (in production, use proper database)
voice_references = {}

# Create a default speaker reference using XTTS v2's built-in speaker
DEFAULT_SPEAKER = "Ana Florence"  # XTTS v2 built-in speaker

@app.route('/upload-voice', methods=['POST'])
def upload_voice():
    """Upload a reference voice sample for cloning"""
    try:
        if 'voice_file' not in request.files:
            return jsonify({'error': 'No voice file provided'}), 400
```

```
        voice_file = request.files['voice_file']
        voice_id = str(uuid.uuid4())

        # Check if file is already WAV format
        if voice_file.filename.endswith('.wav'):
            # File is already WAV, save directly
            voice_path = f"voice_ref_{voice_id}.wav"
            voice_file.save(voice_path)
            print(f"WAV file saved directly: {voice_path}")
        else:
            # File is WebM, need to convert to WAV
            temp_webm_path = f"temp_voice_{voice_id}.webm"
            voice_file.save(temp_webm_path)

            # Convert WebM to WAV using pydub
            try:
                print(f"Converting WebM to WAV: {temp_webm_path}")
                audio = AudioSegment.from_file(temp_webm_path,
format="webm")
                voice_path = f"voice_ref_{voice_id}.wav"  # Save as WAV
                audio.export(voice_path, format="wav")

                # Clean up temporary webm file
                os.remove(temp_webm_path)
                print(f"Successfully converted and saved: {voice_path}")

            except Exception as conversion_error:
                print(f"Audio conversion error: {conversion_error}")
                # Clean up on error
                if os.path.exists(temp_webm_path):
                    os.remove(temp_webm_path)
                return jsonify({'error': 'Failed to convert audio
format'}), 500
```

```
        # Store reference for later use
        voice_references[voice_id] = voice_path

        return jsonify({
            'success': True,
            'voice_id': voice_id,
            'message': 'Voice reference uploaded and processed
successfully!'
        })

    except Exception as e:
        print(f"Error uploading voice: {str(e)}")
        return jsonify({'error': str(e)}), 500

@app.route('/generate-speech', methods=['POST'])
def generate_speech():
    """Generate speech with optional voice cloning"""
    try:
        data = request.json
        text = data.get('text', '')
        voice_id = data.get('voice_id', None)  # Optional voice cloning

        if not text:
            return jsonify({'error': 'No text provided'}), 400

        audio_id = str(uuid.uuid4())
        output_path = f"temp_audio_{audio_id}.wav"

        if voice_id and voice_id in voice_references:
            # Use voice cloning
            reference_path = voice_references[voice_id]
            print(f"Generating speech with voice cloning using
{reference_path}")
```

```
            tts.tts_to_file(
                text=text,
                file_path=output_path,
                speaker_wav=reference_path,
                language="en"
            )
        else:
            # Use built-in XTTS v2 speaker for default voice
            print("Generating speech with built-in XTTS v2 speaker")
            tts.tts_to_file(
                text=text,
                file_path=output_path,
                speaker=DEFAULT_SPEAKER,
                language="en"
            )

        try:
            return send_file(
                output_path,
                mimetype='audio/wav',
                as_attachment=True,
                download_name=f'story_audio_{audio_id}.wav'
            )
        finally:
            # Clean up
            try:
                if os.path.exists(output_path):
                    os.remove(output_path)
            except Exception as cleanup_error:
                print(f"Warning: Could not clean up {output_path}:
{cleanup_error}")
```

```
    except Exception as e:
        print(f"Error generating speech: {str(e)}")
        return jsonify({'error': str(e)}), 500


@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({
        'status': 'XTTS v2 service is running!',
        'model': 'XTTS v2',
        'voice_references': len(voice_references),
        'default_speaker': DEFAULT_SPEAKER
    })


if __name__ == '__main__':
    print("Starting XTTS v2 TTS service on http://localhost:5001")
    app.run(host='0.0.0.0', port=5001, debug=True)
```

FRONTEND

**** interactiveWaitingComponent.jsx ****

```
import React, { useState, useEffect } from 'react';


const InteractiveWaitingComponent = ({ message = "Creating your magical
story", type = "story" }) => {
  const [currentActivity, setCurrentActivity] = useState(0);
  const [clickCount, setClickCount] = useState(0);
  const [stars, setStars] = useState([]);
  const [balloonSize, setBalloonSize] = useState(50);
  const [memoryColors, setMemoryColors] = useState([]);
  const [showMemoryColors, setShowMemoryColors] = useState(true);
  const [hiddenStarPosition, setHiddenStarPosition] = useState({ x: 50, y:
50 });
  const [showHiddenStar, setShowHiddenStar] = useState(true);
```

```
  const activities = [
    { name: "pencil", duration: 8000, message: "Help the magic pencil draw
by clicking it!" },
    { name: "colors", duration: 6000, message: "Keep the story warm by
pressing the colorful buttons!" },
    { name: "stars", duration: 10000, message: "Count the twinkling stars
while your story is being made!" },
    { name: "memory", duration: 8000, message: "Remember the colors! What
was the first color?" },
    { name: "hiddenStar", duration: 6000, message: "Find the hidden star by
clicking around!" },
    { name: "balloon", duration: 7000, message: "Blow to help your story
grow! Click the balloon!" }
  ];


  // Rotate activities every few seconds
  useEffect(() => {
    const interval = setInterval(() => {
      setCurrentActivity((prev) => (prev + 1) % activities.length);
      setClickCount(0);
      setBalloonSize(50);
      setStars([]);
    }, activities[currentActivity].duration);

    return () => clearInterval(interval);
  }, [currentActivity]);


  // Generate memory colors when activity starts
  useEffect(() => {
    if (activities[currentActivity].name === "memory") {
      const colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4',
'#FFEAA7'];
      const selectedColors = colors.slice(0, 3).map((color, index) => ({
        color,
        id: index
      }));
```

```
      setMemoryColors(selectedColors);
      setShowMemoryColors(true);

      setTimeout(() => setShowMemoryColors(false), 3000);
    }
  }, [currentActivity]);

  // Hidden star positioning
  useEffect(() => {
    if (activities[currentActivity].name === "hiddenStar") {
      const interval = setInterval(() => {
        setHiddenStarPosition({
          x: Math.random() * 80 + 10,
          y: Math.random() * 60 + 20
        });
        setShowHiddenStar(prev => !prev);
      }, 1500);

      return () => clearInterval(interval);
    }
  }, [currentActivity]);

  const handlePencilClick = () => {
    setClickCount(prev => prev + 1);
  };

  const handleColorClick = (colorIndex) => {
    setClickCount(prev => prev + 1);
  };

  const handleStarCreate = () => {
    const newStar = {
      id: Date.now(),
```

```
      x: Math.random() * 90 + 5,
      y: Math.random() * 70 + 15
    };
    setStars(prev => [...prev, newStar]);
  };

  const handleBalloonClick = () => {
    setBalloonSize(prev => Math.min(prev + 10, 120));
    setClickCount(prev => prev + 1);
  };

  const handleHiddenStarClick = () => {
    setClickCount(prev => prev + 1);
  };

  const renderCurrentActivity = () => {
    const activity = activities[currentActivity];

    switch (activity.name) {
      case "pencil":
        return (
          <div style={{ textAlign: "center", padding: "2rem" }}>
            <div
              onClick={handlePencilClick}
              style={{
                fontSize: "4rem",
                cursor: "pointer",
                display: "inline-block",
                animation: "bounce 2s infinite",
                transform: `scale(${1 + clickCount * 0.1})`,
                transition: "transform 0.2s"
              }}
            >
```

```
            </div>
            <p style={{ marginTop: "1rem", fontSize: "1.2rem", color:
"#4ECDC4" }}>
              Clicks: {clickCount}
            </p>
          </div>
        );


      case "colors":
        const buttonColors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4',
'#FFEAA7'];
        return (
          <div style={{ textAlign: "center", padding: "2rem" }}>
            <div style={{ display: "flex", gap: "1rem", justifyContent:
"center", flexWrap: "wrap" }}>
              {buttonColors.map((color, index) => (
                <button
                  key={index}
                  onClick={() => handleColorClick(index)}
                  style={{
                    width: "60px",
                    height: "60px",
                    backgroundColor: color,
                    border: "3px solid white",
                    borderRadius: "50%",
                    cursor: "pointer",
                    animation: `pulse 1.5s infinite ${index * 0.3}s`,
                    transform: `scale(${clickCount > index ? 1.2 : 1})`,
                    transition: "transform 0.3s"
                  }}
                />
              ))}
            </div>
```

```
            <p style={{ marginTop: "1rem", fontSize: "1.2rem", color:
"#FF6B6B" }}>
              Buttons warmed: {clickCount}
            </p>
          </div>
        );


      case "stars":
        return (
          <div
            style={{
              position: "relative",
              height: "300px",
              cursor: "pointer",
              backgroundColor: "#1a1a2e",
              borderRadius: "12px",
              overflow: "hidden"
            }}
            onClick={handleStarCreate}
          >
            {stars.map((star) => (
              <div
                key={star.id}
                style={{
                  position: "absolute",
                  left: `${star.x}%`,
                  top: `${star.y}%`,
                  fontSize: "2rem",
                  animation: "twinkle 2s infinite"
                }}
              >
                ⭐
              </div>
```

```
            ))}
            <div style={{
              position: "absolute",
              bottom: "20px",
              left: "50%",
              transform: "translateX(-50%)",
              color: "white",
              fontSize: "1.2rem"
            }}>
              Stars counted: {stars.length}
            </div>
          </div>
        );

      case "memory":
        return (
          <div style={{ textAlign: "center", padding: "2rem" }}>
            {showMemoryColors ? (
              <div>
                <p style={{ fontSize: "1.1rem", marginBottom:
"1rem" }}>Remember these colors!</p>
                <div style={{ display: "flex", gap: "1rem", justifyContent:
"center" }}>
                  {memoryColors.map((item) => (
                    <div
                      key={item.id}
                      style={{
                        width: "80px",
                        height: "80px",
                        backgroundColor: item.color,
                        borderRadius: "50%",
                        border: "3px solid white"
                      }}
                    />
```

```
                ))}
              </div>
            </div>
          ) : (
            <div>
              <p style={{ fontSize: "1.1rem", marginBottom:
"1rem" }}>What was the FIRST color?</p>
              <div style={{ display: "flex", gap: "1rem", justifyContent:
"center" }}>
                {memoryColors.map((item) => (
                  <button
                    key={item.id}
                    onClick={() => setClickCount(prev => prev + (item.id
=== 0 ? 1 : 0))}
                    style={{
                      width: "80px",
                      height: "80px",
                      backgroundColor: item.color,
                      borderRadius: "50%",
                      border: "3px solid white",
                      cursor: "pointer"
                    }}
                  />
                ))}
              </div>
              {clickCount > 0 && (
                <p style={{ marginTop: "1rem", fontSize: "1.2rem", color:
"#4ECDC4" }}>
                  {clickCount > 0 && memoryColors[0] ? "Great memory!" :
"Try again!"}
                </p>
              )}
            </div>
          )}
        </div>
```

```
    );

  case "hiddenStar":
    return (
      <div
        style={{
          position: "relative",
          height: "300px",
          backgroundColor: "#87CEEB",
          borderRadius: "12px",
          cursor: "pointer",
          overflow: "hidden"
        }}
        onClick={(e) => {
          const rect = e.currentTarget.getBoundingClientRect();
          const clickX = ((e.clientX - rect.left) / rect.width) * 100;
          const clickY = ((e.clientY - rect.top) / rect.height) * 100;

          const distance = Math.sqrt(
            Math.pow(clickX - hiddenStarPosition.x, 2) +
            Math.pow(clickY - hiddenStarPosition.y, 2)
          );

          if (distance < 10) {
            handleHiddenStarClick();
          }
        }}
      >
        {showHiddenStar && (
          <div
            style={{
              position: "absolute",
              left: `${hiddenStarPosition.x}%`,
```

```
              top: `${hiddenStarPosition.y}%`,
              fontSize: "3rem",
              animation: "twinkle 1s infinite"
            }}
          >
            ★
          </div>
        )}
        <div style={{
          position: "absolute",
          bottom: "20px",
          left: "50%",
          transform: "translateX(-50%)",
          fontSize: "1.2rem",
          color: "#333"
        }}>
          Stars found: {clickCount}
        </div>
      </div>
    );

  case "balloon":
    return (
      <div style={{ textAlign: "center", padding: "2rem" }}>
        <div
          onClick={handleBalloonClick}
          style={{
            fontSize: `${balloonSize}px`,
            cursor: "pointer",
            display: "inline-block",
            animation: "float 3s ease-in-out infinite",
            transition: "font-size 0.3s"
          }}
```

```
            >
              🎈
            </div>
            <p style={{ marginTop: "1rem", fontSize: "1.2rem", color:
"#FF6B6B" }}>
              Balloon size: {Math.round((balloonSize / 120) * 100)}%
            </p>
          </div>
        );

      default:
        return null;
    }
  };

  return (
    <div style={{
      backgroundColor: "rgba(255, 255, 255, 0.95)",
      borderRadius: "12px",
      padding: "2rem",
      margin: "2rem auto",
      maxWidth: "600px",
      boxShadow: "0 4px 12px rgba(0, 0, 0, 0.1)",
      textAlign: "center"
    }}>
      <h3 style={{ color: "#4ECDC4", marginBottom: "0.5rem" }}>
        {message}
      </h3>

      <p style={{
        color: "#666",
        fontSize: "1.1rem",
        marginBottom: "2rem",
```

```
    animation: "fadeInOut 2s infinite"
  }}>
    {activities[currentActivity].message}
  </p>

  {renderCurrentActivity()}

  <div style={{
    marginTop: "2rem",
    fontSize: "0.9rem",
    color: "#888"
  }}>
    Activity {currentActivity + 1} of {activities.length}
  </div>

  <style jsx>{`
    @keyframes bounce {
      0%, 20%, 50%, 80%, 100% { transform: translateY(0); }
      40% { transform: translateY(-20px); }
      60% { transform: translateY(-10px); }
    }

    @keyframes pulse {
      0% { transform: scale(1); }
      50% { transform: scale(1.1); }
      100% { transform: scale(1); }
    }

    @keyframes twinkle {
      0%, 100% { opacity: 1; }
      50% { opacity: 0.3; }
    }
```

```
        @keyframes float {
          0%, 100% { transform: translateY(0px); }
          50% { transform: translateY(-20px); }
        }

        @keyframes fadeInOut {
          0%, 100% { opacity: 1; }
          50% { opacity: 0.7; }
        }
      `}</style>
    </div>
  );
};

export default InteractiveWaitingComponent;
```

** AdultsPage.jsx **

```
import React from 'react';
import { Link } from 'react-router-dom';
import StorybookGenerator from './StorybookGenerator';
import './LandingPage.css';

export default function AdultsPage() {
  return (
    <div className="landing-container">
      {/* Header */}
      <header className="landing-header">
        <div className="logo">
          <Link to="/" style={{ textDecoration: 'none' }}>
            <h2>Little Legends</h2>
          </Link>
        </div>
        <div style={{ display: "flex", alignItems: "center", gap:
"1rem" }}>
```

```
        <Link to="/children" style={{ textDecoration: 'none' }}>
          <button
            style={{
              backgroundColor: "#7ed3c3",
              color: "white",
              padding: "0.8rem 1.5rem",
              border: "none",
              borderRadius: "20px",
              cursor: "pointer",
              fontSize: "1.1rem",
              fontWeight: "600",
              transition: "all 0.3s ease",
              transform: "scale(1)"
            }}
            onMouseEnter={(e) => {
              e.target.style.backgroundColor = "#6bc4b0";
              e.target.style.transform = "scale(1.05)";
            }}
            onMouseLeave={(e) => {
              e.target.style.backgroundColor = "#7ed3c3";
              e.target.style.transform = "scale(1)";
            }}
          >
            🎨 For Children
          </button>
        </Link>
        <div className="auth-buttons">
          <button className="auth-btn login-btn">Login</button>
          <button className="auth-btn signup-btn">Sign Up</button>
        </div>
      </div>
    </header>
```

```
      {/* Main Content */}
      <main className="landing-main">
        <StorybookGenerator />
      </main>

      {/* Footer */}
      <footer className="landing-footer">
        <div className="footer-content">
          <p>&copy; 2025 Little Legends. Made with ❤️ for young storytellers.</p>
          <div className="footer-links">
            <span>Privacy Policy</span> | <span>Terms of Service</span> | <span>Contact</span>
          </div>
        </div>
      </footer>
    </div>
  );
}
```

** App.css **

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
```

```
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

** App.jsx **

```
import React from 'react';
```

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import LandingPage from './LandingPage';
import ChildrenPage from './ChildrenPage';
import AdultsPage from './AdultsPage';


function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<LandingPage />} />
        <Route path="/children" element={<ChildrenPage />} />
        <Route path="/adults" element={<AdultsPage />} />
      </Routes>
    </Router>
  );
}


export default App;
```

** ChildrenPage.jsx **

```
import React, { useState, useRef, useEffect } from 'react';
import { Link } from 'react-router-dom';
import InteractiveWaitingComponent from
'./components/InteractiveWaitingComponent';
import './LandingPage.css';


// Storybook Generator
function CleanStorybookGenerator() {
  const [drawing, setDrawing] = useState(null);
  const [story, setStory] = useState("");
  const [aiImageUrl, setAiImageUrl] = useState("");
  const [isGenerating, setIsGenerating] = useState(false);
  const [savedDrawingUrl, setSavedDrawingUrl] = useState(null);
```

```
  // Audio states
  const [audioUrl, setAudioUrl] = useState(null);
  const [isGeneratingAudio, setIsGeneratingAudio] = useState(false);
  const [isPlaying, setIsPlaying] = useState(false);
  const [isPaused, setIsPaused] = useState(false);
  const audioRef = useRef(null);

  // Drawing pad states
  const canvasRef = useRef(null);
  const [isDrawing, setIsDrawing] = useState(false);
  const [currentColor, setCurrentColor] = useState("#2c3e50");
  const [brushSize, setBrushSize] = useState(8);
  const [isEraserMode, setIsEraserMode] = useState(false);

  const colors = [
    "#000000", "#FF0000", "#00FF00", "#0000FF", "#FFFF00",
    "#FF00FF", "#00FFFF", "#FFA500", "#800080", "#FFC0CB",
    "#A52A2A", "#808080"
  ];

  // Updated audio instruction functions
  const playInstruction = (step) => {
    const audioFiles = {
      1: 'http://localhost:1337/audio/instruction1.wav',
      2: 'http://localhost:1337/audio/instruction2.wav',
      3: 'http://localhost:1337/audio/instruction3.wav'
    };

    const audio = new Audio(audioFiles[step]);
    audio.play().catch(error => {
      console.error("Error playing instruction audio:", error);
    });
  };
```

```
// Drawing functions
const startDrawing = (e) => {
  setIsDrawing(true);
  draw(e);
};

const draw = (e) => {
  if (!isDrawing) return;

  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');
  const rect = canvas.getBoundingClientRect();

  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  ctx.lineWidth = brushSize;
  ctx.lineCap = 'round';

  if (isEraserMode) {
    ctx.globalCompositeOperation = 'destination-out';
  } else {
    ctx.globalCompositeOperation = 'source-over';
    ctx.strokeStyle = currentColor;
  }

  ctx.lineTo(x, y);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo(x, y);
};
```

```
  const stopDrawing = () => {
    if (!isDrawing) return;
    setIsDrawing(false);
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d');
    ctx.beginPath();
  };

  const clearCanvas = () => {
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d');
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    setDrawing(null);
    setSavedDrawingUrl(null);
  };

  const saveDrawing = () => {
    const canvas = canvasRef.current;
    const imageUrl = canvas.toDataURL('image/png');
    setSavedDrawingUrl(imageUrl);

    canvas.toBlob((blob) => {
      const file = new File([blob], "drawing.png", { type: "image/png" });
      setDrawing(file);
    });
  };

  // Story generation
  const handleGenerate = async () => {
    if (!drawing) {
      saveDrawing();
      await new Promise(resolve => setTimeout(resolve, 100));
    }
```

```
    const formData = new FormData();
    if (drawing) {
      formData.append("drawing", drawing);
    }
    formData.append("prompt", "");
    formData.append("stylePreference", "surprise-me");

    setIsGenerating(true);

    try {
      const response = await fetch("http://localhost:1337/generate", {
        method: "POST",
        body: formData,
      });
      const data = await response.json();
      setStory(data.story);
      setAiImageUrl(data.aiImageUrl);
      setIsGenerating(false);

      setAudioUrl(null);
      setIsPlaying(false);
      setIsPaused(false);

    } catch (error) {
      console.error("Error generating story:", error);
      setStory("Oops! Something went wrong. Let's try again!");
      setIsGenerating(false);
    }
  };

  // Audio generation
  const generateAndPlayAudio = async () => {
```

```
    if (!story) return;

    setIsGeneratingAudio(true);

    try {
      const requestBody = { text: story };

      const response = await fetch("http://localhost:1337/tts", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(requestBody),
      });

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const audioBlob = await response.blob();
      const audioUrl = URL.createObjectURL(audioBlob);
      setAudioUrl(audioUrl);
      setIsGeneratingAudio(false);

    } catch (error) {
      console.error("Error generating audio:", error);
      setIsGeneratingAudio(false);
      alert("Oops! The story voice isn't working. Try again!");
    }
  };

  // Audio control functions
  const playAudio = () => {
```

```
    if (audioRef.current) {
      audioRef.current.play();
      setIsPlaying(true);
      setIsPaused(false);
    }
  };

  const pauseAudio = () => {
    if (audioRef.current) {
      audioRef.current.pause();
      setIsPlaying(false);
      setIsPaused(true);
    }
  };

  const stopAudio = () => {
    if (audioRef.current) {
      audioRef.current.pause();
      audioRef.current.currentTime = 0;
      setIsPlaying(false);
      setIsPaused(false);
    }
  };

  const handleAudioEnded = () => {
    setIsPlaying(false);
    setIsPaused(false);
  };

  const startOver = () => {
    setStory("");
    setAiImageUrl("");
    setDrawing(null);
```

```
    setSavedDrawingUrl(null);
    setAudioUrl(null);
    setIsPlaying(false);
    setIsPaused(false);
    clearCanvas();
  };

  return (
    <div style={{
      minHeight: "100vh",
      background: "linear-gradient(135deg, #a8e6cf 0%, #7ed3c3 50%, #a8e6cf
100%)",
      backgroundAttachment: "fixed",
      display: "flex",
      justifyContent: "center",
      alignItems: "center",
      padding: "2rem",
      fontFamily: "'Segoe UI', Tahoma, Geneva, Verdana, sans-serif"
    }}>

      {/* Main Card */}
      <div style={{
        backgroundColor: "rgba(255, 255, 255, 0.95)",
        padding: "2rem",
        borderRadius: "25px",
        boxShadow: "0 8px 24px rgba(0, 0, 0, 0.15)",
        maxWidth: "900px",
        width: "100%",
        color: "#333"
      }}>

        {/* Header */}
        <div style={{ textAlign: "center", marginBottom: "2rem" }}>
```

```
        <h1 style={{
          fontSize: "2.5rem",
          marginBottom: "0.5rem",
          color: "#2c3e50",
          fontWeight: "600"
        }}>
          ✨ Little Legends ✨
        </h1>
        <p style={{
          fontSize: "1.2rem",
          margin: 0,
          color: "#7f8c8d"
        }}>
          Draw and Create Your Story!
        </p>
      </div>

      {!story ? (
        // Drawing Phase
        <div>

          {/* Step 1 Instruction - RED */}
          <div style={{ textAlign: "center", marginBottom: "1.5rem" }}>
            <button
              onClick={() => playInstruction(1)}
              style={{
                backgroundColor: "#ff6b6b",
                color: "white",
                padding: "1.8rem 3.5rem",
                border: "none",
                borderRadius: "30px",
                cursor: "pointer",
                fontSize: "1.8rem",
```

```
            fontWeight: "600",
            transition: "all 0.3s ease",
            display: "inline-flex",
            alignItems: "center",
            gap: "0.8rem",
            boxShadow: "0 6px 20px rgba(255, 107, 107, 0.4)",
            transform: "scale(1)"
          }}
          onMouseEnter={(e) => {
            e.target.style.backgroundColor = "#ff5252";
            e.target.style.transform = "scale(1.05)";
          }}
          onMouseLeave={(e) => {
            e.target.style.backgroundColor = "#ff6b6b";
            e.target.style.transform = "scale(1)";
          }}
        >
          1 🔊 Press Me First!
        </button>
      </div>

      {/* Color Palette */}
      <div style={{ marginBottom: "1.5rem" }}>
        <h2 style={{
          fontSize: "1.25rem",
          marginBottom: "1rem",
          textAlign: "center",
          color: "#2c3e50"
        }}>
          Pick Your Colors
        </h2>
        <div style={{
          display: "grid",
```

```
                gridTemplateColumns: "repeat(auto-fit, minmax(40px, 1fr))",
                gap: "0.5rem",
                justifyItems: "center",
                maxWidth: "500px",
                margin: "0 auto"
              }}>
                {colors.map((color) => (
                  <button
                    key={color}
                    onClick={() => setCurrentColor(color)}
                    style={{
                      width: "40px",
                      height: "40px",
                      backgroundColor: color,
                      border: currentColor === color ? "3px solid
#2c3e50" : "2px solid #ddd",
                      borderRadius: "15px",
                      cursor: "pointer",
                      transition: "all 0.2s ease"
                    }}
                  />
                ))}
              </div>
            </div>

            {/* Drawing Tools */}
            <div style={{
              display: "grid",
              gridTemplateColumns: "1fr 1fr 1fr",
              gap: "1rem",
              marginBottom: "1.5rem",
              alignItems: "center"
            }}>
```

```
{/* Brush Size */}
<div>
  <label style={{
    display: "block",
    fontSize: "1rem",
    marginBottom: "0.5rem",
    color: "#2c3e50",
    fontWeight: "500"
  }}>
    Brush Size: {brushSize}px
  </label>
  <input
    type="range"
    min="5"
    max="25"
    value={brushSize}
    onChange={(e) => setBrushSize(e.target.value)}
    style={{
      width: "100%",
      cursor: "pointer"
    }}
  />
</div>

{/* Eraser Toggle */}
<div style={{ textAlign: "center" }}>
  <button
    onClick={() => setIsEraserMode(!isEraserMode)}
    style={{
      backgroundColor: isEraserMode ? "#e74c3c" : "#95a5a6",
      color: "white",
      padding: "1rem 1.8rem",
```

```
        border: "none",
        borderRadius: "20px",
        cursor: "pointer",
        fontSize: "1.2rem",
        fontWeight: "600",
        transition: "all 0.3s ease",
        transform: "scale(1)"
      }}
      onMouseEnter={(e) => {
        e.target.style.transform = "scale(1.05)";
      }}
      onMouseLeave={(e) => {
        e.target.style.transform = "scale(1)";
      }}
    >
      {isEraserMode ? "🖍️ Drawing Mode" : "🫳 Eraser Mode"}
    </button>
  </div>

  {/* Clear Canvas */}
  <div style={{ textAlign: "center" }}>
    <button
      onClick={clearCanvas}
      style={{
        backgroundColor: "#e67e22",
        color: "white",
        padding: "1rem 1.8rem",
        border: "none",
        borderRadius: "20px",
        cursor: "pointer",
        fontSize: "1.2rem",
        fontWeight: "600",
        transition: "all 0.3s ease",
```

```
            transform: "scale(1)"
          }}
          onMouseEnter={(e) => {
            e.target.style.backgroundColor = "#d35400";
            e.target.style.transform = "scale(1.05)";
          }}
          onMouseLeave={(e) => {
            e.target.style.backgroundColor = "#e67e22";
            e.target.style.transform = "scale(1)";
          }}
        >
          🗑️ Clear Canvas
        </button>
      </div>
    </div>

    {/* Canvas */}
    <div style={{ textAlign: "center", marginBottom: "1.5rem" }}>
      <canvas
        ref={canvasRef}
        width={700}
        height={400}
        style={{
          border: "2px solid #ddd",
          borderRadius: "20px",
          backgroundColor: "white",
          cursor: "crosshair",
          boxShadow: "0 2px 8px rgba(0, 0, 0, 0.1)",
          maxWidth: "100%"
        }}
        onMouseDown={startDrawing}
        onMouseMove={draw}
        onMouseUp={stopDrawing}
```

```
        onMouseOut={stopDrawing}
      />
    </div>

    {/* Action Buttons */}
    <div style={{
      display: "flex",
      gap: "1rem",
      justifyContent: "center",
      flexWrap: "wrap",
      marginBottom: "1.5rem"
    }}>
      {/* Step 2 Button - BLUE */}
      <button
        onClick={() => playInstruction(2)}
        style={{
          backgroundColor: "#4ecdc4",
          color: "white",
          padding: "1.5rem 2.8rem",
          border: "none",
          borderRadius: "20px",
          cursor: "pointer",
          fontSize: "1.5rem",
          fontWeight: "600",
          transition: "all 0.3s ease",
          transform: "scale(1)",
          boxShadow: "0 4px 15px rgba(78, 205, 196, 0.4)"
        }}
        onMouseEnter={(e) => {
          e.target.style.backgroundColor = "#26d0ce";
          e.target.style.transform = "scale(1.05)";
        }}
        onMouseLeave={(e) => {
```

```
        e.target.style.backgroundColor = "#4ecdc4";
        e.target.style.transform = "scale(1)";
      }}
    >
      2 🔊 Next Step
    </button>
  </div>

  {/* Generate Button - GREEN */}
  <div style={{ textAlign: "center" }}>
    <button
      onClick={handleGenerate}
      style={{
        backgroundColor: "#51cf66",
        color: "white",
        padding: "1.8rem 3.5rem",
        border: "none",
        borderRadius: "25px",
        cursor: "pointer",
        fontSize: "1.8rem",
        fontWeight: "600",
        transition: "all 0.3s ease",
        transform: "scale(1)",
        boxShadow: "0 6px 20px rgba(81, 207, 102, 0.4)"
      }}
      onMouseEnter={(e) => {
        e.target.style.backgroundColor = "#40c057";
        e.target.style.transform = "scale(1.05)";
      }}
      onMouseLeave={(e) => {
        e.target.style.backgroundColor = "#51cf66";
        e.target.style.transform = "scale(1)";
      }}
```

```
          >
            3 OK! ✨
          </button>
        </div>

        {/* Loading */}
        {isGenerating && (
          <div style={{ marginTop: "1.5rem" }}>
            <InteractiveWaitingComponent
              message="Creating your magical story"
              type="story"
            />
          </div>
        )}
      </div>
    ) : (
      // Story Display Phase
      <div>

        {/* Images Section */}
        <div style={{
          display: "grid",
          gridTemplateColumns: "1fr 1fr",
          gap: "2rem",
          marginBottom: "2rem"
        }}>
          {/* Your Drawing */}
          <div className="image-section">
            <h2 style={{
              fontSize: "1.25rem",
              marginBottom: "0.5rem",
              color: "#2c3e50"
            }}>
```

```
        Your Amazing Drawing! 🎨
      </h2>
      {savedDrawingUrl && (
        <img
          src={savedDrawingUrl}
          alt="Your Drawing"
          style={{
            maxWidth: "100%",
            borderRadius: "20px",
            boxShadow: "0 2px 8px rgba(0, 0, 0, 0.1)"
          }}
        />
      )}
    </div>

    {/* AI Generated Image */}
    <div className="image-section">
      <h2 style={{
        fontSize: "1.25rem",
        marginBottom: "0.5rem",
        color: "#2c3e50"
      }}>
        Magic Story Picture! ✨
      </h2>
      {aiImageUrl && (
        <img
          src={aiImageUrl}
          alt="Magic Story Picture"
          style={{
            maxWidth: "100%",
            borderRadius: "20px",
            boxShadow: "0 2px 8px rgba(0, 0, 0, 0.1)"
          }}
```

```
        />
      )}
    </div>
  </div>

  {/* Story Text */}
  <div className="story-section">
    <h2 style={{
      fontSize: "1.5rem",
      marginBottom: "1rem",
      textAlign: "center",
      color: "#2c3e50"
    }}>
      📖 Your Magical Story! 📖
    </h2>
    <p style={{
      fontSize: "1.1rem",
      lineHeight: "1.6",
      whiteSpace: "pre-line",
      textAlign: "justify",
      color: "#2c3e50"
    }}>
      {story}
    </p>
  </div>

  {/* Audio Section */}
  <div style={{
    marginTop: "2rem",
    textAlign: "center",
    padding: "1.5rem",
    backgroundColor: "#f8f9fa",
    borderRadius: "20px"
```

```
      }}>

        {/* Step 3 Instruction - PURPLE */}
        <button
          onClick={() => playInstruction(3)}
          style={{
            backgroundColor: "#9b59b6",
            color: "white",
            padding: "1.2rem 2.5rem",
            border: "none",
            borderRadius: "20px",
            cursor: "pointer",
            fontSize: "1.4rem",
            fontWeight: "600",
            transition: "all 0.3s ease",
            marginBottom: "1rem",
            transform: "scale(1)",
            boxShadow: "0 4px 15px rgba(155, 89, 182, 0.3)"
          }}
          onMouseEnter={(e) => {
            e.target.style.backgroundColor = "#8e44ad";
            e.target.style.transform = "scale(1.05)";
          }}
          onMouseLeave={(e) => {
            e.target.style.backgroundColor = "#9b59b6";
            e.target.style.transform = "scale(1)";
          }}
        >
          3 🔊 Press Me!
        </button>

        {/* Listen Button - YELLOW */}
        {!isGeneratingAudio && !audioUrl && (
```

```
      <div>
        <button
          onClick={generateAndPlayAudio}
          style={{
            backgroundColor: "#ffd93d",
            color: "#2c3e50",
            padding: "1.8rem 3.5rem",
            border: "none",
            borderRadius: "25px",
            cursor: "pointer",
            fontSize: "1.8rem",
            fontWeight: "600",
            transition: "all 0.3s ease",
            transform: "scale(1)",
            boxShadow: "0 6px 20px rgba(255, 217, 61, 0.4)"
          }}
          onMouseEnter={(e) => {
            e.target.style.backgroundColor = "#fcc419";
            e.target.style.transform = "scale(1.05)";
          }}
          onMouseLeave={(e) => {
            e.target.style.backgroundColor = "#ffd93d";
            e.target.style.transform = "scale(1)";
          }}
        >
          4️⃣ 🔊 Listen to Story! 🎶
        </button>
      </div>
    )}

    {/* Audio Loading */}
    {isGeneratingAudio && (
      <InteractiveWaitingComponent
```

```
      message="Making your story voice ready"
      type="audio"
    />
)}

{/* Audio Controls */}
{audioUrl && (
  <div>
    <div style={{
      display: "flex",
      gap: "1rem",
      justifyContent: "center",
      marginBottom: "1rem",
      flexWrap: "wrap"
    }}>

      {!isPlaying && (
        <button
          onClick={playAudio}
          style={{
            backgroundColor: "#51cf66",
            color: "white",
            padding: "1rem 2rem",
            border: "none",
            borderRadius: "20px",
            cursor: "pointer",
            fontSize: "1.3rem",
            fontWeight: "600",
            transition: "all 0.3s ease",
            transform: "scale(1)"
          }}
          onMouseEnter={(e) => {
            e.target.style.backgroundColor = "#40c057";
```

```
        e.target.style.transform = "scale(1.05)";
      }}
      onMouseLeave={(e) => {
        e.target.style.backgroundColor = "#51cf66";
        e.target.style.transform = "scale(1)";
      }}
    >
      ▶ Play
    </button>
  )}

  {isPlaying && (
    <button
      onClick={pauseAudio}
      style={{
        backgroundColor: "#ffd93d",
        color: "#2c3e50",
        padding: "1rem 2rem",
        border: "none",
        borderRadius: "20px",
        cursor: "pointer",
        fontSize: "1.3rem",
        fontWeight: "600",
        transition: "all 0.3s ease",
        transform: "scale(1)"
      }}
      onMouseEnter={(e) => {
        e.target.style.backgroundColor = "#fcc419";
        e.target.style.transform = "scale(1.05)";
      }}
      onMouseLeave={(e) => {
        e.target.style.backgroundColor = "#ffd93d";
        e.target.style.transform = "scale(1)";
```

```
            }}
          >
            ⏸ Pause
          </button>
        )}

        <button
          onClick={stopAudio}
          style={{
            backgroundColor: "#ff6b6b",
            color: "white",
            padding: "1rem 2rem",
            border: "none",
            borderRadius: "20px",
            cursor: "pointer",
            fontSize: "1.3rem",
            fontWeight: "600",
            transition: "all 0.3s ease",
            transform: "scale(1)"
          }}
          onMouseEnter={(e) => {
            e.target.style.backgroundColor = "#ff5252";
            e.target.style.transform = "scale(1.05)";
          }}
          onMouseLeave={(e) => {
            e.target.style.backgroundColor = "#ff6b6b";
            e.target.style.transform = "scale(1)";
          }}
        >
          ⏹ Stop
        </button>
      </div>
```

```
          <p style={{
            fontSize: "1.1rem",
            color: "#2c3e50",
            fontWeight: "500",
            margin: 0
          }}>
            {isPlaying ? "🎶 Playing your story! 🎶" :
             isPaused ? "⏸️ Story paused" :
             "🎶 Story is ready! 🎶"}
          </p>
        </div>
      )}
    </div>

    {/* Start Over Button */}
    <div style={{ textAlign: "center", marginTop: "2rem" }}>
      <button
        onClick={startOver}
        style={{
          backgroundColor: "#e74c3c",
          color: "white",
          padding: "1.2rem 2.5rem",
          border: "none",
          borderRadius: "20px",
          cursor: "pointer",
          fontSize: "1.3rem",
          fontWeight: "600",
          transition: "all 0.3s ease",
          transform: "scale(1)"
        }}
        onMouseEnter={(e) => {
          e.target.style.backgroundColor = "#c0392b";
          e.target.style.transform = "scale(1.05)";
```

```
                }}
                onMouseLeave={(e) => {
                  e.target.style.backgroundColor = "#e74c3c";
                  e.target.style.transform = "scale(1)";
                }}
              >
                🎨 Draw New Picture!
              </button>
            </div>

            {/* Hidden audio element */}
            {audioUrl && (
              <audio
                ref={audioRef}
                src={audioUrl}
                onEnded={handleAudioEnded}
                style={{ display: "none" }}
              />
            )}
          </div>
        )}
      </div>
    </div>
  );
}

export default function ChildrenPage() {
  return (
    <div className="landing-container">
      {/* Header */}
      <header className="landing-header">
        <div className="logo">
          <Link to="/" style={{ textDecoration: 'none' }}>
```

```
            <h2>Little Legends</h2>
          </Link>
        </div>
        <div style={{ display: "flex", alignItems: "center", gap:
"1rem" }}>
          <Link to="/adults" style={{ textDecoration: 'none' }}>
            <button
              style={{
                backgroundColor: "#7ed3c3",
                color: "white",
                padding: "0.8rem 1.5rem",
                border: "none",
                borderRadius: "20px",
                cursor: "pointer",
                fontSize: "1.1rem",
                fontWeight: "600",
                transition: "all 0.3s ease",
                transform: "scale(1)"
              }}
              onMouseEnter={(e) => {
                e.target.style.backgroundColor = "#6bc4b0";
                e.target.style.transform = "scale(1.05)";
              }}
              onMouseLeave={(e) => {
                e.target.style.backgroundColor = "#7ed3c3";
                e.target.style.transform = "scale(1)";
              }}
            >
              👨‍👩‍👧 For Adults
            </button>
          </Link>
          <div className="auth-buttons">
            <button className="auth-btn login-btn">Login</button>
```

```
            <button className="auth-btn signup-btn">Sign Up</button>
          </div>
        </div>
      </header>

      {/* Main Content */}
      <main className="landing-main">
        <CleanStorybookGenerator />
      </main>

      {/* Footer */}
      <footer className="landing-footer">
        <div className="footer-content">
          <p>&copy; 2025 Little Legends. Made with ❤️ for young storytellers.</p>
          <div className="footer-links">
            <span>Privacy Policy</span> | <span>Terms of Service</span> | <span>Contact</span>
          </div>
        </div>
      </footer>
    </div>
  );
}
```

** index.css **

```
/* Global reset */
html, body, #root {
  margin: 0 !important;
  padding: 0 !important;
  width: 100%;
  height: 100%;
}
```

```
/* Base page styles */
body {
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: url('/fairytale.png') no-repeat center center fixed;
  background-size: cover;
  color: #333;
}

/* Center everything */
.app-container {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 2rem;
}

/* Card styling */
.card {
  background-color: rgba(255, 255, 255, 0.9);
  padding: 2rem;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  max-width: 600px;
  width: 100%;
}

/* Heading */
.card h1 {
  font-size: 1.75rem;
  margin-bottom: 1rem;
```

```
  text-align: center;
}

/* Form inputs and buttons */
input[type='file'],
input[type='text'] {
  display: block;
  width: 100%;
  margin-top: 0.5rem;
  margin-bottom: 1rem;
  padding: 0.5rem;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 1rem;
}

button {
  background-color: #4a90e2;
  color: white;
  padding: 0.6rem 1.2rem;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 1rem;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: #357ab7;
}

/* Generated story */
.story-section {
```

```
  margin-top: 1.5rem;
}

.story-section h2 {
  font-size: 1.25rem;
  margin-bottom: 0.5rem;
}

.story-section p {
  white-space: pre-line;
}

/* Image section */
.image-section {
  margin-top: 1.5rem;
}

.image-section h2 {
  font-size: 1.25rem;
  margin-bottom: 0.5rem;
}

.image-section img {
  max-width: 100%;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
```

** LandingPage.css **

```
/* Reset browser defaults and ensure full width */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
```

```
}

/* Landing Page Specific Styles */
.landing-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  background: url('/fairytale.png') no-repeat center center fixed;
  background-size: cover;
  width: 100vw; /* Force full viewport width */
}

/* Header - Full width, break out of any centering */
.landing-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 0;
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(10px);
  width: 100vw; /* Full viewport width */
  position: relative;
  left: 50%;
  right: 50%;
  margin-left: -50vw;
  margin-right: -50vw;
}

.logo {
  padding-left: 2rem;
}

.logo h2 {
```

```
  color: #b8860b;
  margin: 0;
  font-size: 1.5rem;
}

.auth-buttons {
  display: flex;
  gap: 1rem;
  padding-right: 2rem;
}

.auth-btn {
  padding: 0.5rem 1rem;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 0.9rem;
  transition: all 0.3s ease;
}

.login-btn {
  background: transparent;
  color: #4a90e2;
  border: 2px solid #4a90e2;
}

.signup-btn {
  background: #4a90e2;
  color: white;
}

.auth-btn:hover {
  transform: translateY(-2px);
```

```
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
  }

  /* Main Content */
  .landing-main {
    flex: 1;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 2rem;
  }

  .hero-section {
    text-align: center;
    background: rgba(255, 255, 255, 0.9);
    padding: 3rem;
    border-radius: 16px;
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.1);
    max-width: 600px;
    width: 100%;
  }

  .hero-title {
    font-size: 2.5rem;
    color: #b8860b;
    margin-bottom: 1rem;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.1);
  }

  .hero-subtitle {
    font-size: 1.2rem;
    color: #666;
    margin-bottom: 2rem;
```

```
}

/* User Type Buttons */
.user-type-buttons {
  display: flex;
  gap: 2rem;
  justify-content: center;
  flex-wrap: wrap;
}

.user-btn {
  display: flex;
  align-items: center;
  gap: 1rem;
  padding: 1.5rem;
  background: linear-gradient(135deg, #ff6b6b, #ffd93d);
  border-radius: 12px;
  text-decoration: none;
  color: white;
  transition: all 0.3s ease;
  min-width: 200px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
}

.children-btn {
  background: linear-gradient(135deg, #ff9a9e, #fecfef);
}

.adults-btn {
  background: linear-gradient(135deg, #a8edea, #fed6e3);
}

.user-btn:hover {
```

```
    transform: translateY(-5px);
    box-shadow: 0 8px 25px rgba(0, 0, 0, 0.3);
  }

  .btn-icon {
    font-size: 2rem;
  }

  .btn-text h3 {
    margin: 0;
    font-size: 1.3rem;
  }

  .btn-text p {
    margin: 0.2rem 0 0 0;
    font-size: 0.9rem;
    opacity: 0.9;
  }

  /* Footer - Full width, break out of any centering */
  .landing-footer {
    background: rgba(255, 255, 255, 0.95);
    padding: 1rem 0;
    text-align: center;
    backdrop-filter: blur(10px);
    width: 100vw; /* Full viewport width */
    position: relative;
    left: 50%;
    right: 50%;
    margin-left: -50vw;
    margin-right: -50vw;
  }
```

```
.footer-content p {
  margin: 0;
  color: #666;
}

.footer-links {
  margin-top: 0.5rem;
  color: #888;
  font-size: 0.9rem;
}

.footer-links span {
  cursor: pointer;
  transition: color 0.3s ease;
}

.footer-links span:hover {
  color: #4a90e2;
}

/* Responsive Design */
@media (max-width: 768px) {
  .user-type-buttons {
    flex-direction: column;
    align-items: center;
  }

  .hero-title {
    font-size: 2rem;
  }

  .landing-header {
    padding: 1rem;
```

```
    }
  }
```

** LandingPage.jsx **

```
import React from 'react';
import { Link } from 'react-router-dom';
import './LandingPage.css';

export default function LandingPage() {
  return (
    <div className="landing-container">
      {/* Header */}
      <header className="landing-header">
        <div className="logo">
          <h2>Little Legends</h2>
        </div>
        <div className="auth-buttons">
          <button className="auth-btn login-btn">Login</button>
          <button className="auth-btn signup-btn">Sign Up</button>
        </div>
      </header>

      {/* Main Content */}
      <main className="landing-main">
        <div className="hero-section">
          <h1 className="hero-title">Welcome to Little Legends</h1>
          <p className="hero-subtitle">AI-Powered Storytelling for Every
Adventure</p>

          <div className="user-type-buttons">
            <Link to="/children" className="user-btn children-btn">
              <div className="btn-icon">
                🎨
              </div>
```

```
              <div className="btn-text">
                <h3>Children</h3>
                <p>Draw & Create Stories</p>
              </div>
            </Link>

            <Link to="/adults" className="user-btn adults-btn">
              <div className="btn-icon">
                🧑‍🤝‍🧑
              </div>
              <div className="btn-text">
                <h3>Adults</h3>
                <p>Advanced Story Tools</p>
              </div>
            </Link>
          </div>
        </div>
      </main>

      {/* Footer */}
      <footer className="landing-footer">
        <div className="footer-content">
    <p>&copy; 2025 Little Legends. Made with ❤️ for young storytellers.</p>
          <div className="footer-links">
            <span>Privacy Policy</span> | <span>Terms of Service</span> |
<span>Contact</span>
          </div>
        </div>
      </footer>
    </div>
  );
}
```

** main.jsx **

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

** StorybookGenerator.jsx **

```
import React, { useState, useRef, useEffect } from "react";
import InteractiveWaitingComponent from
'./components/InteractiveWaitingComponent';

export default function StorybookGenerator() {
  const [drawing, setDrawing] = useState(null);
  const [prompt, setPrompt] = useState("");
  const [story, setStory] = useState("");
  const [aiImageUrl, setAiImageUrl] = useState("");
  const utteranceRef = useRef(null);
  const [isGenerating, setIsGenerating] = useState(false);
  const [dots, setDots] = useState(".");
  const [savedDrawingUrl, setSavedDrawingUrl] = useState(null);

  // Drawing editing mode state
  const [isEditingDrawing, setIsEditingDrawing] = useState(false);
```

```
// Style preference state
const [stylePreference, setStylePreference] = useState("make-prettier");

// Audio states for XTTS-v2
const [audioUrl, setAudioUrl] = useState(null);
const [isGeneratingAudio, setIsGeneratingAudio] = useState(false);
const [isPlaying, setIsPlaying] = useState(false);
const [isPaused, setIsPaused] = useState(false);
const audioRef = useRef(null);

// Voice selection and recording states
const [voiceMode, setVoiceMode] = useState("default");
const [isRecording, setIsRecording] = useState(false);
const [voiceId, setVoiceId] = useState(null);
const [hasVoiceRecording, setHasVoiceRecording] = useState(false);
const [mediaRecorder, setMediaRecorder] = useState(null);

// Drawing pad states
const canvasRef = useRef(null);
const [isDrawing, setIsDrawing] = useState(false);
const [currentColor, setCurrentColor] = useState("#000000");
const [brushSize, setBrushSize] = useState(5);
const [isEraser, setIsEraser] = useState(false);
const [useDrawingPad, setUseDrawingPad] = useState(true);

const colors = [
  "#000000", "#FF0000", "#00FF00", "#0000FF", "#FFFF00",
  "#FF00FF", "#00FFFF", "#FFA500", "#800080", "#FFC0CB",
  "#A52A2A", "#808080"
];

useEffect(() => {
```

```
    let interval;
    if (isGenerating) {
      interval = setInterval(() => {
        setDots((prev) => (prev.length >= 3 ? "." : prev + "."));
      }, 500);
    } else {
      setDots(".");
    }
    return () => clearInterval(interval);
  }, [isGenerating]);

  // Complete state clearing function
  const clearAllDrawingState = () => {
    setDrawing(null);
    setSavedDrawingUrl(null);
    if (canvasRef.current) {
      const canvas = canvasRef.current;
      const ctx = canvas.getContext('2d');
      ctx.clearRect(0, 0, canvas.width, canvas.height);
    }
  };

  // Mode switching functions with proper state clearing
  const switchToDrawingMode = () => {
    setUseDrawingPad(true);
    clearAllDrawingState();
    setIsEditingDrawing(false);
  };

  const switchToUploadMode = () => {
    setUseDrawingPad(false);
    clearAllDrawingState();
    setIsEditingDrawing(false);
```

```
};

// Function to edit existing drawing
const startEditingDrawing = () => {
  setIsEditingDrawing(true);
  // Don't clear the drawing state, just switch to editing mode
};

// Function to finish editing and save
const finishEditingDrawing = () => {
  saveDrawing();
  setIsEditingDrawing(false);
};

// Drawing functions
const startDrawing = (e) => {
  setIsDrawing(true);
  draw(e);
};

const draw = (e) => {
  if (!isDrawing) return;

  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');
  const rect = canvas.getBoundingClientRect();

  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  ctx.lineWidth = brushSize;
  ctx.lineCap = 'round';
```

```
    if (isEraser) {
      ctx.globalCompositeOperation = 'destination-out';
    } else {
      ctx.globalCompositeOperation = 'source-over';
      ctx.strokeStyle = currentColor;
    }

    ctx.lineTo(x, y);
    ctx.stroke();
    ctx.beginPath();
    ctx.moveTo(x, y);
  };

  const stopDrawing = () => {
    if (!isDrawing) return;
    setIsDrawing(false);
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d');
    ctx.beginPath();
  };

  const clearCanvas = () => {
    clearAllDrawingState();
  };

  const saveDrawing = () => {
    const canvas = canvasRef.current;
    const imageUrl = canvas.toDataURL('image/png');
    setSavedDrawingUrl(imageUrl);

    canvas.toBlob((blob) => {
      const file = new File([blob], "drawing.png", { type: "image/png" });
      setDrawing(file);
```

```
    });
  };

  // File handling functions
  const handleFileChange = (e) => {
    clearAllDrawingState(); // Clear previous state
    setDrawing(e.target.files[0]);
    if (e.target.files[0]) {
      const url = URL.createObjectURL(e.target.files[0]);
      setSavedDrawingUrl(url);
    }
  };

  const handlePromptChange = (e) => {
    setPrompt(e.target.value);
  };

  // Voice Recording Functions
  const startVoiceRecording = async () => {
    try {
      const stream = await navigator.mediaDevices.getUserMedia({ audio:
true });

      let mimeType = 'audio/wav';
      if (!MediaRecorder.isTypeSupported(mimeType)) {
        mimeType = 'audio/webm';
        console.log('WAV not supported, using WebM format');
      } else {
        console.log('Recording in WAV format');
      }

      const recorder = new MediaRecorder(stream, { mimeType });
      const chunks = [];
```

```
      recorder.ondataavailable = (e) => {
        if (e.data.size > 0) {
          chunks.push(e.data);
        }
      };

      recorder.onstop = async () => {
        const audioBlob = new Blob(chunks, { type: mimeType });
        await uploadVoiceReference(audioBlob);
        stream.getTracks().forEach(track => track.stop());
      };

      recorder.start();
      setMediaRecorder(recorder);
      setIsRecording(true);

    } catch (error) {
      console.error("Error starting recording:", error);
      alert("Unable to access microphone. Please check permissions.");
    }
  };

  const stopVoiceRecording = () => {
    if (mediaRecorder && isRecording) {
      mediaRecorder.stop();
      setIsRecording(false);
    }
  };

  const uploadVoiceReference = async (audioBlob) => {
    try {
      const formData = new FormData();
```

```
      const filename = audioBlob.type.includes('wav') ?
'voice_reference.wav' : 'voice_reference.webm';
      formData.append('voice_file', audioBlob, filename);

      const response = await fetch('http://localhost:5001/upload-voice', {
        method: 'POST',
        body: formData,
      });

      const data = await response.json();

      if (data.success) {
        setVoiceId(data.voice_id);
        setHasVoiceRecording(true);
        alert("Voice recorded successfully! Your stories will now use your
voice.");
      } else {
        throw new Error(data.error || 'Failed to upload voice');
      }

    } catch (error) {
      console.error("Error uploading voice:", error);
      alert("Error saving your voice. Please try again.");
    }
  };

  // Story generation function with proper state handling
  const handleGenerate = async () => {
    handleStopAudio();

    if (useDrawingPad && !drawing) {
      saveDrawing();
      await new Promise(resolve => setTimeout(resolve, 100));
    }
```

```
    if (!drawing && !prompt) {
      alert("Please draw something, upload a file, or enter a text
prompt.");
      return;
    }

    const formData = new FormData();
    if (drawing) {
      formData.append("drawing", drawing);
    }
    formData.append("prompt", prompt);
    formData.append("stylePreference", stylePreference);

    setIsGenerating(true);

    try {
      const response = await fetch("http://localhost:1337/generate", {
        method: "POST",
        body: formData,
      });
      const data = await response.json();
      setStory(data.story);
      setAiImageUrl(data.aiImageUrl);
      setIsGenerating(false);

      // Clear audio state
      setAudioUrl(null);
      setIsPlaying(false);
      setIsPaused(false);

      // Exit editing mode if we were in it
      setIsEditingDrawing(false);
```

```
    } catch (error) {
      console.error("Error generating story:", error);
      setStory("Sorry, something went wrong while generating your story.");
      setIsGenerating(false);
    }
  };

  // Audio functions
  const generateAudio = async () => {
    if (!story) return;

    setIsGeneratingAudio(true);

    try {
      const requestBody = { text: story };

      if (voiceMode === "custom" && voiceId) {
        requestBody.voice_id = voiceId;
      }

      const response = await fetch("http://localhost:1337/tts", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(requestBody),
      });

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
```

```
      const audioBlob = await response.blob();
      const audioUrl = URL.createObjectURL(audioBlob);
      setAudioUrl(audioUrl);
      setIsGeneratingAudio(false);

    } catch (error) {
      console.error("Error generating audio:", error);
      setIsGeneratingAudio(false);
      alert("Sorry, there was an error generating the audio. Please try
again.");
    }
  };

  const handleReadStory = async () => {
    if (!audioUrl) {
      await generateAudio();
      return;
    }

    if (audioRef.current) {
      audioRef.current.currentTime = 0;
      audioRef.current.play();
      setIsPlaying(true);
      setIsPaused(false);
    }
  };

  const handlePauseAudio = () => {
    if (audioRef.current && isPlaying) {
      audioRef.current.pause();
      setIsPlaying(false);
      setIsPaused(true);
    }
```

```
};

const handleResumeAudio = () => {
  if (audioRef.current && isPaused) {
    audioRef.current.play();
    setIsPlaying(true);
    setIsPaused(false);
  } else {
    handleReadStory();
  }
};

const handleStopAudio = () => {
  if (audioRef.current) {
    audioRef.current.pause();
    audioRef.current.currentTime = 0;
    setIsPlaying(false);
    setIsPaused(false);
  }
};

const handleAudioEnded = () => {
  setIsPlaying(false);
  setIsPaused(false);
};

const handleAudioError = (e) => {
  console.error("Audio playback error:", e);
  setIsPlaying(false);
  setIsPaused(false);
  alert("Error playing audio. Please try generating the audio again.");
};
```

```
  const containerStyle = {
    width: "100%",
    maxWidth: story ? "1200px" : "600px",
    margin: "0 auto",
    background: "rgba(255, 255, 255, 0.9)",
    padding: "2rem",
    borderRadius: "12px",
    boxShadow: "0 4px 12px rgba(0, 0, 0, 0.1)"
  };

  const layoutStyle = {
    display: story ? "grid" : "block",
    gridTemplateColumns: story ? "1fr 1fr" : "1fr",
    gap: story ? "2rem" : "0",
    alignItems: "flex-start"
  };

  return (
    <div style={containerStyle}>
      <div style={{ textAlign: "center", marginBottom: "1rem" }}>
        <h1 style={{ color: "#b8860b" }}>Little Legends</h1>
        <h3>Children's Storybook AI Generator</h3>
      </div>

      <div style={layoutStyle}>

        {/* Left Column - Drawing Area */}
        <div>
          {/* Drawing Mode Toggle */}
          <div style={{ marginBottom: "1rem", textAlign: "center" }}>
            <button
              onClick={switchToDrawingMode}
              style={{
```

```
                margin: "0 0.5rem",
                backgroundColor: useDrawingPad ? "#4a90e2" : "#ccc",
                color: "white",
                border: "none",
                padding: "0.5rem 1rem",
                borderRadius: "6px",
                cursor: "pointer"
              }}
            >
              🎨 Draw Here
            </button>
            <button
              onClick={switchToUploadMode}
              style={{
                margin: "0 0.5rem",
                backgroundColor: !useDrawingPad ? "#4a90e2" : "#ccc",
                color: "white",
                border: "none",
                padding: "0.5rem 1rem",
                borderRadius: "6px",
                cursor: "pointer"
              }}
            >
              📁 Upload Photo
            </button>
          </div>

          {/* Smart content display logic */}
          {story && savedDrawingUrl && !isEditingDrawing ? (
            // After story generation - show saved image with edit option
            <div style={{ marginBottom: "1rem" }}>
              <div style={{ display: "flex", justifyContent: "space-
between", alignItems: "center", marginBottom: "0.5rem" }}>
```

```
            <h3>{useDrawingPad ? "Your Drawing:" : "Your Photo:"}</h3>
            {useDrawingPad && (
              <button
                onClick={startEditingDrawing}
                style={{
                  padding: "0.3rem 0.6rem",
                  backgroundColor: "#ffa500",
                  color: "white",
                  border: "none",
                  borderRadius: "4px",
                  cursor: "pointer",
                  fontSize: "0.8rem"
                }}
              >
```

```
Edit Drawing
              </button>
            )}
          </div>
          <img
            src={savedDrawingUrl}
            alt={useDrawingPad ? "Your Drawing" : "Your Photo"}
            style={{
              maxWidth: "100%",
              height: "auto",
              border: "2px solid #ccc",
              borderRadius: "8px",
              display: "block",
              margin: "0 auto"
            }}
          />
        </div>
      ) : useDrawingPad || isEditingDrawing ? (
        // Drawing mode OR editing mode - show canvas and tools
```

```
            <div style={{ marginBottom: "1rem" }}>
              <div style={{ display: "flex", justifyContent: "space-
between", alignItems: "center", marginBottom: "1rem" }}>
                <h3>{isEditingDrawing ? "Edit Your Drawing:" : "Draw Your
Picture:"}</h3>
                {isEditingDrawing && (
                  <button
                    onClick={finishEditingDrawing}
                    style={{
                      padding: "0.5rem 1rem",
                      backgroundColor: "#4a90e2",
                      color: "white",
                      border: "none",
                      borderRadius: "6px",
                      cursor: "pointer"
                    }}
                  >
                    ✅ Done Editing
                  </button>
                )}
              </div>

              {/* Drawing Tools - ALWAYS available in drawing/editing mode
*/}
              <div style={{ marginBottom: "1rem" }}>
                {/* Color Palette */}
                <div style={{ marginBottom: "1rem" }}>
                  <strong>Colors:</strong>
                  <div style={{ display: "flex", gap: "0.3rem", marginTop:
"0.3rem", flexWrap: "wrap" }}>
                    {colors.map((color) => (
                      <button
                        key={color}
                        onClick={() => {
```

```
                          setCurrentColor(color);
                          setIsEraser(false);
                        }}
                        style={{
                          width: "30px",
                          height: "30px",
                          backgroundColor: color,
                          border: currentColor === color && !isEraser ?
"3px solid #000" : "1px solid #ccc",
                          borderRadius: "50%",
                          cursor: "pointer"
                        }}
                      />
                    ))}
                  </div>
                </div>

                {/* Brush Size */}
                <div style={{ marginBottom: "1rem" }}>
                  <strong>Brush Size:</strong>
                  <input
                    type="range"
                    min="1"
                    max="20"
                    value={brushSize}
                    onChange={(e) => setBrushSize(e.target.value)}
                    style={{ marginLeft: "0.5rem" }}
                  />
                  <span style={{ marginLeft:
"0.5rem" }}>{brushSize}px</span>
                </div>

                {/* Tools */}
                <div>
```

```
        <button
          onClick={() => setIsEraser(!isEraser)}
          style={{
            backgroundColor: isEraser ? "#ff4444" : "#ccc",
            color: "white",
            border: "none",
            padding: "0.5rem 1rem",
            borderRadius: "6px",
            cursor: "pointer",
            marginRight: "0.5rem"
          }}
        >
          🖌️ Eraser
        </button>
        <button
          onClick={clearCanvas}
          style={{
            backgroundColor: "#ff6666",
            color: "white",
            border: "none",
            padding: "0.5rem 1rem",
            borderRadius: "6px",
            cursor: "pointer"
          }}
        >
          🗑️ Clear All
        </button>
      </div>
    </div>

    {/* Canvas - ALWAYS available */}
    <canvas
      ref={canvasRef}
```

```
                width={500}
                height={400}
                style={{
                  border: "2px solid #ccc",
                  borderRadius: "8px",
                  backgroundColor: "white",
                  cursor: "crosshair",
                  display: "block",
                  margin: "0 auto"
                }}
                onMouseDown={startDrawing}
                onMouseMove={draw}
                onMouseUp={stopDrawing}
                onMouseOut={stopDrawing}
              />
            </div>
          ) : (
            // Upload mode - upload section only
            <div style={{ marginBottom: "1rem" }}>
              <label>Upload your drawing:</label>
              <input type="file" accept="image/*"
onChange={handleFileChange} />
              {savedDrawingUrl && (
                <div style={{ marginTop: "1rem" }}>
                  <img
                    src={savedDrawingUrl}
                    alt="Uploaded Drawing"
                    style={{
                      maxWidth: "100%",
                      height: "auto",
                      border: "2px solid #ccc",
                      borderRadius: "8px",
                      display: "block",
```

```
                    margin: "0 auto"
                  }}
                />
              </div>
            )}
          </div>
        )}

        {/* Voice Selection Section */}
        <div style={{
          marginBottom: "1rem",
          padding: "1rem",
          backgroundColor: "rgba(180, 134, 11, 0.1)",
          borderRadius: "8px",
          border: "2px solid #b8860b"
        }}>
          <h4 style={{ margin: "0 0 0.5rem 0", color: "#b8860b" }}>🎤
Choose Voice for Story:</h4>

          <div style={{ marginBottom: "1rem" }}>
            <label style={{ display: "block", marginBottom: "0.5rem" }}>
              <input
                type="radio"
                name="voiceMode"
                value="default"
                checked={voiceMode === "default"}
                onChange={(e) => setVoiceMode(e.target.value)}
                style={{ marginRight: "0.5rem" }}
              />
              👹 Use Default Voice
            </label>

            <label style={{ display: "block" }}>
```

```
                <input
                  type="radio"
                  name="voiceMode"
                  value="custom"
                  checked={voiceMode === "custom"}
                  onChange={(e) => setVoiceMode(e.target.value)}
                  style={{ marginRight: "0.5rem" }}
                />
                👧 Use My Voice
              </label>
            </div>

            {voiceMode === "custom" && (
              <div>
                {!hasVoiceRecording ? (
                  <div>
                    <p style={{ margin: "0 0 0.5rem 0", fontSize: "0.9rem",
color: "#666" }}>
                      Record 5-10 seconds of you speaking clearly!
                    </p>

                    {!isRecording ? (
                      <button onClick={startVoiceRecording}
style={{ padding: "0.5rem 1rem", backgroundColor: "#ff6b6b", color:
"white", border: "none", borderRadius: "6px", cursor: "pointer", fontSize:
"0.9rem" }}>
                        🎤 Start Recording
                      </button>
                    ) : (
                      <div>
                        <button onClick={stopVoiceRecording}
style={{ padding: "0.5rem 1rem", backgroundColor: "#ff4444", color:
"white", border: "none", borderRadius: "6px", cursor: "pointer", fontSize:
"0.9rem", marginBottom: "0.5rem" }}>
          ⏹️ Stop Recording
                        </button>
```

```
                        <p style={{ margin: 0, fontSize: "0.9rem", color:
"#ff4444" }}>🔴 Recording...</p>

                      </div>

                    )}

                  </div>

                ) : (

                  <div>

                    <p style={{ margin: "0 0 0.5rem 0", fontSize: "0.9rem",
color: "#4a90e2" }}>✅ Your voice is ready!</p>

                    <button onClick={() => { setHasVoiceRecording(false);
setVoiceId(null); }} style={{ padding: "0.5rem 1rem", backgroundColor:
"#ffa500", color: "white", border: "none", borderRadius: "6px", cursor:
"pointer", fontSize: "0.9rem" }}>

                      🎤 Record New Voice

                    </button>

                  </div>

                )}

              </div>

            )}


            {voiceMode === "default" && (

              <p style={{ margin: 0, fontSize: "0.9rem", color:
"#666" }}>Stories will use computer voice.</p>

            )}

          </div>


          {/* Prompt Input */}

          <div style={{ marginBottom: "1rem" }}>

            <label>{story ? "Enter a new keyword or phrase:" : "Enter a
keyword or phrase (optional):"}</label>

            <input

              type="text"

              value={prompt}

              onChange={handlePromptChange}

              style={{ width: "100%", padding: "0.5rem", borderRadius:
"6px", border: "1px solid #ccc", marginTop: "0.5rem" }}
```

```
            />
          </div>


          {/* Style Preference Section */}
          <div style={{
            marginBottom: "1rem",
            padding: "1rem",
            backgroundColor: "rgba(74, 144, 226, 0.1)",
            borderRadius: "8px",
            border: "2px solid #4a90e2"
          }}>
            <h4 style={{ margin: "0 0 0.5rem 0", color: "#4a90e2" }}>🎨
Choose Your Art Style:</h4>
            <p style={{ margin: "0 0 1rem 0", fontSize: "0.9rem", color:
"#666" }}>
              How should the AI create your illustration?
            </p>

            <div style={{ display: "flex", gap: "0.5rem", flexWrap:
"wrap" }}>
              <button
                onClick={() => setStylePreference("keep-drawing")}
                style={{
                  flex: "1",
                  minWidth: "150px",
                  padding: "0.75rem",
                  backgroundColor: stylePreference === "keep-drawing" ?
"#ff6b6b" : "#f0f0f0",
                  color: stylePreference === "keep-drawing" ? "white" :
"#333",
                  border: stylePreference === "keep-drawing" ? "2px solid
#ff4444" : "2px solid #ddd",
                  borderRadius: "8px",
                  cursor: "pointer",
                  fontSize: "0.9rem",
```

```
                textAlign: "center",
                transition: "all 0.2s"
              }}
            >
              🖍️ Keep my drawing
              <br />
              <span style={{ fontSize: "0.8rem", opacity: 0.8 }}>
                (Looks exactly like what I drew)
              </span>
            </button>

            <button
              onClick={() => setStylePreference("make-prettier")}
              style={{
                flex: "1",
                minWidth: "150px",
                padding: "0.75rem",
                backgroundColor: stylePreference === "make-prettier" ?
"#4a90e2" : "#f0f0f0",
                color: stylePreference === "make-prettier" ? "white" :
"#333",
                border: stylePreference === "make-prettier" ? "2px solid
#3a7bc8" : "2px solid #ddd",
                borderRadius: "8px",
                cursor: "pointer",
                fontSize: "0.9rem",
                textAlign: "center",
                transition: "all 0.2s"
              }}
            >
              🎨 Make it prettier
              <br />
              <span style={{ fontSize: "0.8rem", opacity: 0.8 }}>
                (Mix of my drawing + pretty art)
```

```
                </span>
              </button>

              <button
                onClick={() => setStylePreference("surprise-me")}
                style={{
                  flex: "1",
                  minWidth: "150px",
                  padding: "0.75rem",
                  backgroundColor: stylePreference === "surprise-me" ?
"#20b2aa" : "#f0f0f0",
                  color: stylePreference === "surprise-me" ? "white" :
"#333",
                  border: stylePreference === "surprise-me" ? "2px solid
#1a9688" : "2px solid #ddd",
                  borderRadius: "8px",
                  cursor: "pointer",
                  fontSize: "0.9rem",
                  textAlign: "center",
                  transition: "all 0.2s"
                }}
              >
                ✨ Surprise me!
                <br />
                <span style={{ fontSize: "0.8rem", opacity: 0.8 }}>
                  (Professional storybook art)
                </span>
              </button>
            </div>
          </div>

          {/* Generate Button */}
          <button
            onClick={handleGenerate}
```

```
            style={{
              width: "100%",
              padding: "0.8rem",
              backgroundColor: "#4a90e2",
              color: "white",
              border: "none",
              borderRadius: "6px",
              fontSize: "1rem",
              cursor: "pointer",
              marginBottom: "1rem"
            }}
          >
            {story ? (useDrawingPad ? "✨ Generate Another Story" : "✨
Generate New Story") : "✨ Generate Story"}
          </button>

          {/* Create completely fresh story button */}
          {story && (
            <button
              onClick={() => {
                setStory("");
                setAiImageUrl("");
                clearAllDrawingState();
                setPrompt("");
                setAudioUrl(null);
                setIsPlaying(false);
                setIsPaused(false);
                setIsEditingDrawing(false);
              }}
              style={{
                width: "100%",
                padding: "0.8rem",
                backgroundColor: "#20b2aa",
```

```
                color: "white",
                border: "none",
                borderRadius: "6px",
                fontSize: "1rem",
                cursor: "pointer"
              }}
            >
              🎨 Start Completely Fresh
            </button>
          )}

          {/* Interactive loading */}
          {isGenerating && (
            <InteractiveWaitingComponent
              message="Creating your magical story"
              type="story"
            />
          )}
        </div>

        {/* Right Column - Generated Story */}
        {story && (
          <div style={{
            padding: "1rem",
            backgroundColor: "rgba(32, 178, 170, 0.1)",
            borderRadius: "12px",
            border: "2px solid #20b2aa"
          }}>
            <div className="story-section">
              <h2 style={{ color: "#20b2aa", marginTop: 0 }}>📖 Your
Story</h2>
              <p style={{ lineHeight: "1.6", marginBottom: "1.5rem",
fontSize: "1.1rem" }}>{story}</p>
```

```
              <div style={{ marginBottom: "1rem", fontSize: "0.9rem",
color: "#666" }}>

                Voice Mode: {voiceMode === "custom" && hasVoiceRecording ?
"🎤 Your Voice" : "😈 Default Voice"}

              </div>



              {/* Audio Controls */}

              <div style={{ marginBottom: "1.5rem" }}>

                {!audioUrl && !isGeneratingAudio && (

                  <button onClick={generateAudio} style={{ marginRight:
"8px", marginBottom: "8px", padding: "0.5rem 1rem", backgroundColor:
"#20b2aa", color: "white", border: "none", borderRadius: "6px", cursor:
"pointer" }}>

                    🎵 Generate Audio

                  </button>

                )}



                {isGeneratingAudio && (

                  <InteractiveWaitingComponent

                    message="Creating your story voice"

                    type="audio"

                  />

                )}



                {audioUrl && (

                  <>

                    <button onClick={handleReadStory} disabled={isPlaying}
style={{ marginRight: "8px", marginBottom: "8px", padding: "0.5rem 1rem",
backgroundColor: isPlaying ? "#ccc" : "#20b2aa", color: "white", border:
"none", borderRadius: "6px", cursor: isPlaying ? "not-allowed" :
"pointer" }}>

                      🔊 {isPlaying ? "Playing..." : "Read Aloud"}

                    </button>



                    <button onClick={handlePauseAudio}
disabled={!isPlaying} style={{ marginRight: "8px", marginBottom: "8px",
padding: "0.5rem 1rem", backgroundColor: !isPlaying ? "#ccc" : "#ffa500",
color: "white", border: "none", borderRadius: "6px", cursor: !isPlaying ?
"not-allowed" : "pointer" }}>
```

```
          ⏸ Pause
                    </button>


                    <button onClick={handleResumeAudio}
style={{ marginRight: "8px", marginBottom: "8px", padding: "0.5rem 1rem",
backgroundColor: "#4a90e2", color: "white", border: "none", borderRadius:
"6px", cursor: "pointer" }}>
          ▶ Resume
                    </button>


                    <button onClick={handleStopAudio}
style={{ marginBottom: "8px", padding: "0.5rem 1rem", backgroundColor:
"#ff6666", color: "white", border: "none", borderRadius: "6px", cursor:
"pointer" }}>
          ⏹ Stop
                    </button>
                  </>
                )}
              </div>


              {audioUrl && (
                <audio
                  ref={audioRef}
                  src={audioUrl}
                  onEnded={handleAudioEnded}
                  onError={handleAudioError}
                  style={{ display: "none" }}
                />
              )}
            </div>


            {aiImageUrl && (
              <div className="image-section">
                <h2 style={{ color: "#20b2aa" }}>🎨 Magic
Illustration</h2>
                <img
```

```
                    src={aiImageUrl}
                    alt="AI Illustration"
                    style={{
                      width: "100%",
                      borderRadius: "8px",
                      boxShadow: "0 2px 8px rgba(0, 0, 0, 0.1)"
                    }}
                  />
                </div>
              )}
            </div>
          )}
        </div>
      </div>
    );
}
```

** index.html **

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
    <title>Little Legends</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

** END **